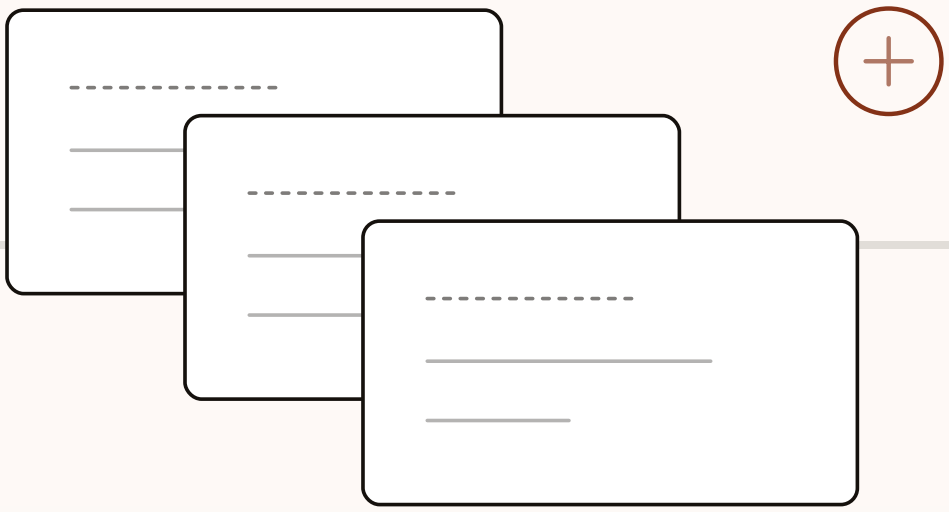
A FIELD GUIDE · 17 CHAPTERS

Building your own component system, *from first principles.*

REACT 19 · TYPESCRIPT · TAILWIND CSS 4 · REACT ARIA



JOSHUA BRILEY

RUDIMENT UI · FIRST EDITION



Building Your Own Component System

From First Principles

JOSHUA BRILEY

Building Your Own Component System: From First Principles

Copyright © 2026 Joshua Briley. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except for brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

This copy is licensed to a single reader. Sharing, reselling, or redistributing the file is not permitted.

The code listings in this book are released separately under the license that accompanies the companion repository, and you may use them in your own projects on those terms.

First edition, August 2026.

Published by the author.
Web edition: rudiment-guide.netlify.app
Companion repository: github.com/Rudiment-UI/rudiment-ui-components

The information in this book is provided as is, without warranty. Neither the author nor any distributor is liable for damages arising from its use. Build tooling and package ecosystems change; each chapter records the versions it was written against.

Product names, logos, and trademarks belong to their respective owners and appear here for identification only. Their use does not imply endorsement.

Set in Inter, Instrument Serif, and IBM Plex Mono.

About the author

I'm a design- and front-end-focused product design engineer with 20 years in production UI. The interface is where I bring the most value: design-grade, accessible, with the judgment most teams have to hire a separate designer to get.

I work across the whole product, from the stakeholder interviews that decide what to build through to the data model the interface has to keep its promises against. Most of that work has been design systems and data-dense product UI. I'm currently a software engineer on a shared component library. Before that: senior UI developer at LogATot, senior software engineer and lead UI developer at America's Test Kitchen, and principal design engineer at Berkshire Hathaway Specialty Insurance — portals, admin tools, tables, filters, and long forms for people who sit in them all day. I've taken contract work continuously since 2010, and I studied biology at Troy University before any of it.

My attention goes to the parts of an interface usually left for later: keyboard paths, focus states, the empty and error screens, and the accessibility behavior that only surfaces when someone actually needs it. That bias is why this book treats React Aria and axe-core as starting equipment rather than a pass to make at the end. I follow the WAI-ARIA Authoring Practices, the Design Tokens Community Group, and the CSS Working Group, because a component library that ignores where the platform is going ages badly.

Rudiment UI, the library this book builds toward, is my open-source project.

> *Josh was always a great design and research partner... He always worked hard to ensure accessibility across our experience.*
>
> Drew DiPasquale, Principal Researcher, HubSpot

> *Working with Josh was an absolute privilege and pleasure. His ability to foresee challenges and simplify implementation was invaluable.*
>
> Rebecca Cachia, PMP, Web Product & Project Manager, Publicis Sapient

> *Josh... consistently delivered exceptional work. Even with a high bar set for timelines and quality, Josh met or exceeded expectations every time.*
>
> John Miller, CEO, Logatot, Inc.

Contents

Introduction: Build your own component library 1

About this guide 2

Who this guide is for 2

How to get the most from this guide 2

Series overview 3

1 The architecture before the code 5

Choosing your primitives layer 6

Choosing your styling approach 7

The three-tier token architecture 7

Layout as a first-class concern 8

Decisions summary 9

2 Project setup 11

Initialize the project 12

Configure TypeScript 12

Configure Vite 13

Set up the directory structure 13

Create the utility function 14

Verify the setup 15

3 Design tokens from scratch 17

Install Style Dictionary 18

Create the token source file 18

Add semantic tokens 21

Add component tokens 23

Configure Style Dictionary 25

Wire tokens into your CSS 27

Verify the pipeline 28

4 Your first layout primitive: Stack 31

The problem 32

The solution 32

Building the component 33

The CSS 35

The recursive prop 36

The splitAfter prop 37

The public export 37

Writing the stories 38

Writing the tests 41

5 The rest of the layout system 45

Box 46

Center 48

Cluster 50

Sidebar 52

Switcher 54

Grid 56

Cover 58

Composing primitives 60

Attribution 61

6 Typography: Heading, Text, and Prose 63

Typography tokens 64

Heading 67

Text 69

Prose 72

Usage 76

7 Accessible components with React Aria 77

Installing React Aria 78

How React Aria hooks work 78

Building Button 79

Button CSS 82

Building Input 85

IconButton 87

8 Form components 91

Select 92

Checkbox and CheckboxGroup 97

RadioGroup 99

Switch 102

9 Overlays and feedback 105

Dialog 106

Tooltip 108

Alert 110

Badge 112

Card 113

10 Storybook as your documentation layer 117

Installation 118

Configuration 118

Story organization 123

What every story file includes 124

The example pages 125

Deploying Storybook 126

11 Testing accessible components 129

Install dependencies 130

Test configuration 130

The two categories of tests 133
Testing layout primitives 133
Testing UI components 134
Running tests 137
Coverage targets 137
12 Packaging and distribution 139
Configure Vite library mode 140
Configure package.json exports 142
Build and verify 144
Apply semantic versioning 144
13 Theming: eight themes from one pipeline 147
What a theme actually is 148
Why an attribute rather than a media query 148
The theme set 149
One config per theme 150
Adding a theme 153
The specificity trap 154
Wiring the toolbar 155
A note on fonts 156
14 The component set beyond the core 159
Menu 160
Icon 161
Text and content 162
Status and indicators 164
Navigation 165
Structure 167
The unexported five 167

15 Data visualization 169

Choosing a charting library 170

The theme bridge 170

The chart components 173

What charts do not solve 175

16 Drag and drop that works from the keyboard 177

Choosing a library 178

The API 178

Sensors: where accessibility is decided 179

Announcing what happened 180

Detecting the destination 182

The drag overlay 183

Testing it 184

17 What comes next 187

Where the gaps are 188

Keeping Figma and code aligned 189

Governance, and the word "design system" 190

Where to go from here 191

Index 193

INTRODUCTION

Build your own component library

A step-by-step guide to building an accessible, token-driven React component library from an empty directory to a deployed Storybook.

About this guide

This guide walks you through building a React component library from scratch. By the end, you'll have a working system with design tokens, accessible UI components, intrinsic layout primitives, Storybook documentation, and tests.

The guide is opinionated. It makes specific technology choices and explains why. Where reasonable alternatives exist, they are noted, but the guide doesn't try to cover every possible approach. Covering every approach is how guides become unusable.

The architecture follows the same patterns used by libraries like HeroUI and Untitled UI React.

Who this guide is for

This guide assumes you're comfortable with:

- **React** — You understand components, props, hooks, and the component lifecycle. You've built at least one project in React and aren't relying on this guide to learn React fundamentals.
- **TypeScript** — You can read and write typed interfaces, generics, and function signatures. You don't need deep expertise, but you shouldn't be encountering TypeScript for the first time.
- **CSS** — You understand the box model, flexbox, and responsive design. Familiarity with CSS custom properties is helpful but not required.
- **The terminal** — You're comfortable running commands, navigating directories, and managing packages with npm or a similar tool.
- **Node.js** — You have Node 18 or later installed. If you're unsure, run `node -v` in your terminal.

You don't need prior experience with Tailwind CSS, Storybook, design tokens, or accessibility tooling. The guide introduces each of these as they come up.

How to get the most from this guide

Work through chapters in order. Chapters 1 through 3 establish the foundation, the architecture decisions and project setup that everything else builds on. Chapters 4 through 12 are more modular, but they reference the token system and file struc-

ture from the earlier chapters. Chapters 13 through 16 extend the finished library and can be read in any order, or skipped until you need them.

Type the code, don't copy it. Pasting code lets you get to the end faster, but typing it forces you to read each line. The goal is understanding the system, not just having a working repo.

Read the "why" sections. This guide explains the reasoning behind each choice. Those explanations are where the transferable knowledge lives. If you skip them, you'll have a library but not the judgment to extend it.

Don't skip Chapter 1. It contains no code. It's also the most important chapter. The decisions made there shape every chapter that follows.

Expect things to break. Build tools and package ecosystems change. If something doesn't work, check the version numbers first, then search for the error message. Each chapter specifies the versions it was written against.

Use the companion repository as a reference. A complete implementation of the library built in this guide is available at GitHub. If you get stuck or want to compare your output against a known-good state, that repository is the reference: the component names, prop signatures, token names, and class names in this guide all match what it ships. It also contains more than the guide covers, and a few things the guide points out as gaps.

Series overview

Chapter	Title	What you'll build
1	The architecture before the code	Nothing (decisions only)
2	Project setup	Empty repo with Vite, React 19, TypeScript, Tailwind CSS 4
3	Design tokens from scratch	Three-tier token system with Style Dictionary
4	Your first layout primitive: Stack	Stack component with token-driven spacing
5	The rest of the layout system	Box, Center, Cluster, Sidebar, Switcher, Grid, Cover
6	Typography: Heading, Text, and Prose	Three typography components with long-form content rhythm
7	Accessible components with React Aria	Button, IconButton, and Input with full keyboard and ARIA support
8	Form components	Select, Checkbox, RadioGroup, Switch
9	Overlays and feedback	Dialog, Tooltip, Alert, Badge, Card

Chapter	Title	What you'll build
10	Storybook as your documentation layer	Configuration, story conventions, example pages
11	Testing accessible components	Vitest, Testing Library, axe-core patterns
12	Packaging and distribution	Vite library mode, npm publishing, versioning
13	Theming: eight themes from one pipeline	Per-theme token builds switched by `data-theme`
14	The component set beyond the core	Menu, navigation, status, and content components
15	Data visualization	Bar, line, and donut charts that follow the theme
16	Drag and drop that works from the keyboard	A Kanban board built on `@dnd-kit`
17	What comes next	Remaining gaps, Figma alignment, governance

CHAPTER 1

The architecture before the code

This chapter covers the decisions you need to make before you write a single line of code.



Don't open your terminal yet. The most expensive mistakes in a component library are architectural, and they compound. A bad token structure means renaming hundreds of references later. A wrong CSS strategy means rewriting every component's styles. A missing accessibility layer means adding it retroactively, which always costs more than building it in.

Choosing your primitives layer

The accessibility primitives layer handles keyboard navigation, focus management, ARIA attributes, and screen reader behavior for interactive components like buttons, modals, selects, and tabs.

You have three options:

Build it yourself. This means implementing WAI-ARIA patterns from scratch for every interactive component. A correct modal implementation alone requires focus trapping, scroll locking, Escape key handling, focus restoration on close, and `aria-modal` management. Multiply that by every interactive component in your library. Unless accessibility engineering is your core skill, this path produces components that work for mouse users and break for everyone else.

Use Radix primitives. Radix provides unstyled, accessible components that you style yourself. It's the layer underneath shadcn/ui, and it has the largest mindshare in the React ecosystem. Radix was originally built by Modulz and is now maintained by WorkOS. It's a mature, production-proven library. This guide uses React Aria instead of Radix, not because Radix is a bad choice, but because React Aria offers deeper accessibility coverage out of the box, particularly for mobile screen readers and internationalization. That translates directly to less custom work per component.

Use React Aria. React Aria is maintained by Adobe's design system team, the same team behind the Spectrum design system. HeroUI (formerly NextUI) has built on React Aria since v2. Untitled UI React, launched in 2026, also chose React Aria. React Aria handles mobile screen reader interactions, press event normalization across pointer types, and internationalization of ARIA labels.

This guide uses React Aria because it gives you the most accessibility coverage with the least custom work, backed by a maintenance commitment tied to Adobe's own production needs.

Choosing your styling approach

The styling layer determines how your token values reach the rendered components and how adopters customize the visual output.

This guide styles components with **plain CSS class names reading CSS custom properties**. Each component owns a stylesheet next to it, and every value in that stylesheet is a `var()` pointing at a token. There is no CSS-in-JS runtime, no utility class generation, and no build step between the token and the rule.

The decision that matters here is a negative one: the published package does not ship Tailwind. That deserves an explanation, because Tailwind is the obvious choice and this guide still installs it.

Tailwind CSS 4's `@theme` directive is genuinely good. It maps custom properties to utility classes with no JavaScript config, and for an application it is hard to beat. A component library is not an application. Importing Tailwind ships its preflight, a global reset that restyles the consumer's headings, lists, tables, and form controls the moment they import your CSS. Consumers already using Tailwind have their own preflight and now have two. Consumers who are not never asked for one. Either way the library has reached outside its own components and changed the page.

So Tailwind stays, scoped to development. Storybook loads it, the documentation pages and example compositions use it, and none of it reaches the package. Chapter 3 sets up the two CSS entry points that enforce the split.

The cost is real: you write stylesheets instead of utility strings, and that is slower. What you get is a package that cannot restyle the application that installs it.

CSS Modules, Sass, and styled-components all work too. Sass and CSS Modules add a compile step between the token and the rule for no benefit once tokens are already custom properties. styled-components adds a runtime and a peer dependency that consumers inherit.

The three-tier token architecture

If you've worked with design tokens before, you've probably seen systems that define a color palette and reference those colors directly in components. That works until you need a dark theme, a rebrand, or a second product that shares components but looks different.

A three-tier architecture avoids this by separating *what values exist* from *what values mean* from *what values a specific component uses*.

Tier 1 (global tokens) defines the raw palette. `color.blue.500` is `#3B82F6`. It has no opinion about what that color is for. It just exists as a named value in the system.

Tier 2 (semantic tokens) assigns meaning. `color.brand.primary` references `color.blue.500`. Now `color.blue.500` is aliased as the primary brand color. When the brand changes to teal, you update one alias. Every component that uses `color.brand.primary` changes with it.

Tier 3 (component tokens) scopes values to individual components. `component.button.primary-bg` references `color.brand.primary`. A button needs padding at three sizes, a font size at three sizes, a radius, an icon gap, a focus outline color, and background, text, hover, and active colors for each of four variants. None of those are statements about the system; they are statements about a button. Left in tier 2 they crowd the semantic namespace and make every component's values reachable by every other component.

Plenty of guides call this tier optional. It is optional right up until you have more than a handful of components, at which point retrofitting it means touching every stylesheet you have written. This guide builds it from the start.

This guide implements all three tiers so you understand how they work. The token schema uses the Design Tokens Community Group (DTCG) format, the leading interchange format for design tokens. The DTCG specification reached its first stable release (2025.10) in October 2025, with support from more than 10 tools including Figma, Style Dictionary, Tokens Studio, and Penpot. It is a W3C Community Group specification, not a W3C Recommendation, but it is production-ready and widely adopted. Style Dictionary v4 ships with first-class DTCG support.

Layout as a first-class concern

Most component libraries stop at UI components: buttons, inputs, modals, cards. They leave layout to the consumer, who either writes custom CSS for every page or copies page templates and strips out what they don't need.

This guide treats layout as part of the component library. You'll build eight layout primitives (`Stack`, `Box`, `Center`, `Cluster`, `Sidebar`, `Switcher`, `Grid`, `Cover`) that follow the intrinsic design principles described by Heydon Pickering and Andy

Bell in Every Layout. These primitives respond to their container's available space rather than the viewport width, which means they work in any context without media query breakpoints.

The layout primitives are driven by the same token system as the UI components. Changing `layout.sidebar.width` from `20rem` to `24rem` propagates to every Sidebar instance in the application. Changing `layout.stack.space.default` from `1.5rem` to `2rem` adjusts vertical rhythm globally.

Decisions summary

Before you proceed, here's what you're committing to:

Decision	Choice	Why
Framework	React 19 + TypeScript	Largest ecosystem; TypeScript makes component APIs self-documenting
Accessibility primitives	React Aria	Adobe-maintained; deepest a11y coverage; used by HeroUI and Untitled UI React
Component styling	Plain CSS classes + custom properties	Colocated per-component stylesheets reading `var(--rudi-*)`; no runtime, no preflight shipped to consumers
Docs styling	Tailwind CSS 4	Storybook and example pages only; never imported by the published package
Token format	DTCG JSON + Style Dictionary v4	Leading interchange format; platform-agnostic source; automated CSS custom property output. Style Dictionary v5 exists with a compatible config format; this guide uses v4
Documentation	Storybook 10 (installed via `@latest` — pin the version if reproducibility matters)	Industry standard; built-in a11y addon; ESM-only maintenance release that reduced install size by 29%
Testing	Vitest + Testing Library	Fast; behavior-focused; accessible query selectors
Build tool	Vite	Fast dev server; clean library build output
Layout approach	Intrinsic primitives (Every Layout-inspired)	Context-independent; no media queries; token-driven

If any of these choices don't fit your situation, substitute freely. The architecture still holds. The specific tools are less important than the separation of concerns: tokens define values, components consume tokens, layout primitives handle spatial arrangement, and tests verify behavior.

CHAPTER 2

Project setup

This chapter gets you from an empty directory to a working development environment with Vite, React 19, TypeScript, and Tailwind CSS 4. No components yet, just the foundation.



Before any token or component exists, the project needs a foundation that later chapters can build on without revisiting. That's what this chapter is: install dependencies, configure the toolchain, lay out the directory structure, and add the single utility — `cn()` — that every component will use to merge Tailwind classes. The order matters. Each step assumes the previous one, so work through them top to bottom.

Initialize the project

```bash
mkdir rudiment-ui && cd rudiment-ui
npm init -y
```

Install the core dependencies:

```bash
npm install react react-dom
npm install -D typescript @types/react @types/react-dom
npm install -D vite@^7.3.1 @vitejs/plugin-react@^5.2.0
npm install -D @tailwindcss/vite@^4.2.1 tailwindcss@^4.2.1
```

> **NOTE**
> As of this writing `@tailwindcss/vite@4.2.1` doesn't support `Vite 8` yet (only ^5.2.0 || ^6 || ^7). Install `Vite 7.3.1` to avoid compatibility issues. If you try to use Vite 8, you'll get an error about the plugin not being compatible.

Configure TypeScript

Create `tsconfig.json` at the project root:

```json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
```

```
    "declarationMap": true,
    "outDir": "dist",
    "rootDir": "src",
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

`strict: true` catches type errors early. `jsx: "react-jsx"` enables the automatic JSX runtime, so you don't need to import React in every file. `declaration: true` generates `.d.ts` files that consumers need for TypeScript support.

Configure Vite

Create `vite.config.ts`:

TYPESCRIPT

```typescript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import { resolve } from 'path'
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src'),
    },
  },
})
```

For now, this configuration covers the dev server for Storybook development. Chapter 11 adds the test configuration to this same file, and Chapter 12 adds the library build for npm distribution. All three live in `vite.config.ts`; there is no separate Vitest config.

Set up the directory structure

BASH

```bash
mkdir -p src/components src/layouts src/typography src/hooks src/utils
mkdir -p src/docs src/stories
mkdir -p tokens/build
```

```
mkdir -p .storybook
```

There is no `src/styles` directory. Each component owns a stylesheet that sits next to it, imported by the component module itself, so `src/components/Button/` will hold `Button.tsx` and `button.css` together. Chapter 4 explains why that colocation matters for the published build.

> **NOTE**
> The `.storybook` directory holds configuration for Storybook, which you install and set up in Chapter 10. `src/docs` holds the MDX documentation pages written in that chapter, and `src/stories` holds the example page compositions. Vitest arrives with the Storybook setup in Chapter 10; Testing Library in Chapter 11. Chapters throughout the guide add story and test files alongside each component, with notes explaining they won't run until those chapters are complete.

Now create the two CSS entry points. They are not interchangeable, and keeping them separate is what stops the library from restyling the applications that install it.

`src/styles.css` is what the published package ships:

```css
/* Library CSS entry. Deliberately does NOT import Tailwind. */
```

Leave it as a comment for now; chapter 3 adds the token import.

`src/app.css` is the development and documentation entry point, loaded by Storybook:

```css
@import 'tailwindcss';
```

That single line is all Tailwind CSS 4 needs to start. Tailwind is a development dependency here in every sense: it styles Storybook pages and example compositions, and it never reaches the package. Chapter 3 adds the token imports to both files.

Create the utility function

Every component needs a way to merge CSS classes. Create `src/utils/cn.ts`:

```typescript
```

```
import { type ClassValue, clsx } from 'clsx'
import { twMerge } from 'tailwind-merge'
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

Install the dependencies:

```bash
npm install clsx tailwind-merge
```

`clsx` handles conditional class joining. `tailwind-merge` resolves Tailwind class conflicts: for example, if a component applies `bg-blue-500` and the consumer passes `bg-red-500`, only the consumer's class survives. Together, they let consumers override any component style with Tailwind utilities without specificity conflicts.

Verify the setup

Create a minimal `src/index.ts`:

```typescript
import './styles.css'
export { cn } from './utils/cn'
```

That first line is a side-effect import, and it is the reason consumers get working styles from a single `import 'rudiment-ui'`. Every component stylesheet reaches the bundle the same way, through the component that owns it.

Run the TypeScript compiler to verify everything is wired up:

```bash
npx tsc --noEmit
```

If this passes with no errors, your foundation is solid.

What you have now

- A Vite project with React 19 and TypeScript in strict mode
- Tailwind CSS 4 integrated via the Vite plugin, scoped to development

- A directory structure that separates components, layouts, typography, hooks, utils, tokens, and docs
- Two CSS entry points: `src/styles.css` for the package, `src/app.css` for Storybook
- A `cn()` utility for class merging

No components, no tokens, no Storybook. That's intentional. The next chapter builds the token system, and every component you write after that references tokens from day one. If you build components first and add tokens later, you end up retrofitting: slower work and more inconsistencies.

CHAPTER 3

Design tokens from scratch

This chapter builds the token system that every component and layout primitive will reference.



This chapter builds the token system that every component and layout primitive will reference. By the end you'll have three token source files, a Style Dictionary pipeline that compiles them into namespaced CSS custom properties, and the two CSS entry points that decide what ships to consumers and what stays in development.

All three tiers from chapter 1 get built here, including the component tier that chapter called optional. It turns out not to be.

Install Style Dictionary

```bash
npm install -D style-dictionary@4
```

This guide uses Style Dictionary v4. Version 5 exists and uses a compatible configuration format, but the examples here use v4.

Create the token source file

Create `tokens/tokens.json`. This is the single source of truth for every visual and layout value in the system.

The file uses the DTCG format. Each token has a `$value` (the actual value) and a `$type` (the kind of value). Tokens can reference other tokens using curly braces: `{color.blue.500}`. The DTCG format is the community standard for design token interchange. Style Dictionary v4 includes first-class support for it.

Start with the global tokens (tier 1):

```json
{
  "color": {
    "blue": {
      "50": { "$value": "#EFF6FF", "$type": "color" },
      "100": { "$value": "#DBEAFE", "$type": "color" },
      "200": { "$value": "#BFDBFE", "$type": "color" },
      "300": { "$value": "#93C5FD", "$type": "color" },
      "400": { "$value": "#60A5FA", "$type": "color" },
      "500": { "$value": "#3B82F6", "$type": "color" },
      "600": { "$value": "#2563EB", "$type": "color" },
      "700": { "$value": "#1D4ED8", "$type": "color" },
      "800": { "$value": "#1E40AF", "$type": "color" },
      "900": { "$value": "#1E3A8A", "$type": "color" }
    },
```

```
    "neutral": {
      "0": { "$value": "#FFFFFF", "$type": "color" },
      "50": { "$value": "#FAFAFA", "$type": "color" },
      "100": { "$value": "#F5F5F5", "$type": "color" },
      "200": { "$value": "#E5E5E5", "$type": "color" },
      "300": { "$value": "#D4D4D4", "$type": "color" },
      "400": { "$value": "#A3A3A3", "$type": "color" },
      "500": { "$value": "#737373", "$type": "color" },
      "600": { "$value": "#525252", "$type": "color" },
      "700": { "$value": "#404040", "$type": "color" },
      "800": { "$value": "#262626", "$type": "color" },
      "900": { "$value": "#171717", "$type": "color" }
    },
    "red": {
      "50": { "$value": "#FEF2F2", "$type": "color" },
      "100": { "$value": "#FEE2E2", "$type": "color" },
      "500": { "$value": "#EF4444", "$type": "color" },
      "600": { "$value": "#DC2626", "$type": "color" },
      "700": { "$value": "#B91C1C", "$type": "color" },
      "800": { "$value": "#CB0A04", "$type": "color" }
    },
    "green": {
      "50": { "$value": "#F0FDF4", "$type": "color" },
      "100": { "$value": "#DCFCE7", "$type": "color" },
      "500": { "$value": "#22C55E", "$type": "color" },
      "600": { "$value": "#16A34A", "$type": "color" },
      "700": { "$value": "#15803D", "$type": "color" }
    },
    "amber": {
      "50": { "$value": "#FFFBEB", "$type": "color" },
      "100": { "$value": "#FEF3C7", "$type": "color" },
      "500": { "$value": "#F59E0B", "$type": "color" },
      "700": { "$value": "#B45309", "$type": "color" }
    },
    "dataviz": {
      "1": { "$value": "#2563EB", "$type": "color" },
      "2": { "$value": "#D946EF", "$type": "color" },
      "3": { "$value": "#F59E0B", "$type": "color" },
      "4": { "$value": "#10B981", "$type": "color" },
      "5": { "$value": "#EF4444", "$type": "color" },
      "6": { "$value": "#8B5CF6", "$type": "color" },
      "7": { "$value": "#06B6D4", "$type": "color" },
      "8": { "$value": "#F97316", "$type": "color" }
    }
  },
  "spacing": {
    "0": { "$value": "0", "$type": "dimension" },
    "1": { "$value": "0.25rem", "$type": "dimension" },
    "2": { "$value": "0.5rem", "$type": "dimension" },
    "3": { "$value": "0.75rem", "$type": "dimension" },
    "4": { "$value": "1rem", "$type": "dimension" },
```

```
    "6": { "$value": "1.5rem", "$type": "dimension" },
    "8": { "$value": "2rem", "$type": "dimension" },
    "12": { "$value": "3rem", "$type": "dimension" }
  },
  "radius": {
    "none": { "$value": "0", "$type": "dimension" },
    "sm": { "$value": "0.125rem", "$type": "dimension" },
    "md": { "$value": "0.375rem", "$type": "dimension" },
    "lg": { "$value": "0.5rem", "$type": "dimension" },
    "full": { "$value": "9999px", "$type": "dimension" }
  },
  "font": {
    "family": {
      "sans": {
        "$value": "'IBM Plex Sans', ui-sans-serif, system-ui, sans-serif",
        "$type": "fontFamily"
      },
      "mono": {
        "$value": "'IBM Plex Mono', ui-monospace, SFMono-Regular, Menlo, monospace",
        "$type": "fontFamily"
      }
    },
    "size": {
      "xs": { "$value": "0.75rem", "$type": "dimension" },
      "sm": { "$value": "0.875rem", "$type": "dimension" },
      "base": { "$value": "1rem", "$type": "dimension" },
      "lg": { "$value": "1.125rem", "$type": "dimension" },
      "xl": { "$value": "1.25rem", "$type": "dimension" },
      "2xl": { "$value": "1.5rem", "$type": "dimension" },
      "3xl": { "$value": "1.875rem", "$type": "dimension" },
      "4xl": { "$value": "2.25rem", "$type": "dimension" }
    },
    "weight": {
      "regular": { "$value": "400", "$type": "fontWeight" },
      "medium": { "$value": "500", "$type": "fontWeight" },
      "semibold": { "$value": "600", "$type": "fontWeight" },
      "bold": { "$value": "700", "$type": "fontWeight" }
    },
    "lineHeight": {
      "tight": { "$value": "1.25", "$type": "number" },
      "normal": { "$value": "1.5", "$type": "number" },
      "relaxed": { "$value": "1.75", "$type": "number" }
    }
  },
  "motion": {
    "duration": {
      "fast": { "$value": "100ms", "$type": "duration" },
      "base": { "$value": "150ms", "$type": "duration" },
      "slow": { "$value": "300ms", "$type": "duration" },
      "spinner": { "$value": "600ms", "$type": "duration" }
    },
```

```
    "easing": {
      "standard": { "$value": "cubic-bezier(0.4, 0, 0.2, 1)", "$type": "cubicBezier" }
    }
  }
}
```

The `dataviz` ramp exists so charts can assign a distinct, accessible color per series without reaching into the brand palette. The `motion` namespace does the same job for animation: every transition and spinner in the library reads a duration and easing token instead of hardcoding milliseconds, which means a theme can slow the whole system down or flatten it to near-instant in one place.

These are placeholder values. The structure matters more than the specific hex codes at this stage. Whoever adopts the library swaps these values to match their brand; the structure stays the same.

Add semantic tokens

Create a separate file for the semantic tokens at `tokens/semantic.json`. Style Dictionary merges all files listed in its `source` array, so keeping global and semantic tokens in separate files avoids an invalid JSON structure (two root objects in one file won't parse). The semantic tokens reference the global tokens by path:

JSON

```
{
  "color": {
    "brand": {
      "primary": { "$value": "{color.neutral.900}" },
      "primary-hover": { "$value": "{color.neutral.700}" },
      "primary-active": { "$value": "{color.neutral.800}" }
    },
    "text": {
      "default": { "$value": "{color.neutral.900}" },
      "subtle": { "$value": "{color.neutral.600}" },
      "disabled": { "$value": "{color.neutral.400}" },
      "on-brand": { "$value": "{color.neutral.0}" },
      "on-inverted": { "$value": "{color.neutral.0}" }
    },
    "background": {
      "surface-base": { "$value": "{color.neutral.0}" },
      "surface": { "$value": "{color.background.surface-base}" },
      "surface-raised": { "$value": "{color.neutral.50}" },
      "surface-sunken": { "$value": "{color.neutral.100}" },
      "surface-overlay": { "$value": "rgb(0 0 0 / 0.5)" },
      "disabled": { "$value": "{color.neutral.100}" },
      "overlay": { "$value": "{color.background.surface-overlay}" },
```

```
      "inverted": { "$value": "{color.neutral.900}" }
    },
    "border": {
      "default": { "$value": "{color.neutral.200}" },
      "focus": { "$value": "{color.blue.500}" },
      "error": { "$value": "{color.red.500}" }
    },
    "feedback": {
      "error": { "$value": "{color.red.500}" },
      "error-surface": { "$value": "{color.red.50}" },
      "error-border": { "$value": "{color.red.100}" },
      "error-text": { "$value": "{color.red.700}" },
      "success": { "$value": "{color.green.500}" },
      "success-surface": { "$value": "{color.green.50}" },
      "success-border": { "$value": "{color.green.100}" },
      "success-text": { "$value": "{color.green.700}" },
      "warning": { "$value": "{color.amber.500}" },
      "warning-surface": { "$value": "{color.amber.50}" },
      "warning-border": { "$value": "{color.amber.100}" },
      "warning-text": { "$value": "{color.amber.700}" },
      "info-surface": { "$value": "{color.blue.50}" },
      "info-border": { "$value": "{color.blue.100}" },
      "info-text": { "$value": "{color.blue.700}" }
    },
    "dataviz": {
      "series-1": { "$value": "{color.dataviz.1}" },
      "series-2": { "$value": "{color.dataviz.2}" },
      "series-3": { "$value": "{color.dataviz.3}" },
      "series-4": { "$value": "{color.dataviz.4}" },
      "series-5": { "$value": "{color.dataviz.5}" },
      "series-6": { "$value": "{color.dataviz.6}" },
      "series-7": { "$value": "{color.dataviz.7}" },
      "series-8": { "$value": "{color.dataviz.8}" }
    }
  }
}
```

Three things in that file are worth pausing on.

The brand color is a neutral, not a blue. That surprises people. A starter library that ships loud blue buttons forces every adopter to change them before shipping anything. A near-black primary looks deliberate on day one and gets replaced when the adopter has an actual brand. The palette still carries a full blue ramp for focus rings and informational states.

Each feedback color comes as a set of four, not one. `error` is the saturated stroke color, `error-surface` is the tinted background an alert sits on, `error-border` is

the hairline around it, and `error-text` is the darker value that stays legible on that tint. One value per state is enough for a border and nothing else.

Some semantic tokens alias other semantic tokens. `background.surface` points at `background.surface-base`, and `background.overlay` points at `background.surface-overlay`. The indirection lets a theme retarget the whole surface family at its base while leaving every component reference intact.

Note that this file contains only a `color` object. Spacing, radius, and typography values do not get a semantic tier. They go straight from tier 1 to the component tier.

Add component tokens

Chapter 1 described the third tier as the one you can defer. In practice you cannot, and it is worth understanding why before you write a component.

A button needs a horizontal padding at three sizes, a vertical padding at three sizes, a font size at three sizes, a radius, a gap between icon and label, a focus outline color, and a background, text, hover, and active color for each of four variants. None of those are semantic decisions about the system. They are decisions about a button. Putting them in the semantic tier means every future component competes for names in the same namespace, and changing a button's radius means changing a value that something else may also read.

Create `tokens/components.json`. It holds three namespaces: `component` for per-component values, `layout` for the layout primitives you build in chapters 4 and 5, and `typography` for the type scale you build in chapter 6.

Here is the shape, using the button as the example:

```json
{
  "component": {
    "button": {
      "gap": { "$value": "{spacing.2}" },
      "font-weight": { "$value": "{font.weight.medium}" },
      "radius": { "$value": "{radius.sm}" },
      "focus-outline-color": { "$value": "{color.border.focus}" },
      "padding-y-sm": { "$value": "{spacing.1}" },
      "padding-x-sm": { "$value": "{spacing.3}" },
      "padding-y-md": { "$value": "{spacing.2}" },
      "padding-x-md": { "$value": "{spacing.4}" },
      "font-size-sm": { "$value": "{font.size.sm}" },
      "font-size-md": { "$value": "{font.size.sm}" },
      "primary-bg": { "$value": "{color.brand.primary}" },
```

```
      "primary-text": { "$value": "{color.text.on-brand}" },
      "primary-hover-bg": { "$value": "{color.brand.primary-hover}" }
    }
  },
  "layout": {
    "stack": {
      "space": {
        "default": { "$value": "{spacing.6}" },
        "dense": { "$value": "{spacing.3}" },
        "loose": { "$value": "{spacing.12}" }
      }
    },
    "sidebar": {
      "width": { "$value": "20rem" },
      "content-min": { "$value": "50%" },
      "space": { "$value": "{layout.stack.space.default}" }
    },
    "grid": {
      "min-cell": { "$value": "15rem" },
      "space": { "$value": "{layout.stack.space.default}" }
    },
    "center": {
      "max-width": { "$value": "60rem" },
      "gutters": { "$value": "{spacing.4}" }
    },
    "cover": {
      "min-height": { "$value": "100vh" },
      "space": { "$value": "{layout.stack.space.default}" }
    },
    "switcher": {
      "threshold": { "$value": "30rem" },
      "space": { "$value": "{layout.stack.space.default}" }
    },
    "cluster": {
      "space": { "$value": "{spacing.4}" }
    },
    "box": {
      "padding": { "$value": "{spacing.4}" },
      "border-color": { "$value": "{color.border.default}" },
      "border-radius": { "$value": "{radius.md}" },
      "invert-bg": { "$value": "{color.background.inverted}" },
      "invert-text": { "$value": "{color.text.on-inverted}" }
    }
  }
}
```

Look at the `space` entries under `sidebar`, `grid`, `cover`, and `switcher`. Each one aliases `layout.stack.space.default` rather than reading it directly. Out of the box every primitive shares the same rhythm, which is what you want. But a theme that

wants roomier sidebars without touching global vertical rhythm has a single value to change. Pointing all four primitives at the Stack token directly would take that option away.

The same reasoning applies to `box.border-color`. It aliases `color.border.default`, so Box borders match every other border by default and can be retargeted alone.

Chapter 6 adds the `typography` namespace to this file. Chapters 7 through 9 add the remaining `component` entries as each component is built. By the end the file carries entries for 36 components.

Configure Style Dictionary

Create `tokens/style-dictionary.config.mjs`:

```javascript
export default {
  source: [
    'tokens/tokens.json',
    'tokens/semantic.json',
    'tokens/components.json',
  ],
  platforms: {
    css: {
      transformGroup: 'css',
      prefix: 'rudi',
      buildPath: 'tokens/build/',
      files: [
        {
          destination: 'tokens.css',
          format: 'css/variables',
          options: {
            outputReferences: true,
          },
        },
      ],
    },
  },
}
```

The `.mjs` extension makes the ES module syntax work regardless of what the nearest `package.json` says about module type.

> **NOTE**
> List the source files explicitly. A glob like `tokens/**/*.json` looks tidier and works fine today, but it will also match the theme override files you add in chapter 13. Those files exist to be built separately, one theme at a time. Sweeping them into the base build merges every theme's overrides into `:root`, and the failure is quiet: the CSS still compiles, the colors are just wrong.

The `prefix: 'rudi'` option puts a `--rudi-` namespace on every generated property. Consumers drop this library into applications that already have their own custom properties, and `--color-text-default` is a name somebody else has probably used. `--rudi-color-text-default` is not.

The `outputReferences: true` option preserves the alias relationships in the output. Instead of resolving `color.brand.primary` to its final hex value, you get:

```css
--rudi-color-brand-primary: var(--rudi-color-neutral-900);
```

This matters for debugging. When you inspect an element in browser dev tools, you can trace a value back through its aliases to understand why it resolves the way it does. It also matters for theming: a theme that overrides `--rudi-color-neutral-900` moves the brand color with it, without having to know that the brand token exists.

Add a build script to `package.json`, and verify that `type: "module"` is set at the root level:

```json
{
  "type": "module",
  "scripts": {
    "build:tokens": "style-dictionary build --config tokens/style-dictionary.config.mjs"
  }
}
```

Run it:

```bash
npm run build:tokens
```

This generates `tokens/build/tokens.css`, containing every token as a CSS custom property under `:root`. Chapter 13 adds seven more configurations to this script, one per theme, each writing its own file.

> **NOTE**
> Commit `tokens/build/`. Generated files usually belong in `.gitignore`, but these ship in the published package and every component depends on them. Committing them means a fresh clone can run tests and Storybook without a build step, and it makes token changes visible in code review as a diff of actual CSS values.

Wire tokens into your CSS

The library needs two CSS entry points, and the difference between them is the most consequential decision in this chapter.

Create `src/styles.css`. This is what the published package ships:

```css
/**
 * Library CSS entry — imported by src/index.ts so the published build ships
 * token definitions alongside the per-component stylesheets.
 *
 * Deliberately does NOT import Tailwind. Components use plain class names and
 * custom properties only, so pulling in Tailwind would ship a preflight reset
 * that clobbers consumer styles for no benefit.
 */
@import '../tokens/build/tokens.css';
```

Then create `src/app.css`. This is the development and documentation entry point, loaded by Storybook in chapter 10:

```css
@import 'tailwindcss';
@import '../tokens/build/tokens.css';
```

Chapter 2 installed Tailwind, and it earns its place here: the Storybook documentation pages and the example compositions are ordinary application code, and writing them with utilities is faster than writing a stylesheet for each one. What Tailwind does not do is style the components themselves.

That is a deliberate constraint, and it costs something. You give up authoring components as utility strings, which many React developers find quicker. What you get back is a package that cannot break the application that installs it. Tailwind's preflight is a global reset; a component library that ships one silently restyles the consumer's headings, lists, and form controls. Consumers who use Tailwind already have their own preflight, and consumers who do not never asked for one.

So components read tokens directly:

```css
.rudi-button--primary {
  background-color: var(--rudi-component-button-primary-bg);
  color: var(--rudi-component-button-primary-text);
}
```

No `@theme` block, no utility class generation, no build step between the token and the style. The chain is three hops and every hop is inspectable in dev tools: `--rudi-component-button-primary-bg` resolves to `--rudi-color-brand-primary`, which resolves to `--rudi-color-neutral-900`, which resolves to `#171717`.

Verify the pipeline

Confirm the pipeline works end to end:

1. Change `color.neutral.900` in `tokens/tokens.json` to a different hex value.
2. Run `npm run build:tokens`.
3. Open `tokens/build/tokens.css` and confirm `--rudi-color-neutral-900` reflects the new value.
4. Confirm that `--rudi-color-brand-primary` still reads `var(--rudi-color-neutral-900)` rather than a resolved hex. That alias is what makes one edit propagate.

Edit a token, run the build, and every custom property that references it updates. One source, automated all the way through.

What you have now

- `tokens/tokens.json` with the raw palette, spacing, radius, type, and motion values
- `tokens/semantic.json` with color meanings, including the four-value feedback sets
- `tokens/components.json` with the `component` and `layout` namespaces
- A Style Dictionary config that merges all three into `tokens/build/tokens.css` under a `--rudi-` prefix, with aliases preserved
- Two CSS entry points: `src/styles.css` for the package, `src/app.css` for Storybook

The next chapter uses these tokens to build your first layout primitive.

CHAPTER 4

Your first layout primitive: Stack

The Stack is the most frequently used layout primitive you'll build.



The Stack is the most frequently used layout primitive you'll build. It solves a problem so fundamental that most developers don't realize they're working around it every day: how to apply consistent vertical spacing between elements without creating orphaned margins.

The problem

Consider a typical approach to vertical spacing:

```css
p {
  margin-bottom: 1.5rem;
}
```

This creates a margin below every paragraph. When a paragraph is followed by another element, the spacing looks right. But the last paragraph in a container gets a margin that combines with the container's padding, producing double the intended space at the bottom. The margin exists regardless of context, because the style belongs to the element rather than to the relationship between elements.

You end up writing `:last-child` overrides, or using a utility class to remove the bottom margin, or wrapping elements in containers that cancel out the extra space. These are all workarounds for the same root problem: styling elements individually when spacing is a property of adjacency.

The solution

The Stack primitive styles the *context* rather than the individual elements. It applies margin only between adjacent siblings, using a pattern known as the owl selector:

```css
.rudi-stack > * + * {
  margin-block-start: var(--stack-space);
}
```

The `* + *` selector targets any element that is immediately preceded by another element. The `>` child combinator scopes it to direct children only. The result: spacing appears between elements, but never before the first child or after the last child. No `:last-child` overrides needed.

Building the component

Create `src/layouts/Stack/Stack.tsx`:

TSX

```tsx
// src/layouts/Stack/Stack.tsx
import { Children, cloneElement, forwardRef, isValidElement } from 'react'
import { cn } from '@/utils/cn'
import './stack.css'
export interface RudiStackProps extends React.HTMLAttributes<HTMLElement> {
  /** CSS spacing value. Defaults to --rudi-layout-stack-space-default token. */
  space?: string
  /** Apply spacing recursively to all nested elements, not just direct children. */
  recursive?: boolean
  /** 1-based index of the child after which to insert an auto margin, splitting the
   stack. */
  splitAfter?: number
  /** HTML element to render. */
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiStack = forwardRef<HTMLElement, RudiStackProps>(function RudiStack(
  {
    space,
    recursive = false,
    splitAfter,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (space) customProperties['--stack-space'] = space
  // Apply margin-block-end: auto to the splitAfter child via React.
  // CSS :nth-child() does not accept custom properties, so this must
  // be handled in the component rather than in the stylesheet.
  // Children.toArray filters null/false values so conditional children
  // do not shift the numeric index.
  const childArray = Children.toArray(children)
  const styledChildren = splitAfter
    ? childArray.map((child, index) => {
        if (index === splitAfter - 1 && isValidElement(child)) {
          return cloneElement(
            child as React.ReactElement<{ style?: React.CSSProperties }>,
            {
              style: {
                ...(child.props as { style?: React.CSSProperties }).style,
                marginBlockEnd: 'auto',
```

```
              },
            },
          )
        }
        return child
      })
    : children
  return (
    <Element
      ref={ref}
      className={cn(
        'rudi-stack',
        recursive && 'rudi-stack--recursive',
        splitAfter && 'rudi-stack--split',
        className,
      )}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {styledChildren}
    </Element>
  )
})
```

Three implementation details are worth understanding:

The component doesn't import or use React Aria. Layout primitives have no interactive behavior, so they don't need accessibility hooks. They're pure CSS layout wrapped in a React component for composability and token integration.

The `space` prop is optional. When omitted, no inline `--stack-space` property is set. The CSS class sets `--stack-space` to `var(--rudi-layout-stack-space-default, 1.5rem)`, so the token provides the value. When provided, the inline property overrides the class-level value. This pattern gives you token-driven defaults with per-instance overrides, and the CSS does all the work.

The `as` prop lets the consumer render a semantic element. `<RudiStack as="ul">` renders a `<ul>` instead of a `<div>`, which matters for accessibility when the Stack's children are list items.

The component is wrapped with `forwardRef`, exposing the underlying DOM node to consumers via a `ref`. This is necessary when the Stack is used as a measurement target: scroll containers, `IntersectionObserver` roots, or elements that need programmatic focus. The ref is typed as `HTMLElement` rather than a more specific type because the `as` prop can render any element. This is a deliberate trade-off: stricter typing (where the ref type matches the rendered element) requires a poly-

morphic ref pattern that TypeScript doesn't support natively without a manual generic wrapper function. For most use cases, `HTMLElement` is sufficient. If your project requires the fully typed polymorphic ref, the pattern is documented in the companion repository.

The CSS

Create `src/layouts/Stack/stack.css`, next to the component:

```css
/* Stack */
.rudi-stack {
  --stack-space: var(--rudi-layout-stack-space-default, 1.5rem);
  display: flex;
  flex-direction: column;
  justify-content: flex-start;
}
.rudi-stack > * {
  margin-block: 0;
}
.rudi-stack > * + * {
  margin-block-start: var(--stack-space);
}
/* Recursive: apply spacing at any nesting depth */
.rudi-stack--recursive * + * {
  margin-block-start: var(--stack-space);
}
/* Split: push elements after the nth child to the bottom */
.rudi-stack--split {
  min-block-size: 100%;
}
```

Note: the `splitAfter` behavior is applied via React, not pure CSS. CSS `:nth-child()` accepts selector math, not custom properties, so `:nth-child(var(--n))` is invalid. Instead, the Stack component applies an inline `margin-block-end: auto` style to the correct child element.

Notice what you are not doing: adding an `@import` to a global stylesheet. `Stack.tsx` already imports `./stack.css` directly, and that side-effect import is the only thing connecting the two files.

That choice is worth a paragraph, because it is the one structural decision in this chapter that is hard to reverse.

A single global `layouts.css` is simpler to reason about, and it is what most guides do. It also means a consumer who imports only `RudiStack` gets the CSS for all eight primitives, because a bundler cannot tell which rules in a shared file are reachable. Colocating the stylesheet with the component, and importing it from the component, gives the bundler a graph it can actually follow. Combined with `sideEffects: ["**/*.css"]` in `package.json` (chapter 12), unused component styles get dropped.

The cost is one extra file per component and no single place to read the whole system's CSS. That trade is worth making for a package other people install.

The first rule resets vertical margins on all direct children to zero, preventing any inherited margin from interfering. The second rule applies the stack spacing only between adjacent siblings. The reset-then-apply pattern is what eliminates the orphaned margin problem.

The `--stack-space` custom property has a double fallback: it first checks for a value set inline by the `space` prop, then falls back to the `--rudi-layout-stack-space-default` token, then to `1.5rem` as a hardcoded safety net. In practice, the token value always exists (Style Dictionary generates it), so the hardcoded fallback is just insurance.

With the component and CSS in place, here's the base Stack rendered from the companion repository's Storybook. The `Default` story shown below is the same one you'll define later in this chapter.



LIVE COMPONENT

Stacks in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-stack--default

The recursive prop

By default, Stack only spaces its direct children. The `>` combinator ensures nested elements aren't affected. But sometimes you want uniform spacing at every nesting level, for example when rendering Markdown content where you don't control the nesting structure.

The `recursive` variant removes the child combinator:

```css
.rudi-stack--recursive * + * {
```

```
  margin-block-start: var(--stack-space);
}
```

Use this sparingly. Recursive spacing affects every nested element, which means list items, nested divs, and other structures will all receive the same margin. For most layouts, nesting non-recursive Stacks with different spacing values gives you more precise control.

The splitAfter prop

Making the Stack a flexbox column context gives it one more capability: splitting. By applying `margin-block-end: auto` to a specific child, you push everything after that child to the bottom of the available space.

This is useful for card-like layouts where you want some content at the top and a button pinned to the bottom:

```tsx
<RudiStack splitAfter={2} style={{ minHeight: '300px' }}>
  <h3 className="text-lg">Card title</h3>
  <p>Card description that might vary in length.</p>
  <RudiButton>Action</RudiButton> {/* Pushed to the bottom */}
</RudiStack>
```

The `splitAfter` prop tells the Stack component which child to apply `margin-block-end: auto` to via `Children.toArray` and `cloneElement`. The Stack needs a defined height (or `min-height`) for the split to produce a visible gap. Without a height constraint, the flexbox column collapses to its content height and the auto margin has no space to distribute.

The public export

Add the component to the main entry point in `src/index.ts`, exporting from the module directly:

```typescript
import './styles.css'
export { cn } from './utils/cn'
export { RudiStack } from './layouts/Stack/Stack'
export type { RudiStackProps } from './layouts/Stack/Stack'
```

Two conventions are established here, and both hold for every component in the rest of the guide.

Every exported name carries the `Rudi` prefix. The internal function is `RudiStack`, the interface is `RudiStackProps`, and the file is still `Stack.tsx`. A component library exports into a namespace it does not control: applications have their own `Stack`, `Text`, `Card`, and `Switch`, and a bare `Stack` forces every consumer to write `import { Stack as UIStack }`. Prefixing costs six characters at each call site and removes the collision entirely. Pick your own prefix; just pick one before you have 50 components.

There are no per-component `index.ts` barrel files. `src/index.ts` reaches straight into `./layouts/Stack/Stack`. Barrel files feel tidier and add a layer of indirection that bundlers have to see through, which weakens tree-shaking for exactly the consumers who import one component. One entry point is enough.

Writing the stories

Create `src/layouts/Stack/Stack.stories.tsx`:

> NOTE
>
> `@storybook/react-vite` is not installed until Chapter 10. This story file is included here so you can see the full component API, but it won't run until you complete the Storybook setup in that chapter.

TSX

```tsx
import type { Meta, StoryObj } from '@storybook/react'
import { RudiStack } from './Stack'
const meta = {
  title: 'Layouts/Stack',
  component: RudiStack,
  tags: ['autodocs'],
  parameters: {
    docs: {
      description: {
        component:
          'Arranges children in a vertical stack with consistent spacing between each
   item. Supports recursive spacing into nested elements and split positioning.',
      },
    },
  },
  args: {
    recursive: false,
  },
```

```
  argTypes: {
    as: {
      control: 'text',
      description: "The HTML element to render (e.g., 'div', 'ul', 'ol')",
      table: { category: 'Content' },
    },
    children: {
      description: 'Content to display inside the stack',
      table: { category: 'Content' },
    },
    recursive: {
      control: 'boolean',
      description:
        'When enabled, applies spacing to all nested elements, not just direct children',
      table: { category: 'Appearance' },
    },
    space: {
      control: 'text',
      description:
        "Vertical gap between items using any CSS length value (e.g., '1.5rem')",
      table: { category: 'Layout' },
    },
    splitAfter: {
      control: 'number',
      description:
        'Pushes all items after this index to the bottom of the stack (1-based)',
      table: { category: 'Layout' },
    },
  },
} satisfies Meta<typeof RudiStack>
export default meta
type Story = StoryObj<typeof meta>
function Placeholder({ label }: { label: string }) {
  return (
    <div className="border border-border-default rounded-md p-4 bg-surface-raised">
      {label}
    </div>
  )
}
export const Default: Story = {
  render: (args) => (
    <RudiStack {...args}>
      <Placeholder label="First child" />
      <Placeholder label="Second child" />
      <Placeholder label="Third child" />
    </RudiStack>
  ),
}
export const Dense: Story = {
  args: { space: '0.5rem' },
  render: (args) => (
```

```
    <RudiStack {...args}>
      <Placeholder label="First" />
      <Placeholder label="Second" />
      <Placeholder label="Third" />
    </RudiStack>
  ),
}
export const Loose: Story = {
  args: { space: '3rem' },
  render: (args) => (
    <RudiStack {...args}>
      <Placeholder label="First" />
      <Placeholder label="Second" />
      <Placeholder label="Third" />
    </RudiStack>
  ),
}
export const SplitAfterSecond: Story = {
  args: { splitAfter: 2 },
  decorators: [
    (Story) => (
      <div style={{ height: '400px', border: '1px dashed #ccc' }}>
        <Story />
      </div>
    ),
  ],
  render: (args) => (
    <RudiStack {...args} style={{ height: '100%' }}>
      <Placeholder label="Top item 1" />
      <Placeholder label="Top item 2" />
      <Placeholder label="Pushed to bottom" />
    </RudiStack>
  ),
}
export const AsUnorderedList: Story = {
  render: () => (
    <RudiStack as="ul" space="0.75rem" role="list">
      <li>First item</li>
      <li>Second item</li>
      <li>Third item</li>
    </RudiStack>
  ),
}
```

Each story demonstrates a specific feature. `Default` shows the token-driven spacing. `Dense` and `Loose` show prop overrides. `SplitAfterSecond` shows the split behavior with a height-constrained container. `AsUnorderedList` demonstrates the `as` prop with semantic HTML.

Three details in the `meta` block are conventions, not decoration, and every story file in the rest of the guide repeats them.

`parameters.docs.description.component` is the paragraph that appears at the top of the generated documentation page. Without it the page opens with a bare component name and a props table, which tells a reader what the props are but never what the component is for.

Every `argTypes` entry carries a `description`. Storybook renders these in the props table, so this is where a consumer finds out that `splitAfter` is 1-based, or that `space` takes any CSS length rather than a size keyword. A props table without descriptions is a list of types.

Every `argTypes` entry also carries `table: { category }`. Chapter 10 pins the category order to Appearance, Content, Layout, State, Events, which groups the controls panel into something scannable instead of one alphabetical list. The categories only work if you assign them consistently, so assign them as you write each story rather than retrofitting later.

Writing the tests

Create `src/layouts/Stack/Stack.test.tsx`:

> **NOTE**
>
> `@testing-library/react` is not installed until Chapter 11. (`vitest` is installed earlier, as part of the Storybook setup in Chapter 10.) These tests are included here alongside the component so you can see what behavior is being verified, but they won't run until you complete the testing setup in Chapter 11.

TSX

```tsx
import { render, screen } from '@testing-library/react'
import { axe } from 'vitest-axe'
import { describe, it, expect } from 'vitest'
import { RudiStack } from './Stack'
describe('Stack', () => {
  it('renders children', () => {
    render(
      <RudiStack>
        <p>First</p>
        <p>Second</p>
      </RudiStack>,
    )
    expect(screen.getByText('First')).toBeInTheDocument()
    expect(screen.getByText('Second')).toBeInTheDocument()
```

```
})
it('applies the default class', () => {
  const { container } = render(
    <RudiStack>
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild).toHaveClass('rudi-stack')
})
it('merges a custom className', () => {
  const { container } = render(
    <RudiStack className="mt-4">
      <p>Child</p>
    </RudiStack>,
  )
  const el = container.firstChild as HTMLElement
  expect(el).toHaveClass('rudi-stack')
  expect(el).toHaveClass('mt-4')
})
it('sets --stack-space when the space prop is passed', () => {
  const { container } = render(
    <RudiStack space="2rem">
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild).toHaveStyle('--stack-space: 2rem')
})
it('does not set inline --stack-space when space prop is omitted', () => {
  const { container } = render(
    <RudiStack>
      <p>Child</p>
    </RudiStack>,
  )
  const style = (container.firstChild as HTMLElement).getAttribute('style')
  expect(style).toBeNull()
})
it('applies the recursive class when recursive is true', () => {
  const { container } = render(
    <RudiStack recursive>
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild).toHaveClass('rudi-stack--recursive')
})
it('does not apply the recursive class by default', () => {
  const { container } = render(
    <RudiStack>
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild).not.toHaveClass('rudi-stack--recursive')
```

```
})
it('applies the split class when splitAfter is passed', () => {
  const { container } = render(
    <RudiStack splitAfter={2}>
      <p>A</p>
      <p>B</p>
      <p>C</p>
    </RudiStack>,
  )
  expect(container.firstChild).toHaveClass('rudi-stack--split')
})
it('applies margin-block-end: auto to the splitAfter child', () => {
  const { container } = render(
    <RudiStack splitAfter={2}>
      <p>A</p>
      <p>B</p>
      <p>C</p>
    </RudiStack>,
  )
  const secondChild = container.firstChild?.childNodes[1] as HTMLElement
  expect(secondChild.style.marginBlockEnd).toBe('auto')
})
it('renders the correct element via the as prop', () => {
  const { container } = render(
    <RudiStack as="section">
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild?.nodeName).toBe('SECTION')
})
it('renders a div by default', () => {
  const { container } = render(
    <RudiStack>
      <p>Child</p>
    </RudiStack>,
  )
  expect(container.firstChild?.nodeName).toBe('DIV')
})
it('forwards additional HTML attributes', () => {
  const { container } = render(
    <RudiStack data-testid="my-stack" id="stack-1">
      <p>Child</p>
    </RudiStack>,
  )
  const el = container.firstChild as HTMLElement
  expect(el).toHaveAttribute('data-testid', 'my-stack')
  expect(el).toHaveAttribute('id', 'stack-1')
})
it('has no accessibility violations', async () => {
  const { container } = render(
    <RudiStack>
```

```
        <p>First</p>
        <p>Second</p>
      </RudiStack>,
    )
    expect(await axe(container)).toHaveNoViolations()
  })
})
```

That last test runs axe-core against the rendered output, and every component in this guide gets one. It needs `import { axe } from 'vitest-axe'` at the top of the file. Chapter 11 wires up the matcher that makes `toHaveNoViolations()` work; until then this one test fails while the rest pass.

The other tests verify props and behavior rather than rendered CSS, which is the browser's job. Several of them do assert on class names and custom properties, because those are the contract between the component and its stylesheet. A test that only queried by role would pass while `splitAfter` silently stopped emitting `rudi-stack--split`.

What you have now

One working layout primitive, fully documented and tested. The pattern you used here (token-driven CSS custom properties, prop-based overrides, the `as` prop, class-based modifiers, `cn()` merging) applies to every layout primitive and every UI component you build from here. The next chapter builds the remaining seven layout primitives using the same pattern.

CHAPTER 5

The rest of the layout system

You have the pattern down from the Stack chapter. This chapter applies that same pattern seven more times.



You have the pattern down from the Stack chapter: a React component that renders a semantic element with a CSS class and optional inline custom property overrides. The CSS does the layout work, tokens provide the defaults, and props allow per-instance control. This chapter applies that same pattern seven more times.

Each section covers the primitive's CSS, TypeScript interface, and a brief explanation of the layout technique. Stories follow the same conventions established for Stack, and the full story files are in the companion repository. This chapter focuses on the CSS techniques that make each primitive work.

> **NOTE**
> A note on the companion repository's test coverage, since it is easy to assume otherwise: of the eight layout primitives, only Stack and Switcher currently have test files. The other six ship with stories but no unit tests. Their accessibility coverage comes from the Storybook browser test project in chapter 11, which renders every story and runs axe against it. Writing the six missing test files is a good first contribution if you want one.

One convention carries through every prop in this chapter: spacing props take a concrete CSS length, not a size keyword. There is no `space="md"`. Pass `space="1.5rem"`, or better, pass a token reference like `space="var(--rudi-spacing-6)"` so the value still moves when the theme does.

Box

The Box is the simplest layout primitive. It applies consistent padding and an optional border. You might reasonably ask why this needs a component at all, when a consumer could write `padding: 1rem` or a utility class and be done. The answer is consistency: a Box guarantees that padding comes from the token system, so when the default container padding moves from `1rem` to `1.25rem`, every Box in the application moves with it. A hand-written `1rem` does not.

```css
.rudi-box {
  --box-padding: var(--rudi-layout-box-padding, 1rem);
  padding: var(--box-padding);
}
.rudi-box--bordered {
  border: 1px solid var(--rudi-layout-box-border-color);
  border-radius: var(--rudi-layout-box-border-radius);
}
```

```
.rudi-box--invert {
  background-color: var(--rudi-layout-box-invert-bg);
  color: var(--rudi-layout-box-invert-text);
}
```

Every value here is a `layout.box.*` token from `tokens/components.json`, and each one aliases a semantic token: `border-color` points at `{color.border.default}`, `invert-bg` at `{color.background.inverted}`, `invert-text` at `{color.text.on-inverted}`.

Reading the semantic tokens directly would work and would be shorter. The indirection is what lets a theme restyle Box borders without moving every other border in the system. It is the same argument as the button tokens in chapter 3, applied to layout.

Note that there are no comma fallbacks in these rules. Style Dictionary generates all four properties, so a fallback would only ever fire if the token build had not run, in which case the rest of the library is broken too.

TSX

```tsx
// src/layouts/Box/Box.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './box.css'
export interface RudiBoxProps extends React.HTMLAttributes<HTMLElement> {
  padding?: string
  bordered?: boolean
  invert?: boolean
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiBox = forwardRef<HTMLElement, RudiBoxProps>(function RudiBox(
  {
    padding,
    bordered = false,
    invert = false,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (padding) customProperties['--box-padding'] = padding
  return (
    <Element
```

```
      ref={ref}
      className={cn(
        'rudi-box',
        bordered && 'rudi-box--bordered',
        invert && 'rudi-box--invert',
        className,
      )}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```



LIVE COMPONENT

Box in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-box--bordered

Every primitive in this chapter follows the same `forwardRef` pattern. The ref is typed as `HTMLElement` in all cases for the same reason discussed in Chapter 4: the `as` prop accepts any element type, so a more specific ref type (for example, `HTMLDivElement`) would be incorrect when the consumer passes `as="section"`. `HTMLElement` is the accurate common base. If your project needs the fully typed polymorphic ref, the pattern is documented in the companion repository.

Center

The Center constrains content to a maximum width and horizontally centers it. It's the primitive you wrap around page-level content to prevent text lines from stretching to the full viewport width (which makes them difficult to read past about 70 characters per line).

CSS

```
.rudi-center {
  --center-max-width: var(--rudi-layout-center-max-width, 60rem);
  --center-gutters: var(--rudi-layout-center-gutters, 1rem);
  box-sizing: content-box;
  max-inline-size: var(--center-max-width);
  margin-inline: auto;
  padding-inline: var(--center-gutters);
}
.rudi-center--intrinsic {
```

```
  display: flex;
  flex-direction: column;
  align-items: center;
}
```

The `box-sizing: content-box` declaration is deliberate. It means the `max-inline-size` applies to the content only, and the `padding-inline` (gutters) is added outside that. The total width is `max-width + gutters`. Without this, the gutters eat into the content width, and your max-width is effectively narrower than specified.

The `intrinsic` variant uses flexbox centering to size the Center based on its content width rather than filling the max-width. This is useful for centering a button or a narrow form that shouldn't stretch to 60rem.

TSX

```tsx
// src/layouts/Center/Center.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './center.css'
export interface RudiCenterProps extends React.HTMLAttributes<HTMLElement> {
  maxWidth?: string
  gutters?: string
  intrinsic?: boolean
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiCenter = forwardRef<HTMLElement, RudiCenterProps>(function RudiCenter(
  {
    maxWidth,
    gutters,
    intrinsic = false,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (maxWidth) customProperties['--center-max-width'] = maxWidth
  if (gutters) customProperties['--center-gutters'] = gutters
  return (
    <Element
      ref={ref}
      className={cn(
        'rudi-center',
        intrinsic && 'rudi-center--intrinsic',
        className,
```

```
      )}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```



LIVE COMPONENT

Centered in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-center--default

Cluster

The Cluster arranges inline-like children (tags, buttons, badges, navigation links) in a horizontal row with consistent spacing. When the children don't fit in one row, they wrap to the next line. The `gap` property handles both horizontal and vertical spacing between wrapped rows.

CSS

```
.rudi-cluster {
  --cluster-space: var(--rudi-layout-cluster-space, 1rem);
  --cluster-justify: flex-start;
  --cluster-align: center;
  display: flex;
  flex-wrap: wrap;
  gap: var(--cluster-space);
  justify-content: var(--cluster-justify);
  align-items: var(--cluster-align);
}
```

TSX

```
// src/layouts/Cluster/Cluster.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './cluster.css'
export interface RudiClusterProps extends React.HTMLAttributes<HTMLElement> {
  space?: string
  justify?:
    | 'flex-start'
    | 'flex-end'
    | 'center'
    | 'space-between'
    | 'space-around'
```

```
  align?: 'flex-start' | 'flex-end' | 'center' | 'baseline' | 'stretch'
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiCluster = forwardRef<HTMLElement, RudiClusterProps>(function RudiCluster(
  {
    space,
    justify,
    align,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (space) customProperties['--cluster-space'] = space
  if (justify) customProperties['--cluster-justify'] = justify
  if (align) customProperties['--cluster-align'] = align
  return (
    <Element
      ref={ref}
      className={cn('rudi-cluster', className)}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```



LIVE COMPONENT

Cluster in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-cluster--default

A common use case is a header with a logo on the left and navigation links on the right.

TSX

```
<RudiCluster justify="space-between" align="center">
  <Logo />
  <RudiCluster space="0.5rem">
    <NavLink>About</NavLink>
    <NavLink>Blog</NavLink>
    <NavLink>Contact</NavLink>
```

```
  </RudiCluster>
</RudiCluster>
```

Sidebar

The Sidebar is the most complex layout primitive. It creates a two-panel layout where one panel (the sidebar) has a fixed width and the other (the content) fills the remaining space. When the content panel would be squeezed below a minimum width, both panels stack vertically. No media queries. The layout responds to its own container, not the viewport.

The technique uses flexbox with asymmetric `flex-grow` values:

```css
.rudi-sidebar {
  --sidebar-width: var(--rudi-layout-sidebar-width, 20rem);
  --sidebar-content-min: var(--rudi-layout-sidebar-content-min, 50%);
  --sidebar-space: var(--rudi-layout-sidebar-space, 1.5rem);
  display: flex;
  flex-wrap: wrap;
  gap: var(--sidebar-space);
}
.rudi-sidebar > :first-child {
  flex-basis: var(--sidebar-width);
  flex-grow: 1;
}
.rudi-sidebar > :last-child {
  flex-basis: 0;
  flex-grow: 999;
  min-inline-size: var(--sidebar-content-min);
}
.rudi-sidebar--right > :first-child {
  flex-basis: 0;
  flex-grow: 999;
  min-inline-size: var(--sidebar-content-min);
}
.rudi-sidebar--right > :last-child {
  flex-basis: var(--sidebar-width);
  flex-grow: 1;
}
.rudi-sidebar--no-stretch {
  align-items: flex-start;
}
```

LIVE COMPONENT
Sidebar in Storybook
rudiment-ui.netlify.app/?path=/story/layouts-sidebar--default

The key insight is `flex-grow: 999` on the content panel. Because this value is so much larger than the sidebar's `flex-grow: 1`, the content panel consumes all available space beyond the sidebar's `flex-basis`. The `min-inline-size: 50%` on the content panel forces wrapping: when the content would be narrower than 50% of the container, it wraps below the sidebar and both panels take full width.

This is an intrinsic layout. The same Sidebar component works in a 1200px container (side by side) and a 400px container (stacked) without any changes to the component or its props. The container's width is the only input.

TSX

```tsx
// src/layouts/Sidebar/Sidebar.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './sidebar.css'
export interface RudiSidebarProps extends React.HTMLAttributes<HTMLElement> {
  side?: 'left' | 'right'
  sideWidth?: string
  contentMin?: string
  space?: string
  noStretch?: boolean
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiSidebar = forwardRef<HTMLElement, RudiSidebarProps>(function RudiSidebar(
  {
    side = 'left',
    sideWidth,
    contentMin,
    space,
    noStretch = false,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (sideWidth) customProperties['--sidebar-width'] = sideWidth
```

```
  if (contentMin) customProperties['--sidebar-content-min'] = contentMin
  if (space) customProperties['--sidebar-space'] = space
  return (
    <Element
      ref={ref}
      className={cn(
        'rudi-sidebar',
        side === 'right' && 'rudi-sidebar--right',
        noStretch && 'rudi-sidebar--no-stretch',
        className,
      )}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```

The Sidebar expects exactly two children. The first child is the sidebar panel (when `side="left"`), and the second is the content panel. When `side="right"`, the roles reverse. The component doesn't enforce this at runtime, but the companion repository's documentation and stories clarify the expectation.

Switcher

The Switcher creates an N-column layout that collapses to a single column when the container is narrower than a threshold. Unlike the Sidebar (which has one fixed-width panel), the Switcher divides space equally among all its children.

The technique uses a `flex-basis` calculation that evaluates to either a very large positive number (forcing wrapping) or zero (allowing horizontal layout):

```css
.rudi-switcher {
  --switcher-threshold: var(--rudi-layout-switcher-threshold, 30rem);
  --switcher-space: var(--rudi-layout-switcher-space, 1.5rem);
  display: flex;
  flex-wrap: wrap;
  gap: var(--switcher-space);
}
.rudi-switcher > * {
  flex-grow: 1;
  flex-basis: calc((var(--switcher-threshold) - 100%) * 999);
}
```

The `calc()` expression is the heart of the trick. When the container is wider than the threshold, `(threshold - 100%)` is negative, and multiplying by 999 produces a very large negative number. Since `flex-basis` can't be negative, the browser clamps it to zero, and the children share the space equally in a row. When the container is narrower than the threshold, the result is positive, and each child's `flex-basis` exceeds the container width, forcing every child to wrap onto its own line.

TSX

```tsx
// src/layouts/Switcher/Switcher.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './switcher.css'
export interface RudiSwitcherProps extends React.HTMLAttributes<HTMLElement> {
  threshold?: string
  space?: string
  limit?: number
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiSwitcher = forwardRef<HTMLElement, RudiSwitcherProps>(
  function RudiSwitcher(
    {
      threshold,
      space,
      limit,
      as: Element = 'div',
      className,
      style,
      children,
      ...props
    },
    ref,
  ) {
    const customProperties: Record<string, string> = {}
    if (threshold) customProperties['--switcher-threshold'] = threshold
    if (space) customProperties['--switcher-space'] = space
    return (
      <Element
        ref={ref}
        className={cn('rudi-switcher', className)}
        style={{ ...customProperties, ...style } as React.CSSProperties}
        {...props}
      >
        {children}
      </Element>
    )
  },
)
```



LIVE COMPONENT

Switcher in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-switcher--default

NOTE

The `limit` prop is declared and destructured, and then never used. It is a stub in the companion repository too, where the Storybook control for it is wired up and moves nothing. It is documented here because you will find it in the type signature and wonder what it does. The answer is nothing, yet.

The intent behind it is real. Every Layout's Switcher supports capping how many items may share a row: past that count, every child goes full-width instead. The implementation is a rule like `.rudi-switcher > :nth-child(n + 4)` setting `flex-basis: 100%`, and the awkward part is that the `4` has to be baked into a selector. CSS `:nth-child()` takes selector math, not custom properties, so `:nth-child(n + var(--limit))` is invalid, and the value cannot arrive through the same inline-custom-property channel every other prop uses.

Two ways out. Generate a small scoped `<style>` element per instance, keyed on the limit. Or ship a fixed set of modifier classes (`rudi-switcher--limit-2` through `--limit-6`) and map the prop onto them, which covers the realistic range without runtime style injection. The second is less clever and easier to reason about.

Either way, the core switching behavior works without it, which is why it has stayed a stub.

Grid

The Grid creates a responsive grid without media queries. You specify a minimum cell width, and the browser determines how many columns fit. When the container narrows, columns drop off naturally.

CSS

```css
.rudi-grid {
  --grid-min-cell: var(--rudi-layout-grid-min-cell, 15rem);
  --grid-space: var(--rudi-layout-grid-space, 1.5rem);
  display: grid;
  grid-template-columns: repeat(
    auto-fill,
    minmax(min(var(--grid-min-cell), 100%), 1fr)
```

```
  );
  gap: var(--grid-space);
}
```

The `min(var(--grid-min-cell), 100%)` inside the `minmax()` is critical. Without it, cells wider than their container overflow. The `min()` function ensures cells never exceed 100% of the container width, so on a very narrow screen, you get a single column of full-width cells without any overflow.

TSX

```tsx
// src/layouts/Grid/Grid.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './grid.css'
export interface RudiGridProps extends React.HTMLAttributes<HTMLElement> {
  minCellWidth?: string
  space?: string
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiGrid = forwardRef<HTMLElement, RudiGridProps>(function RudiGrid(
  {
    minCellWidth,
    space,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (minCellWidth) customProperties['--grid-min-cell'] = minCellWidth
  if (space) customProperties['--grid-space'] = space
  return (
    <Element
      ref={ref}
      className={cn('rudi-grid', className)}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```



LIVE COMPONENT
Grid in Storybook
rudiment-ui.netlify.app/?path=/story/layouts-grid--default

Cover

The Cover vertically centers a principal child element within a minimum-height container. It's the primitive for hero sections, login pages, and any full-viewport layout where the main content should be vertically centered with optional header and footer elements pinned to the top and bottom.

CSS

```css
.rudi-cover {
  --cover-min-height: var(--rudi-layout-cover-min-height, 100vh);
  --cover-space: var(--rudi-layout-cover-space, 1.5rem);
  display: flex;
  flex-direction: column;
  min-block-size: var(--cover-min-height);
}
.rudi-cover > * {
  margin-block: var(--cover-space);
}
.rudi-cover > :first-child:not(.rudi-cover__centered) {
  margin-block-start: 0;
}
.rudi-cover > :last-child:not(.rudi-cover__centered) {
  margin-block-end: 0;
}
.rudi-cover > .rudi-cover__centered {
  margin-block: auto;
}
```

The `margin-block: auto` on the centered child is the vertical centering mechanism. In a flex column context, `auto` margin absorbs available space, pushing the element to the center. The first and last child overrides prevent extra spacing at the edges, so a header sits flush at the top and a footer sits flush at the bottom.

TSX

```tsx
// src/layouts/Cover/Cover.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './cover.css'
```

```tsx
export interface RudiCoverProps extends React.HTMLAttributes<HTMLElement> {
  minHeight?: string
  space?: string
  as?: React.ElementType
  children?: React.ReactNode
}
export const RudiCover = forwardRef<HTMLElement, RudiCoverProps>(function RudiCover(
  {
    minHeight,
    space,
    as: Element = 'div',
    className,
    style,
    children,
    ...props
  },
  ref,
) {
  const customProperties: Record<string, string> = {}
  if (minHeight) customProperties['--cover-min-height'] = minHeight
  if (space) customProperties['--cover-space'] = space
  return (
    <Element
      ref={ref}
      className={cn('rudi-cover', className)}
      style={{ ...customProperties, ...style } as React.CSSProperties}
      {...props}
    >
      {children}
    </Element>
  )
})
```



LIVE COMPONENT

Cover in Storybook

rudiment-ui.netlify.app/?path=/story/layouts-cover--default

The consumer marks the centered child with the `rudi-cover__centered` class:

TSX

```tsx
<RudiCover>
  <header>Site header</header>
  <main className="rudi-cover__centered">
    <h1>Welcome</h1>
    <p>This content is vertically centered.</p>
  </main>
  <footer>Site footer</footer>
```

```
</RudiCover>
```

An alternative approach is to export a `CoverCentered` wrapper component that applies the class automatically. Either approach works. The class-based approach is simpler and avoids an extra component.

Composing primitives

These primitives become most useful when nested. A complete page layout requires no bespoke CSS, just composition:

```tsx
<RudiStack space="0">
  <RudiBox as="header" padding="1rem" bordered>
    <RudiCenter maxWidth="72rem">
      {/* Placeholder components like Logo and NavLink represent your own app's
   navigation. */}
      <RudiCluster justify="space-between" align="center">
        <strong>App Name</strong>
        <RudiCluster space="0.5rem">
          <a href="/docs">Docs</a>
          <a href="/settings">Settings</a>
        </RudiCluster>
      </RudiCluster>
    </RudiCenter>
  </RudiBox>
  <RudiCenter maxWidth="72rem" gutters="1.5rem">
    <RudiSidebar sideWidth="14rem" contentMin="60%" space="2rem">
      <RudiStack as="nav" space="0.25rem">
        <a href="/dashboard">Dashboard</a>
        <a href="/projects">Projects</a>
        <a href="/team">Team</a>
      </RudiStack>
      <RudiStack as="main" space="2rem">
        <h1>Dashboard</h1>
        <RudiSwitcher threshold="20rem" space="1rem">
          <RudiBox bordered padding="1.5rem">
            Metric A
          </RudiBox>
          <RudiBox bordered padding="1.5rem">
            Metric B
          </RudiBox>
          <RudiBox bordered padding="1.5rem">
            Metric C
          </RudiBox>
        </RudiSwitcher>
        <RudiGrid minCellWidth="16rem" space="1rem">
          <RudiBox bordered padding="1rem">
```

```
            Card 1
          </RudiBox>
          <RudiBox bordered padding="1rem">
            Card 2
          </RudiBox>
          <RudiBox bordered padding="1rem">
            Card 3
          </RudiBox>
          <RudiBox bordered padding="1rem">
            Card 4
          </RudiBox>
        </RudiGrid>
      </RudiStack>
    </RudiSidebar>
  </RudiCenter>
</RudiStack>
```

This layout handles narrow viewports automatically. The Sidebar stacks when the content panel would be too narrow. The Switcher collapses the metric cards to a single column. The Grid drops columns as space decreases. No breakpoints, no media queries, no viewport-specific code.

The companion repository's example pages are the best place to see this at scale. The CRM dashboard below is built from these primitives and nothing else structural:

LIVE COMPONENT
Layout composition in a full page
rudiment-ui.netlify.app/?path=/docs/examples-crm-dashboard--docs

Attribution

The layout primitives in this guide are inspired by the intrinsic layout patterns described in Every Layout by Heydon Pickering and Andy Bell. Their work on algorithmic, context-independent CSS layout is recommended reading for anyone building layout systems.

What you have now

Eight layout primitives, all following the same pattern: token-driven CSS, React component wrapper, `as` prop for semantic HTML, `className` merging, prop-based overrides via inline custom properties. The layout system is usable independently of the UI components.

Before moving to interactive components, the next chapter adds the typography layer: Heading, Text, and Prose components that handle your UI's text content (Chapter 6).

CHAPTER 6

Typography: Heading, Text, and Prose

This chapter adds three components that enforce typographic consistency through the same token system that drives the rest of the library.



Layout primitives handle where things go. Typography components handle how text looks and how it flows. Most component libraries skip this layer entirely, leaving every heading size, text variant, and content spacing decision to the consumer. The result is inconsistency: five developers make five different choices about how large an `h2` should be.

Typography tokens

Add a `typography` namespace to `tokens/components.json`, alongside the `component` and `layout` namespaces from chapter 3. Typography is a component-tier concern: `heading.2.size` is a statement about a heading, not about the system's type scale, which already lives in tier 1 as `font.size.xl`.

JSON

```
{
  "typography": {
    "heading": {
      "1": {
        "size": { "$value": "{font.size.2xl}" },
        "weight": { "$value": "{font.weight.bold}" },
        "line-height": { "$value": "{font.lineHeight.tight}" },
        "tracking": { "$value": "-0.025em" },
        "color": { "$value": "{color.text.default}" }
      },
      "2": {
        "size": { "$value": "{font.size.xl}" },
        "weight": { "$value": "{font.weight.semibold}" },
        "line-height": { "$value": "{font.lineHeight.tight}" },
        "tracking": { "$value": "-0.025em" },
        "color": { "$value": "{color.text.default}" }
      },
      "3": {
        "size": { "$value": "{font.size.lg}" },
        "weight": { "$value": "{font.weight.semibold}" },
        "line-height": { "$value": "{font.lineHeight.tight}" },
        "tracking": { "$value": "0" },
        "color": { "$value": "{color.text.default}" }
      },
      "4": {
        "size": { "$value": "{font.size.base}" },
        "weight": { "$value": "{font.weight.semibold}" },
        "line-height": { "$value": "{font.lineHeight.tight}" },
        "tracking": { "$value": "0" },
        "color": { "$value": "{color.text.default}" }
      },
      "5": {
        "size": { "$value": "{font.size.sm}" },
```

```
        "weight": { "$value": "{font.weight.semibold}" },
        "line-height": { "$value": "{font.lineHeight.tight}" },
        "tracking": { "$value": "0" },
        "color": { "$value": "{color.text.default}" }
      },
      "6": {
        "size": { "$value": "{font.size.xs}" },
        "weight": { "$value": "{font.weight.semibold}" },
        "line-height": { "$value": "{font.lineHeight.normal}" },
        "tracking": { "$value": "0" },
        "color": { "$value": "{color.text.default}" }
      }
    },
    "body": {
      "size": { "$value": "{font.size.base}" },
      "weight": { "$value": "{font.weight.regular}" },
      "line-height": { "$value": "{font.lineHeight.normal}" },
      "color": { "$value": "{color.text.default}" }
    },
    "body-sm": {
      "size": { "$value": "{font.size.sm}" },
      "weight": { "$value": "{font.weight.regular}" },
      "line-height": { "$value": "{font.lineHeight.normal}" },
      "color": { "$value": "{color.text.default}" }
    },
    "caption": {
      "size": { "$value": "{font.size.sm}" },
      "weight": { "$value": "{font.weight.regular}" },
      "line-height": { "$value": "{font.lineHeight.normal}" },
      "color": { "$value": "{color.text.subtle}" }
    },
    "overline": {
      "size": { "$value": "{font.size.xs}" },
      "weight": { "$value": "{font.weight.semibold}" },
      "line-height": { "$value": "{font.lineHeight.normal}" },
      "tracking": { "$value": "0.1em" },
      "color": { "$value": "{color.text.subtle}" }
    },
    "code": {
      "font-size": { "$value": "0.875em" },
      "padding-y": { "$value": "0.125em" },
      "padding-x": { "$value": "0.25em" },
      "bg": { "$value": "{color.background.surface-raised}" },
      "radius": { "$value": "{radius.sm}" },
      "font-family": { "$value": "{font.family.mono}" }
    },
    "prose": {
      "space": { "$value": "1.5em" },
      "heading-before": { "$value": "2em" },
      "heading-after": { "$value": "0.75em" },
      "size-sm": { "$value": "{font.size.sm}" },
```

```
      "size-lg": { "$value": "{font.size.lg}" },
      "link-color": { "$value": "{color.brand.primary}" },
      "link-hover-color": { "$value": "{color.brand.primary-hover}" },
      "strong-weight": { "$value": "{font.weight.bold}" },
      "strong-color": { "$value": "{color.text.default}" },
      "blockquote-border": { "$value": "{color.border.default}" },
      "blockquote-color": { "$value": "{color.text.subtle}" },
      "code-bg": { "$value": "{color.background.surface-raised}" },
      "pre-bg": { "$value": "{color.background.surface-raised}" },
      "pre-border": { "$value": "{color.border.default}" },
      "pre-radius": { "$value": "{radius.md}" },
      "pre-font-size": { "$value": "{font.size.sm}" },
      "pre-line-height": { "$value": "{font.lineHeight.relaxed}" },
      "hr-color": { "$value": "{color.border.default}" },
      "table-border": { "$value": "{color.border.default}" },
      "table-header-weight": { "$value": "{font.weight.semibold}" },
      "figure-radius": { "$value": "{radius.md}" },
      "caption-color": { "$value": "{typography.caption.color}" },
      "h6-color": { "$value": "{color.text.subtle}" }
    }
  }
}
```

Run `npm run build:tokens` to regenerate the CSS custom properties.

Four things are worth noting before the components consume these.

All six heading levels are defined. It is tempting to stop at three, since most interfaces rarely go deeper. The `level` prop accepts `1` through `6`, so stopping at three means `<RudiHeading level=5>` renders correct semantics with no styling at all, and nothing in the type system warns you.

Every entry carries its own `color`. They all alias `{color.text.default}` today, which makes the tokens look redundant. They are not. A theme that wants slightly muted small headings, or a marketing surface where `h1` picks up the brand color, changes one token instead of adding a modifier class to every heading in the application.

The key is `line-height`, not `lineHeight`. Style Dictionary's CSS transform converts token paths to kebab-case, so both spellings would produce `--rudi-typography-heading-1-line-height`. Matching the CSS property name keeps the JSON readable next to `font-size` and `font-weight`, and matches every sibling key in the file.

`typography.code` and `typography.prose` are large. The `code` group exists so inline code has one definition rather than five hardcoded values repeated across

Text and Prose. The `prose` group carries 23 tokens because Prose styles raw HTML, and every element it touches needs a themeable value. Both are covered below.

Heading

The Heading component renders the correct semantic HTML element (`<h1>` through `<h6>`) based on a `level` prop. It also accepts a `size` prop for cases where the visual size needs to differ from the semantic level.

This distinction matters for accessibility. Screen reader users navigate by heading level to understand page structure. If every visually large heading is an `<h1>`, the document structure is flat and unhelpful. The `level` prop enforces correct document structure. The `size` prop lets you render an `<h2>` that looks like an `<h1>` when the design calls for it.

TSX

```tsx
// src/typography/Heading/Heading.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './heading.css'
import type {
  RudiTypographyAlign,
  RudiTypographyTone,
  RudiTypographyWeight,
} from '@/typography/Text/Text'
export interface RudiHeadingProps extends React.HTMLAttributes<HTMLHeadingElement> {
  level: 1 | 2 | 3 | 4 | 5 | 6
  size?: 1 | 2 | 3 | 4 | 5 | 6
  /** Font weight, overriding the size default. */
  weight?: RudiTypographyWeight
  /** Semantic text color. */
  tone?: RudiTypographyTone
  /** Text alignment. */
  align?: RudiTypographyAlign
  /** Removes the element's default block margin. */
  noMargin?: boolean
  children: React.ReactNode
  className?: string
}
export const RudiHeading = forwardRef<HTMLHeadingElement, RudiHeadingProps>(
  function RudiHeading(
    { level, size, weight, tone, align, noMargin, className, children, ...props },
    ref,
  ) {
    const Element = `h${level}` as const
    const visualSize = size ?? level
```

```
    return (
      <Element
        ref={ref}
        className={cn(
          `rudi-heading rudi-heading--${visualSize}`,
          weight && `rudi-heading--weight-${weight}`,
          tone && `rudi-heading--tone-${tone}`,
          align && `rudi-heading--align-${align}`,
          noMargin && 'rudi-heading--flush',
          className,
        )}
        {...props}
      >
        {children}
      </Element>
    )
  },
)
```

`weight`, `tone`, `align`, and `noMargin` are shared with Text, and their types are defined there. The Text section covers what each accepts; the important structural point is that one component owns the vocabulary and the other imports it, so the two can never drift apart.

The CSS applies typography tokens per heading size. All six levels get a block:

```css
.rudi-heading--1 {
  font-size: var(--rudi-typography-heading-1-size);
  font-weight: var(--rudi-typography-heading-1-weight);
  line-height: var(--rudi-typography-heading-1-line-height);
  letter-spacing: var(--rudi-typography-heading-1-tracking);
  color: var(--rudi-typography-heading-1-color);
}
/* Repeat for --2 through --6, swapping the token index. */
```

Then the shared modifiers, which are the same set on Text with a different class prefix:

```css
/* Weight modifiers */
.rudi-heading--weight-regular { font-weight: var(--rudi-font-weight-regular); }
.rudi-heading--weight-medium { font-weight: var(--rudi-font-weight-medium); }
.rudi-heading--weight-semibold { font-weight: var(--rudi-font-weight-semibold); }
.rudi-heading--weight-bold { font-weight: var(--rudi-font-weight-bold); }
/* Tone modifiers */
.rudi-heading--tone-default { color: var(--rudi-color-text-default); }
.rudi-heading--tone-subtle { color: var(--rudi-color-text-subtle); }
.rudi-heading--tone-disabled { color: var(--rudi-color-text-disabled); }
```

```
.rudi-heading--tone-brand { color: var(--rudi-color-brand-primary); }
.rudi-heading--tone-inverted { color: var(--rudi-color-text-on-inverted); }
.rudi-heading--tone-success { color: var(--rudi-color-feedback-success-text); }
.rudi-heading--tone-warning { color: var(--rudi-color-feedback-warning-text); }
.rudi-heading--tone-error { color: var(--rudi-color-feedback-error-text); }
.rudi-heading--tone-info { color: var(--rudi-color-feedback-info-text); }
/* Alignment modifiers */
.rudi-heading--align-start { text-align: start; }
.rudi-heading--align-center { text-align: center; }
.rudi-heading--align-end { text-align: end; }
.rudi-heading--align-justify { text-align: justify; }
/* Margin modifier */
.rudi-heading--flush { margin: 0; }
```

The tone modifiers are where the four-value feedback sets from chapter 3 pay off. `tone="error"` reads `--rudi-color-feedback-error-text`, the darker value, rather than `--rudi-color-feedback-error`, the saturated one. The saturated red is a good border and a poor body color; using it for text would fail contrast against a light surface at small sizes.

LIVE COMPONENT
Heading Levels
rudiment-ui.netlify.app/?path=/story/typography-heading--all-levels

Text

The Text component covers non-heading text: body paragraphs, small text, captions, overlines, and inline code. The `variant` prop selects the typographic style, and the `as` prop controls the rendered element.

TSX

```tsx
// src/typography/Text/Text.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './text.css'
export type RudiTypographyWeight = 'regular' | 'medium' | 'semibold' | 'bold'
export type RudiTypographyTone =
  | 'default'
  | 'subtle'
  | 'disabled'
  | 'brand'
  | 'inverted'
  | 'success'
  | 'warning'
```

```
  | 'error'
  | 'info'
export type RudiTypographyAlign = 'start' | 'center' | 'end' | 'justify'
export interface RudiTextProps extends React.HTMLAttributes<HTMLElement> {
  variant?: 'body' | 'body-sm' | 'caption' | 'overline' | 'code'
  as?: React.ElementType
  /** Font weight, overriding the variant default. */
  weight?: RudiTypographyWeight
  /** Semantic text color. */
  tone?: RudiTypographyTone
  /** Text alignment. */
  align?: RudiTypographyAlign
  /** Removes the element's default block margin. */
  noMargin?: boolean
  children: React.ReactNode
  className?: string
}
export const RudiText = forwardRef<HTMLElement, RudiTextProps>(function RudiText(
  {
    variant = 'body',
    as: Element = 'p',
    weight,
    tone,
    align,
    noMargin,
    className,
    children,
    ...props
  },
  ref,
) {
  return (
    <Element
      ref={ref}
      className={cn(
        `rudi-text rudi-text--${variant}`,
        weight && `rudi-text--weight-${weight}`,
        tone && `rudi-text--tone-${tone}`,
        align && `rudi-text--align-${align}`,
        noMargin && 'rudi-text--flush',
        className,
      )}
      {...props}
    >
      {children}
    </Element>
  )
})
```

Export those three type aliases. They are the shared typography vocabulary, and Heading imports them rather than redeclaring its own copies. Consumers get them from the package root too, which is what makes it possible to write a wrapper component that accepts a `tone` and passes it straight through.

Each type is a closed union for a reason. `tone` maps to semantic colors that exist in the token system, so `tone="purple"` fails at compile time instead of silently emitting a class with no rule behind it. The same applies to `weight` and `align`.

```css
.rudi-text--body {
  font-size: var(--rudi-typography-body-size);
  font-weight: var(--rudi-typography-body-weight);
  line-height: var(--rudi-typography-body-line-height);
  color: var(--rudi-typography-body-color);
}
.rudi-text--body-sm {
  font-size: var(--rudi-typography-body-sm-size);
  font-weight: var(--rudi-typography-body-sm-weight);
  line-height: var(--rudi-typography-body-sm-line-height);
  color: var(--rudi-typography-body-sm-color);
}
.rudi-text--caption {
  font-size: var(--rudi-typography-caption-size);
  font-weight: var(--rudi-typography-caption-weight);
  line-height: var(--rudi-typography-caption-line-height);
  color: var(--rudi-typography-caption-color);
}
.rudi-text--overline {
  font-size: var(--rudi-typography-overline-size);
  font-weight: var(--rudi-typography-overline-weight);
  line-height: var(--rudi-typography-overline-line-height);
  letter-spacing: var(--rudi-typography-overline-tracking);
  color: var(--rudi-typography-overline-color);
  text-transform: uppercase;
}
.rudi-text--code {
  font-family: var(--rudi-typography-code-font-family);
  font-size: var(--rudi-typography-code-font-size);
  background: var(--rudi-typography-code-bg);
  padding: var(--rudi-typography-code-padding-y)
    var(--rudi-typography-code-padding-x);
  border-radius: var(--rudi-typography-code-radius);
}
```

Then the same four modifier groups as Heading, with the `rudi-text--` prefix: `weight-*`, `tone-*`, `align-*`, and `flush`.

The `code` variant is the clearest argument for the `typography.code` token group. Written with literal values it needs a font stack, a size, a background, two padding values, and a radius, all hardcoded, and all repeated again inside Prose for inline `<code>` elements. As tokens they are defined once and both places read them.

Note that `code` sets `font-size` to a token whose value is `0.875em`, not `rem`. Inline code sits inside running text, so it should scale with whatever text surrounds it. An absolute size would make code inside a heading look wrong.



LIVE COMPONENT
Text Levels
rudiment-ui.netlify.app/?path=/story/typography-text--all-variants

Prose

The Prose component solves the problem that Stack and Heading alone cannot: context-aware vertical rhythm in long-form content.

Stack applies uniform spacing between its children. But long-form content needs variable spacing. The gap above a heading should be larger because it signals a new section. The gap below a heading should be smaller because the heading belongs to what follows. Paragraphs need consistent but moderate spacing. Code blocks need extra breathing room.

Prose handles all of this with CSS selectors that target element-type relationships:

TSX

```tsx
// src/typography/Prose/Prose.tsx
import { forwardRef } from 'react'
import { cn } from '@/utils/cn'
import './prose.css'
export interface RudiProseProps extends React.HTMLAttributes<HTMLElement> {
  size?: 'sm' | 'base' | 'lg'
  as?: React.ElementType
  children: React.ReactNode
  className?: string
}
export const RudiProse = forwardRef<HTMLElement, RudiProseProps>(function RudiProse(
  { size = 'base', as: Element = 'div', className, children, ...props },
  ref,
) {
  return (
    <Element
```

```
      ref={ref}
      className={cn(
        'rudi-prose',
        size !== 'base' && `rudi-prose--${size}`,
        className,
      )}
      {...props}
    >
      {children}
    </Element>
  )
})
```

```css
.rudi-prose {
  --prose-space: var(--rudi-typography-prose-space, 1.5em);
  --prose-heading-before: var(--rudi-typography-prose-heading-before, 2em);
  --prose-heading-after: var(--rudi-typography-prose-heading-after, 0.75em);
  color: var(--rudi-typography-body-color);
  font-size: var(--rudi-typography-body-size);
  font-weight: var(--rudi-typography-body-weight);
  line-height: var(--rudi-typography-body-line-height);
}
/* Base spacing between all adjacent elements */
.rudi-prose > * + * {
  margin-block-start: var(--prose-space);
}
/* More space before headings (they signal new sections) */
.rudi-prose > :is(h1, h2, h3, h4, h5, h6) {
  margin-block-start: var(--prose-heading-before);
}
/* Less space after headings (they belong to what follows) */
.rudi-prose > :is(h1, h2, h3, h4, h5, h6) + * {
  margin-block-start: var(--prose-heading-after);
}
/* Consecutive headings sit closer together */
.rudi-prose > :is(h1, h2, h3, h4, h5, h6) + :is(h1, h2, h3, h4, h5, h6) {
  margin-block-start: var(--prose-heading-after);
}
/* Extra breathing room around code blocks */
.rudi-prose > pre {
  margin-block-start: 2em;
}
.rudi-prose > pre + * {
  margin-block-start: 2em;
}
/* No top margin on the first child */
.rudi-prose > :first-child {
  margin-block-start: 0;
}
/* Size variants scale the base font size; em-based spacing adjusts proportionally */
```

```
.rudi-prose--sm {
  font-size: var(--rudi-typography-prose-size-sm);
}
.rudi-prose--lg {
  font-size: var(--rudi-typography-prose-size-lg);
}
```

The spacing values use `em`, not `rem`. This is the key detail. An `em` value scales with the element's own font size. When you render `<RudiProse size="sm">` inside a sidebar, the spacing contracts proportionally with the smaller text. When you render `<RudiProse size="lg">` for a blog post, the spacing expands. The rhythm stays correct regardless of size.

That `em` argument only holds because `.rudi-prose` sets its own `font-size` rather than inheriting one. Without those four base declarations, Prose adopts whatever typography surrounds it, and the `em` spacing tracks a value you did not choose.

The consecutive-heading rule is the kind of thing you notice only after shipping. Without it, an `<h3>` directly following an `<h2>` picks up `--prose-heading-before` (2em) and floats away from the section title it belongs to. The rule pulls it back to `--prose-heading-after`.

Prose styles raw HTML

Everything above is spacing, and spacing is roughly a fifth of what Prose actually does. The remaining rules style unclassed HTML elements, because that is the whole point of the component: content arrives from a CMS, a Markdown renderer, or an API as plain `<h2>`, `<p>`, `<ul>`, `<blockquote>`, and `<table>` tags with no class names to hook into.

The full stylesheet in the companion repository runs to about 240 lines and covers ten groups:

Group	Elements	Representative tokens
Headings	`h1` through `h6`	`--rudi-typography-heading-{n}-*`, plus `--rudi-typography-prose-h6-color`
Links	`a`, `a:hover`	`--rudi-typography-prose-link-color`, `-link-hover-color`
Inline emphasis	`strong`, `em`	`--rudi-typography-prose-strong-weight`, `-strong-color`

Group	Elements	Representative tokens
Lists	`ul`, `ol`, `li`, nested lists	indent and marker values
Blockquotes	`blockquote`	`--rudi-typography-prose-blockquote-border`, `-blockquote-color`
Inline code	`code`	`--rudi-typography-code-*`, `--rudi-typography-prose-code-bg`
Code blocks	`pre`, `pre code`	`--rudi-typography-prose-pre-{bg,border,radius,font-size,line-height}`
Rules	`hr`	`--rudi-typography-prose-hr-color`
Tables	`table`, `thead th`, `tbody tr`, `tbody td`	`--rudi-typography-prose-table-border`, `-table-header-weight`
Figures	`figure > img`, `figcaption`	`--rudi-typography-prose-figure-radius`, `-caption-color`

Two patterns are worth copying rather than reading past.

`pre code` resets what `code` sets. Inline code gets a tinted background, padding, and its own size; the same element inside a `<pre>` should get none of those, because the block already provides them:

```css
.rudi-prose pre code {
  background: none;
  padding: 0;
  border-radius: 0;
  font-size: inherit;
}
```

And every one of these rules is scoped under `.rudi-prose`. A component library that styles bare `h2` and `table` selectors globally is doing the same thing Tailwind's preflight does, which chapter 1 rejected. Inside an opt-in wrapper the same rules are exactly what the consumer asked for.

LIVE COMPONENT

Prose Levels

rudiment-ui.netlify.app/?path=/story/typography-prose--all-sizes

Usage

```tsx
<RudiProse as="article" size="base">
  <RudiHeading level={1}>Building accessible component libraries</RudiHeading>
  <RudiText>
    Component libraries are only as useful as their accessibility coverage. This
    article explores what that means in practice.
  </RudiText>
  <RudiHeading level={2}>The problem with retrofitting</RudiHeading>
  <RudiText>
    Adding accessibility after the fact costs more than building it in from the
    start. The reason is structural: accessible behavior affects component APIs,
    keyboard handling, and focus management.
  </RudiText>
  <RudiText>
    Consider a modal dialog. An accessible implementation requires focus
    trapping, Escape to close, scroll locking, and aria-modal. These behaviors
    are not cosmetic. They change how the component works.
  </RudiText>
  <pre>
    <code>{`const { dialogProps } = useDialog({}, ref);`}</code>
  </pre>
  <RudiText>
    React Aria provides these behaviors as hooks. The hook returns props that
    you spread onto your elements.
  </RudiText>
</RudiProse>
```

The headings get more space above (new section) and less space below (connected to the following paragraph). Consecutive paragraphs get standard spacing. The code block gets extra room. You don't need manual spacing classes.

What's next

You now have three typography components (Heading, Text, Prose) alongside eight layout primitives, all driven by the same token system. The library handles spatial arrangement, typographic consistency, and long-form content rhythm. The next chapter adds interactive components with React Aria.

CHAPTER 7

Accessible components with React Aria

This chapter introduces React Aria and uses it to build three interactive components: Button, IconButton, and Input.



Layout primitives handle spatial arrangement. UI components handle interaction. This chapter introduces React Aria and uses it to build three interactive components: Button, IconButton, and Input. The pattern you learn here applies to every interactive component in the library.

Installing React Aria

```bash
npm install react-aria react-stately @react-aria/utils react-aria-components
npm install @iconify/react
```

`react-aria` provides the hooks that manage keyboard interaction, focus behavior, and ARIA attributes. `react-stately` provides the state management hooks that React Aria depends on, such as tracking whether a select is open or which item is highlighted. `@react-aria/utils` supplies the ref helper every one of these components needs.

`react-aria-components` is the higher-level sibling: prebuilt accessible components rather than hooks. This guide builds from hooks, because assembling a component from `useButton` is how you learn what the hook is doing for you. But some patterns are not worth hand-rolling, and the Menu family in the companion repository is built on `react-aria-components` for exactly that reason. Installing it now means the dependency is in place when you reach it.

`@iconify/react` renders the icons. It loads icon data on demand by name, so `iconBefore="mdi:plus"` fetches only that glyph instead of bundling a set. The Button below takes icons as name strings rather than React nodes, which is what makes that work.

How React Aria hooks work

React Aria hooks return props objects that you spread onto your DOM elements. The hook manages the complex interaction logic, and you manage the rendering. Here's the concept, simplified:

```tsx
const { buttonProps } = useButton({ onPress, isDisabled }, ref)
return (
  <button {...buttonProps} ref={ref}>
    Click me
  </button>
```

```
)
```

The `buttonProps` object includes `onClick`, `onKeyDown`, `tabIndex`, `aria-disabled`, and other attributes that the hook calculates based on the state you pass in. You don't manage these attributes manually. React Aria handles the edge cases (pointer type normalization, touch delay cancellation, keyboard activation) so you don't have to discover them in production.

Building Button

Button and IconButton both have a loading state, and they share identical logic for it: disable the button via `useButton`, and set `aria-busy` on the element. Extract that into a shared `useLoadingButton` hook so neither component duplicates the behavior.

TSX

```tsx
// src/hooks/useLoadingButton.ts
import { useButton } from 'react-aria'
import type { AriaButtonProps } from 'react-aria'
import type { RefObject } from 'react'
export function useLoadingButton(
  ariaProps: AriaButtonProps,
  isLoading: boolean,
  ref: RefObject<HTMLButtonElement | null>,
) {
  const { buttonProps } = useButton(
    { ...ariaProps, isDisabled: ariaProps.isDisabled || isLoading },
    ref,
  )
  return {
    buttonProps: {
      ...buttonProps,
      'aria-busy': isLoading || undefined,
    },
  }
}
```

`useObjectRef` from `@react-aria/utils` converts a forwarded ref (which may be a callback ref or `null`) into the stable `RefObject` that React Aria hooks require. Any component that uses `React.forwardRef` alongside a React Aria hook needs this utility.

Note the ref parameter type: `RefObject<HTMLButtonElement | null>`, not `RefObject<HTMLButtonElement>`. React 19's type definitions made `RefObject.current`

nullable, and `useObjectRef` returns a ref matching that shape. Writing the non-null version produces a type error at every call site that is genuinely confusing to read, because the error points at the caller rather than the signature.

TSX

```tsx
// src/components/Button/Button.tsx
import React from 'react'
import { useObjectRef } from '@react-aria/utils'
import type { AriaButtonProps } from 'react-aria'
import { cn } from '@/utils/cn'
import { RudiIcon } from '@/components/Icon/Icon'
import { useLoadingButton } from '@/hooks/useLoadingButton'
import './button.css'
export interface RudiButtonProps extends AriaButtonProps {
  variant?: 'primary' | 'secondary' | 'destructive' | 'ghost'
  size?: 'sm' | 'md' | 'lg'
  isLoading?: boolean
  /** Iconify icon name rendered before the label, e.g. 'mdi:plus'. */
  iconBefore?: string
  /** Iconify icon name rendered after the label, e.g. 'mdi:arrow-right'. */
  iconAfter?: string
  className?: string
  children: React.ReactNode
}
export const RudiButton = React.forwardRef<HTMLButtonElement, RudiButtonProps>(
  function RudiButton(
    {
      variant = 'primary',
      size = 'md',
      isLoading = false,
      iconBefore,
      iconAfter,
      className,
      children,
      ...ariaProps
    },
    forwardedRef,
  ) {
    const ref = useObjectRef(forwardedRef)
    const { buttonProps } = useLoadingButton(ariaProps, isLoading, ref)
    return (
      <button
        {...buttonProps}
        ref={ref}
        className={cn(
          'rudi-button',
          `rudi-button--${variant}`,
          `rudi-button--${size}`,
          isLoading && 'rudi-button--loading',
          className,
```

```
        )}
      >
        {isLoading ? (
          <>
            <span className="rudi-button__spinner" role="img" aria-hidden="true" />
            <span>{children}</span>
          </>
        ) : (
          <>
            {iconBefore && (
              <RudiIcon icon={iconBefore} size={size} className="rudi-button__icon" />
            )}
            <span>{children}</span>
            {iconAfter && (
              <RudiIcon icon={iconAfter} size={size} className="rudi-button__icon" />
            )}
          </>
        )}
      </button>
    )
  },
)
```

Key points in this implementation:

`useButton` normalizes press events. The hook provides `onPress` instead of `onClick`. The difference matters. `onClick` fires on mouse click and Enter key, but its behavior varies across pointer types and browsers. `onPress` fires consistently for mouse, touch, and keyboard, with proper handling for touch delay cancellation and preventing double-fires on Android.

Loading state disables the button. The `useLoadingButton` hook passes `isDisabled: true` to `useButton` when loading, so `onPress` won't fire. It also sets `aria-busy="true"` instead of `aria-disabled`, because screen readers announce these differently. `aria-disabled` means permanently unavailable; `aria-busy` means processing.

`forwardRef` exposes the underlying element. Wrapping the component in `React.forwardRef` lets consumers pass a ref and call imperative methods like `.focus()` or `.blur()`. `useObjectRef` from `@react-aria/utils` converts the forwarded ref (which may be a callback ref or null) into the stable `RefObject` that React Aria hooks require.

Icons are name strings, not nodes. `iconBefore="mdi:plus"` is less flexible than `iconBefore={<PlusIcon />}`, and that is deliberate. A string keeps the icon set out of the consumer's bundle until it is used, keeps the prop serializable so Storybook

can offer it as a text control, and stops consumers from passing arbitrary markup into a slot sized for a glyph. If you need a genuinely custom node, compose it into `children`.

The label is always wrapped in a `<span>`. Even with no icons, `children` goes inside an element rather than sitting as a bare text node. That gives the flexbox layout a real item to align and size, so an icon and its label share a baseline instead of the text collapsing against the icon box. It also gives tests a stable handle on the label independent of the icons.

The spinner carries `role="img"`. It is an empty `<span>` styled into a ring by CSS, which means assistive technology sees nothing at all without a role. `role="img"` plus `aria-hidden="true"` makes it a recognizable element in the accessibility tree that is then correctly skipped, rather than an anonymous box. The `aria-busy` attribute on the button is what actually announces the loading state.

The focus ring. React Aria handles focus management, but the visible focus indicator is your responsibility in CSS. The component's CSS uses `:focus-visible` rather than `:focus`:

```css
.rudi-button:focus-visible {
  outline: 2px solid var(--rudi-component-button-focus-outline-color);
  outline-offset: 2px;
}
```

`:focus-visible` shows the focus ring only for keyboard navigation, not for mouse clicks. Mouse users don't need a focus indicator because they can see where they're clicking. Keyboard users need the ring to track their position.

Button CSS

Create `src/components/Button/button.css` next to the component. This is the first stylesheet where the component token tier from chapter 3 earns its keep, so read the `var()` names carefully: every one of them is a `component.button.*` token, not a semantic or global one.

```css
.rudi-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: var(--rudi-component-button-gap);
```

```
  font-weight: var(--rudi-component-button-font-weight);
  border-radius: var(--rudi-component-button-radius);
  cursor: pointer;
  transition:
    background-color var(--rudi-motion-duration-base) var(--rudi-motion-easing-standard),
    color var(--rudi-motion-duration-base) var(--rudi-motion-easing-standard);
  border: 1px solid transparent;
}
.rudi-button:focus-visible {
  outline: 2px solid var(--rudi-component-button-focus-outline-color);
  outline-offset: 2px;
}
.rudi-button[aria-disabled='true'] {
  cursor: not-allowed;
  opacity: 0.5;
}
/* Sizes */
.rudi-button--sm {
  padding: var(--rudi-component-button-padding-y-sm)
    var(--rudi-component-button-padding-x-sm);
  font-size: var(--rudi-component-button-font-size-sm);
}
.rudi-button--md {
  padding: var(--rudi-component-button-padding-y-md)
    var(--rudi-component-button-padding-x-md);
  font-size: var(--rudi-component-button-font-size-md);
}
.rudi-button--lg {
  padding: var(--rudi-component-button-padding-y-lg)
    var(--rudi-component-button-padding-x-lg);
  font-size: var(--rudi-component-button-font-size-lg);
}
/* Variants */
.rudi-button--primary {
  background-color: var(--rudi-component-button-primary-bg);
  color: var(--rudi-component-button-primary-text);
}
.rudi-button--primary:hover:not([aria-disabled='true']) {
  background-color: var(--rudi-component-button-primary-hover-bg);
}
.rudi-button--secondary {
  background-color: var(--rudi-component-button-secondary-bg);
  color: var(--rudi-component-button-secondary-text);
  border-color: var(--rudi-component-button-secondary-border);
}
.rudi-button--secondary:hover:not([aria-disabled='true']) {
  background-color: var(--rudi-component-button-secondary-hover-bg);
}
.rudi-button--destructive {
  background-color: var(--rudi-component-button-destructive-bg);
  color: var(--rudi-component-button-destructive-text);
```

```
}
.rudi-button--ghost {
  background-color: transparent;
  color: var(--rudi-component-button-ghost-text);
}
.rudi-button--ghost:hover:not([aria-disabled='true']) {
  background-color: var(--rudi-component-button-ghost-hover-bg);
}
/* Icons */
.rudi-button .rudi-button__icon {
  color: inherit;
}
/* Loading */
.rudi-button--loading {
  position: relative;
}
.rudi-button__spinner {
  width: 1em;
  height: 1em;
  border: 2px solid currentColor;
  border-right-color: transparent;
  border-radius: 50%;
  animation: rudi-spin var(--rudi-motion-duration-spinner) linear infinite;
}
@keyframes rudi-spin {
  to {
    transform: rotate(360deg);
  }
}
```

Not one hardcoded value, and not one direct reference to a semantic or global token. `var(--rudi-component-button-primary-bg)` resolves to `{color.brand.primary}`, which resolves to `{color.neutral.900}`, which resolves to a hex value. Three hops, all visible in dev tools.

The indirection buys two things. Someone rebranding the library changes `color.brand.primary` once and every component follows. Someone who wants a different button specifically, without disturbing anything else that uses the brand color, changes `component.button.primary-bg` and nothing else moves. Reading the semantic token directly from the stylesheet would give you the first without the second.

The transition reads motion tokens rather than a literal `150ms`. That is what lets a theme slow the whole system down, or flatten it, without editing every stylesheet.

Add the matching entries to `tokens/components.json` under `component.button` before you build: `gap`, `font-weight`, `radius`, `focus-outline-color`, the

`padding-{x,y}-{sm,md,lg}` and `font-size-{sm,md,lg}` sets, and the background, text, hover, and border values for each of the four variants.



LIVE COMPONENT

Button variants in Storybook

rudiment-ui.netlify.app/?path=/story/components-button--all-variants

Building Input

TSX

```tsx
// src/components/Input/Input.tsx
import React from 'react'
import { useTextField } from 'react-aria'
import { useObjectRef } from '@react-aria/utils'
import { cn } from '@/utils/cn'
import './input.css'
export interface RudiInputProps {
  label: string
  type?: 'text' | 'email' | 'password' | 'url' | 'tel' | 'search' | 'number'
  placeholder?: string
  description?: string
  errorMessage?: string
  isRequired?: boolean
  isDisabled?: boolean
  value?: string
  defaultValue?: string
  onChange?: (value: string) => void
  className?: string
}
export const RudiInput = React.forwardRef<HTMLInputElement, RudiInputProps>(
  function RudiInput(
    {
      label,
      type = 'text',
      placeholder,
      description,
      errorMessage,
      isRequired = false,
      isDisabled = false,
      value,
      defaultValue,
      onChange,
      className,
    },
    forwardedRef,
  ) {
```

```
  const ref = useObjectRef(forwardedRef)
  const {
    labelProps,
    inputProps,
    descriptionProps,
    errorMessageProps,
    isInvalid,
  } = useTextField(
    {
      label,
      type,
      placeholder,
      description,
      errorMessage,
      isRequired,
      isDisabled,
      value,
      defaultValue,
      onChange,
      isInvalid: !!errorMessage,
    },
    ref,
  )
  return (
    <div className={cn('rudi-input', className)}>
      <label {...labelProps} className="rudi-input__label">
        {label}
        {isRequired && (
          <span className="rudi-input__required" aria-hidden="true">
            {' '}
            *
          </span>
        )}
      </label>
      <input
        {...inputProps}
        ref={ref}
        className={cn(
          'rudi-input__field',
          isInvalid && 'rudi-input__field--error',
        )}
      />
      {description && !isInvalid && (
        <p {...descriptionProps} className="rudi-input__description">
          {description}
        </p>
      )}
      {isInvalid && errorMessage && (
        <p {...errorMessageProps} className="rudi-input__error">
          {errorMessage}
        </p>
```

```
        )}
      </div>
    )
  },
)
```

`useTextField` handles all the ARIA wiring. The hook:

- Associates the label with the input via matching `id` and `htmlFor` attributes.
- Links the description and error message to the input via `aria-describedby`.
- Sets `aria-required="true"` when `isRequired` is true.
- Sets `aria-invalid="true"` when `isInvalid` is true.
- Returns `isInvalid` so you can use it for conditional rendering and styling.

You don't set any of these attributes manually. The hook returns them inside `inputProps`, `labelProps`, `descriptionProps`, and `errorMessageProps`, and you spread them onto the corresponding elements.

The error state is conditional: when `isInvalid` is true (which happens when `errorMessage` is passed), the input shows the error message and hides the description. When the input is valid, it shows the description (if one exists). This prevents screen readers from announcing both the description and the error simultaneously.



LIVE COMPONENT

Input default state in Storybook

rudiment-ui.netlify.app/?path=/story/components-input--default

IconButton

IconButton follows the same pattern as Button, but requires an `aria-label` because it has no visible text:

TSX

```tsx
// src/components/IconButton/IconButton.tsx
import React from 'react'
import { useObjectRef } from '@react-aria/utils'
import type { AriaButtonProps } from 'react-aria'
import { cn } from '@/utils/cn'
import { useLoadingButton } from '@/hooks/useLoadingButton'
import './icon-button.css'
```

```
export interface RudiIconButtonProps extends AriaButtonProps {
  'aria-label': string
  variant?: 'primary' | 'secondary' | 'destructive' | 'ghost'
  size?: 'sm' | 'md' | 'lg'
  isLoading?: boolean
  className?: string
  children: React.ReactElement
}
export const RudiIconButton = React.forwardRef<HTMLButtonElement, RudiIconButtonProps>(
  function RudiIconButton(
    {
      variant = 'secondary',
      size = 'md',
      isLoading = false,
      className,
      children,
      ...ariaProps
    },
    forwardedRef,
  ) {
    const ref = useObjectRef(forwardedRef)
    const { buttonProps } = useLoadingButton(ariaProps, isLoading, ref)
    return (
      <button
        {...buttonProps}
        ref={ref}
        className={cn(
          'rudi-icon-button',
          `rudi-icon-button--${variant}`,
          `rudi-icon-button--${size}`,
          className,
        )}
      >
        {children}
      </button>
    )
  },
)
```

The `'aria-label': string` type (without the `?` optional marker) forces consumers to provide an accessible label. TypeScript will produce a compile error if they forget. This is a deliberate design choice: an icon-only button without an accessible name is unusable for screen reader users (it's announced as "button" with no label), and making it a required prop prevents that failure mode at development time rather than in an accessibility audit.



LIVE COMPONENT

Icon Button Variants

rudiment-ui.netlify.app/?path=/story/components-iconbutton--all-variants

NOTE

Button imports `RudiIcon`, a thin wrapper over `@iconify/react` that normalizes sizing against the token scale. Chapter 14 builds it along with the rest of the presentational components. Until then, either stub it as a component that renders `<Icon icon={icon} />` from `@iconify/react` directly, or leave `iconBefore` and `iconAfter` out and add them when you reach that chapter.

What you have now

Three interactive components (Button, IconButton, Input), each built on React Aria. The pattern is consistent: import a hook, pass state and configuration, spread the returned props onto your elements. React Aria handles the ARIA attributes, keyboard behavior, and focus management. You handle the rendering and styling.

The next chapter builds the remaining form components using the same approach.

CHAPTER 8

Form components

This chapter builds Select, Checkbox, CheckboxGroup, RadioGroup, and Switch. Each one follows the React Aria hook pattern from the previous chapter.



This chapter builds Select, Checkbox, CheckboxGroup, RadioGroup, and Switch. Each one follows the React Aria hook pattern from the previous chapter. The implementations get progressively more complex: Select involves a popover, Checkbox supports indeterminate state, and RadioGroup handles arrow key navigation within a group.

Select

Select is the most complex form component in the library. It combines multiple React Aria hooks: `useSelect` for the trigger button, `useListBox` for the options list, `useOption` for each individual option, and `usePopover` for the floating panel that contains the list. It also uses `useSelectState` from `react-stately`, which manages the open/closed state of the popover and tracks the currently selected option.

TSX

```tsx
// src/components/Select/Select.tsx (simplified structure)
// Note: this example references Popover, ListBox, ChevronIcon, and DismissButton
// as helper components internal to the Select implementation. The companion
// repository contains the complete implementations. Here we focus on the hook
// wiring pattern.
import { useRef } from 'react'
import { useSelectState, type SelectState } from 'react-stately'
import {
  useSelect,
  useButton,
  useListBox,
  useOption,
  usePopover,
  HiddenSelect,
} from 'react-aria'
import type { RefObject } from 'react'
import type { Key } from 'react-aria'
import { cn } from '@/utils/cn'
import './select.css'
export interface RudiSelectProps<T extends object> {
  label: string
  items: Iterable<T>
  children: (item: T) => React.ReactNode
  selectedKey?: Key | null
  defaultSelectedKey?: Key | null
  onSelectionChange?: (key: Key | null) => void
  placeholder?: string
  description?: string
  errorMessage?: string
  isRequired?: boolean
```

```
  isDisabled?: boolean
  className?: string
}
// eslint-disable-next-line @typescript-eslint/no-explicit-any
function ChevronIcon(props: React.SVGProps<SVGSVGElement>) {
  return (
    <svg
      {...props}
      xmlns="http://www.w3.org/2000/svg"
      width="16"
      height="16"
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth="2"
      strokeLinecap="round"
      strokeLinejoin="round"
    >
      <polyline points="6 9 12 15 18 9" />
    </svg>
  )
}
interface PopoverInternalProps {
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  state: SelectState<any>
  triggerRef: RefObject<HTMLButtonElement | null>
  popoverRef: RefObject<HTMLDivElement | null>
  children: React.ReactNode
}
function Popover({
  state,
  triggerRef,
  popoverRef,
  children,
}: PopoverInternalProps) {
  const { popoverProps } = usePopover({ triggerRef, popoverRef }, state)
  return (
    <div
      {...popoverProps}
      ref={popoverRef}
      className="rudi-select__popover"
    >
      {children}
    </div>
  )
}
interface ListBoxInternalProps {
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  state: SelectState<any>
  listBoxRef: RefObject<HTMLUListElement | null>
  [key: string]: unknown
```

```
}
function ListBox({ state, listBoxRef, ...props }: ListBoxInternalProps) {
  const { listBoxProps } = useListBox(props, state, listBoxRef)
  return (
    <ul {...listBoxProps} ref={listBoxRef} className="rudi-select__listbox">
      {[...state.collection].map((item) => (
        <Option key={item.key} item={item} state={state} />
      ))}
    </ul>
  )
}
interface OptionInternalProps {
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  item: any
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  state: SelectState<any>
}
function Option({ item, state }: OptionInternalProps) {
  const optionRef = useRef<HTMLLIElement>(null)
  const { optionProps, isSelected, isFocused } = useOption(
    { key: item.key },
    state,
    optionRef,
  )
  return (
    <li
      {...optionProps}
      ref={optionRef}
      className="rudi-select__option"
      data-focused={isFocused}
      aria-selected={isSelected}
    >
      {item.rendered}
    </li>
  )
}
export function RudiSelect<T extends object>(props: RudiSelectProps<T>) {
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  const state = useSelectState(props as any)
  const triggerRef = useRef<HTMLButtonElement>(null)
  const popoverRef = useRef<HTMLDivElement>(null)
  const listBoxRef = useRef<HTMLUListElement>(null)
  const {
    labelProps,
    triggerProps,
    valueProps,
    menuProps,
    descriptionProps,
    errorMessageProps,
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
  } = useSelect(props as any, state, triggerRef)
```

```
  const { buttonProps } = useButton(triggerProps, triggerRef)
  return (
    <div className={cn('rudi-select', props.className)}>
      <label {...labelProps} className="rudi-select__label">
        {props.label}
      </label>
      <HiddenSelect state={state} triggerRef={triggerRef} label={props.label} />
      <button
        {...buttonProps}
        ref={triggerRef}
        className={cn(
          'rudi-select__trigger',
          props.errorMessage && 'rudi-select__trigger--error',
        )}
      >
        <span {...valueProps}>
          {state.selectedItem
            ? state.selectedItem.rendered
            : props.placeholder || 'Select an option'}
        </span>
        <ChevronIcon aria-hidden="true" />
      </button>
      {state.isOpen && (
        <Popover state={state} triggerRef={triggerRef} popoverRef={popoverRef}>
          <ListBox {...menuProps} state={state} listBoxRef={listBoxRef} />
        </Popover>
      )}
      {props.description && !props.errorMessage && (
        <p {...descriptionProps} className="rudi-select__description">
          {props.description}
        </p>
      )}
      {props.errorMessage && (
        <p {...errorMessageProps} className="rudi-select__error">
          {props.errorMessage}
        </p>
      )}
    </div>
  )
}
```

The `HiddenSelect` component from React Aria renders a hidden native `<select>` element for form submission compatibility. Unlike the hooks imported alongside it, `HiddenSelect` is a React component — the first time a component is imported directly from `react-aria` in this guide rather than following the hook-only pattern. The visible trigger is a `<button>` that opens the popover. The listbox inside the popover handles arrow key navigation, typeahead (type a letter to jump to matching options), and `Home` / `End` to jump to the first/last option.

This is a lot of code for a select input. That's the point. Getting keyboard navigation, ARIA attributes, popover positioning, and focus management right for a select is genuinely difficult. React Aria handles the behavioral complexity so your component code is mostly rendering and styling.

Here's the finished Select rendered from the companion repository's Storybook. Open the popover to exercise the keyboard navigation, typeahead, and focus behavior described above.



LIVE COMPONENT
Select in Storybook
rudiment-ui.netlify.app/?path=/story/components-select--with-description

The option element

Select takes an `items` array and a render function, and that render function has a constraint that is easy to get wrong. It must return a React Stately collection element, not arbitrary JSX.

React Stately builds its collection by walking the returned elements and reading a static `getCollectionNode` property off each component type. It never renders them in the usual sense. A `<span>` has no such property, so returning one throws inside `useSelectState` before anything reaches the DOM, with a stack trace that points into React Stately rather than at your code.

The element that does carry it is `Item` from `react-stately`. Rather than leaving consumers to import from a transitive dependency, re-export it under your own name. Create `src/components/Select/Option.ts`:

TYPESCRIPT

```ts
// `Item` is a react-stately *collection* component (identified by a static
// `getCollectionNode`), so it must be re-exported rather than wrapped — a
// function wrapper would break the collection builder that RudiSelect relies on.
export { Item as RudiOption } from 'react-stately'
export type { ItemProps as RudiOptionProps } from 'react-stately'
```

The comment is worth keeping in your own copy. The obvious refactor here is to wrap `Item` in a component that adds a class name or default props, and that refactor silently breaks the collection: the wrapper is a new component type without the static property, so React Stately stops recognizing it as an option. Re-exporting is not laziness, it is the only thing that works.

Add both to `src/index.ts` alongside Select. Without `RudiOption` in the public API, the only way to use `RudiSelect` is to import `Item` from `react-stately` directly, which leaks a dependency the consumer never installed.

Checkbox and CheckboxGroup

`useToggleState` from `react-stately` tracks a single boolean on/off value — it's the state layer for any component that toggles between selected and not selected. Checkbox and Switch both use it.

TSX

```tsx
// src/components/Checkbox/Checkbox.tsx
import { forwardRef } from 'react'
import { useCheckbox } from 'react-aria'
import { useObjectRef } from '@react-aria/utils'
import { useToggleState } from 'react-stately'
import { cn } from '@/utils/cn'
import './checkbox.css'
export interface RudiCheckboxProps {
  children: React.ReactNode
  isSelected?: boolean
  defaultSelected?: boolean
  isIndeterminate?: boolean
  onChange?: (isSelected: boolean) => void
  isDisabled?: boolean
  value?: string
  className?: string
}
export const RudiCheckbox = forwardRef<HTMLInputElement, RudiCheckboxProps>(
  function RudiCheckbox(props, forwardedRef) {
    const ref = useObjectRef(forwardedRef)
    const state = useToggleState(props)
    const { inputProps } = useCheckbox(props, state, ref)
    return (
      <label
        className={cn(
          'rudi-checkbox',
          props.isDisabled && 'rudi-checkbox--disabled',
          props.className,
        )}
      >
        <input {...inputProps} ref={ref} className="rudi-checkbox__input" />
        <span
          className={cn(
            'rudi-checkbox__control',
            state.isSelected && 'rudi-checkbox__control--checked',
            props.isIndeterminate &&
              'rudi-checkbox__control--indeterminate',
```

```
          )}
          aria-hidden="true"
        />
        <span className="rudi-checkbox__label">{props.children}</span>
      </label>
    )
  },
)
```

The hidden native `<input>` provides the actual checkbox behavior. The `rudi-checkbox__control` span is the visual indicator, styled via CSS to show a checkmark, dash (indeterminate), or empty box. The `aria-hidden="true"` on the visual indicator prevents screen readers from announcing it twice (once for the hidden input, once for the visual).

Using `forwardRef` here lets a parent component call `.focus()` on the underlying input directly, which matters when you need to programmatically focus a checkbox, for example after form validation reveals an error. The React Aria hooks expect a `RefObject<T>`, but `forwardRef` gives you a `ForwardedRef<T>`, which can be either an object ref or a callback ref. `useObjectRef` from `@react-aria/utils` normalizes both forms into the `RefObject` the hooks need, so you get full compatibility without extra boilerplate. `@react-aria/utils` is already a transitive dependency of `react-aria`, so no new package is required.

`CheckboxGroup` wraps multiple checkboxes with a group label. It uses `useCheckboxGroupState` from `react-stately`, which manages the array of currently selected values across the group.

TSX

```tsx
// src/components/Checkbox/CheckboxGroup.tsx
import { useCheckboxGroup } from 'react-aria'
import { useCheckboxGroupState } from 'react-stately'
import { cn } from '@/utils/cn'
import './checkbox.css'
export interface RudiCheckboxGroupProps {
  label: string
  description?: string
  value?: string[]
  defaultValue?: string[]
  onChange?: (value: string[]) => void
  isDisabled?: boolean
  errorMessage?: string
  children: React.ReactNode
  className?: string
}
export function RudiCheckboxGroup(props: RudiCheckboxGroupProps) {
```

```
  const state = useCheckboxGroupState(props)
  const { groupProps, labelProps, descriptionProps, errorMessageProps } =
    useCheckboxGroup(props, state)
  return (
    <div
      {...groupProps}
      className={cn('rudi-checkbox-group', props.className)}
    >
      <span {...labelProps} className="rudi-checkbox-group__label">
        {props.label}
      </span>
      {props.children}
      {props.description && !props.errorMessage && (
        <p
          {...descriptionProps}
          className="rudi-checkbox-group__description"
        >
          {props.description}
        </p>
      )}
      {props.errorMessage && (
        <p {...errorMessageProps} className="rudi-checkbox-group__error">
          {props.errorMessage}
        </p>
      )}
    </div>
  )
}
```

Here's the finished Checkbox rendered from the companion repository's Storybook, including the indeterminate state and grouped variants.



LIVE COMPONENT

Checkbox in Storybook

rudiment-ui.netlify.app/?path=/story/components-checkbox--group

RadioGroup

RadioGroup has an important keyboard behavior difference from individual checkboxes: arrow keys move the selection between radio options within the group, while Tab moves focus into and out of the group (not between options). React Aria handles this automatically. `useRadioGroupState` from `react-stately` tracks the currently selected value within the group.

TSX

```tsx
// src/components/RadioGroup/RadioGroup.tsx
import { createContext } from 'react'
import { useRadioGroup } from 'react-aria'
import { useRadioGroupState, type RadioGroupState } from 'react-stately'
import { cn } from '@/utils/cn'
import './radio-group.css'
export const RadioGroupContext = createContext<RadioGroupState | null>(null)
export interface RudiRadioGroupProps {
  label: string
  description?: string
  value?: string
  defaultValue?: string
  onChange?: (value: string) => void
  orientation?: 'horizontal' | 'vertical'
  isDisabled?: boolean
  errorMessage?: string
  children: React.ReactNode
  className?: string
}
export function RudiRadioGroup(props: RudiRadioGroupProps) {
  const state = useRadioGroupState(props)
  const { radioGroupProps, labelProps, descriptionProps, errorMessageProps } =
    useRadioGroup(props, state)
  return (
    <RadioGroupContext.Provider value={state}>
      <div
        {...radioGroupProps}
        className={cn('rudi-radio-group', props.className)}
      >
        <span {...labelProps} className="rudi-radio-group__label">
          {props.label}
        </span>
        <div
          className={cn(
            'rudi-radio-group__options',
            props.orientation === 'horizontal' &&
              'rudi-radio-group__options--horizontal',
          )}
        >
          {props.children}
        </div>
        {props.description && !props.errorMessage && (
          <p {...descriptionProps} className="rudi-radio-group__description">
            {props.description}
          </p>
        )}
        {props.errorMessage && (
          <p {...errorMessageProps} className="rudi-radio-group__error">
            {props.errorMessage}
          </p>
```

```
        )}
      </div>
    </RadioGroupContext.Provider>
  )
}
```

The context is yours to create and provide. This is the one place in the chapter where React Aria does less than you might expect: `useRadio` requires the group's `RadioGroupState` as an argument, and it has no opinion about how the individual radio obtains it. Every implementation ends up building this same bridge, so it is worth being explicit that the `createContext` call and the `Provider` wrapper are load-bearing. Drop either one and each radio reads `null`, `useRadio` throws, and nothing renders.

> **NOTE**
>
> The companion repository ships `RudiRadioGroup` without a matching `Radio` component. There is a minimal one inside the story file for demonstration, and `radio-group.css` carries the styles for it, but nothing is exported. That makes `RudiRadioGroup` unusable from the published package as it stands. Build the `Radio` described below and export it; do not assume it is there because the group is.

The individual `Radio` component reads the state from `RadioGroupContext`, passes it to `useRadio` along with its own props, and follows the same hidden-input-plus-visual-indicator pattern as Checkbox: a visually hidden native `<input type="radio">` carrying the semantics and the focus target, and a styled `<span>` next to it that draws the control.

Both `CheckboxGroup` and `RadioGroup` include a `description` prop and spread `descriptionProps` onto a visible element, following the same pattern established in Chapter 7's Input. The description is hidden when an error is active, so screen readers don't announce both simultaneously.

Here's the finished RadioGroup rendered from the companion repository's Storybook. Use the arrow keys once focus is inside the group to see the keyboard behavior described above.



LIVE COMPONENT

RadioGroup in Storybook

`rudiment-ui.netlify.app/?path=/story/components-radiogroup--with-description`

Switch

Switch is semantically distinct from Checkbox. A checkbox is a selection control ("agree to terms"). A switch is a toggle control ("enable notifications"). The distinction matters for screen reader users: `role="switch"` communicates a different interaction model than `role="checkbox"`.

TSX

```tsx
// src/components/Switch/Switch.tsx
import { forwardRef } from 'react'
import { useSwitch, VisuallyHidden } from 'react-aria'
import { useObjectRef } from '@react-aria/utils'
import { useToggleState } from 'react-stately'
import { cn } from '@/utils/cn'
import './switch.css'
export interface RudiSwitchProps {
  children: React.ReactNode
  isSelected?: boolean
  defaultSelected?: boolean
  onChange?: (isSelected: boolean) => void
  isDisabled?: boolean
  className?: string
}
export const RudiSwitch = forwardRef<HTMLInputElement, RudiSwitchProps>(
  function RudiSwitch(props, forwardedRef) {
    const ref = useObjectRef(forwardedRef)
    const state = useToggleState(props)
    const { inputProps } = useSwitch(props, state, ref)
    return (
      <label
        className={cn(
          'rudi-switch',
          props.isDisabled && 'rudi-switch--disabled',
          props.className,
        )}
      >
        <VisuallyHidden>
          <input {...inputProps} ref={ref} />
        </VisuallyHidden>
        <span
          className={cn(
            'rudi-switch__track',
            state.isSelected && 'rudi-switch__track--on',
          )}
          aria-hidden="true"
        >
          <span className="rudi-switch__thumb" />
        </span>
        <span className="rudi-switch__label">{props.children}</span>
      </label>
```

```
    )
  },
)
```

`VisuallyHidden` is a React Aria utility that hides the native input from sight while keeping it accessible to screen readers. The visible switch track and thumb are styled via CSS to animate between on and off positions.

Here's the finished Switch rendered from the companion repository's Storybook. Toggle it to see the track-and-thumb animation described above.

LIVE COMPONENT
Switch in Storybook
rudiment-ui.netlify.app/?path=/story/components-switch--all-states

What you have now

The core form components are built: Button, IconButton, Input, Select, Checkbox, CheckboxGroup, RadioGroup, Switch. Each uses React Aria for behavior and accessibility, with your token-driven CSS handling the visual layer. You can compose a complete form using these components inside a Stack:

TSX

```tsx
import { RudiSelect } from '@/components/Select/Select'
import { RudiOption } from '@/components/Select/Option'
const roles = [
  { id: 'design', label: 'Design' },
  { id: 'engineering', label: 'Engineering' },
  { id: 'product', label: 'Product' },
]
;<RudiStack space="1.5rem" style={{ maxWidth: '24rem' }}>
  <RudiInput label="Name" isRequired />
  <RudiInput label="Email" type="email" isRequired />
  <RudiSelect label="Role" placeholder="Choose a role" items={roles}>
    {(role) => <RudiOption key={role.id}>{role.label}</RudiOption>}
  </RudiSelect>
  <RudiCheckboxGroup label="Interests">
    <RudiCheckbox value="design">Design</RudiCheckbox>
    <RudiCheckbox value="engineering">Engineering</RudiCheckbox>
    <RudiCheckbox value="product">Product</RudiCheckbox>
  </RudiCheckboxGroup>
  <RudiSwitch>Receive email updates</RudiSwitch>
  <RudiButton variant="primary">Submit</RudiButton>
```

```
</RudiStack>
```

The Select render function returns `RudiOption`, for the collection reason covered earlier. This is the detail that catches people, because returning a `<span>` looks reasonable and fails at runtime rather than at compile time.

Two form components are still missing at this point. `RudiTextarea` follows Input almost exactly, adding `rows` and a `resize` prop. `RudiRadio` is the companion to RadioGroup described above. Chapter 14 covers both alongside the rest of the component set.

Every form element now behaves the same way: a React Aria hook supplies the accessibility layer, your CSS supplies the visuals, and layout primitives from chapters 4 and 5 arrange them on the page without any form-specific layout code. The next chapter steps beyond forms into the overlays and feedback surfaces that sit on top of this interactive layer.

CHAPTER 9

Overlays and feedback

This chapter builds Dialog, Tooltip, Alert, Badge, and Card.



Overlays and feedback components complete the interactive layer. This chapter builds Dialog, Tooltip, Alert, Badge, and Card: modal interactions, informational overlays, status feedback, and content containers.

The first three lean on React Aria. The last two do not, and the shift is worth watching for. Badge and Card render no interactive state and manage no focus, so a React Aria hook would add a dependency and buy nothing. Reaching for the accessibility layer by reflex is a habit worth breaking.

Dialog

The Dialog (modal) is the most complex overlay component. It requires focus trapping (Tab cycles within the dialog, not outside it), scroll locking (the page behind the dialog doesn't scroll), Escape to close, focus restoration (focus returns to the trigger when the dialog closes), and `aria-modal="true"` to tell screen readers that content outside the dialog is inert.

TSX

```tsx
// src/components/Dialog/Dialog.tsx
import { forwardRef, useRef } from 'react'
import {
  useDialog,
  useModalOverlay,
  OverlayContainer,
  usePreventScroll,
  FocusScope,
} from 'react-aria'
import { useObjectRef } from '@react-aria/utils'
import type { OverlayTriggerState } from 'react-stately'
import { cn } from '@/utils/cn'
import './dialog.css'
export interface RudiDialogProps {
  isOpen: boolean
  onClose: () => void
  title: string
  isDismissable?: boolean
  size?: 'sm' | 'md' | 'lg'
  children: React.ReactNode
  className?: string
}
export const RudiDialog = forwardRef<HTMLDivElement, RudiDialogProps>(function RudiDialog(
  {
    isOpen,
    onClose,
    title,
    isDismissable = true,
```

```
    size = 'md',
    children,
    className,
  }: RudiDialogProps,
  ref,
) {
  const underlayRef = useRef<HTMLDivElement>(null)
  const dialogRef = useObjectRef(ref)
  usePreventScroll({ isDisabled: !isOpen })
  // Create a state object compatible with useModalOverlay
  const state: OverlayTriggerState = {
    isOpen,
    close: onClose,
    open: () => {},
    toggle: () => {},
    setOpen: () => {},
  }
  // dialogRef must point to the modal element (not the underlay) so that
  // useModalOverlay can correctly detect clicks outside the dialog panel.
  const { modalProps, underlayProps } = useModalOverlay(
    { isDismissable },
    state,
    dialogRef,
  )
  const { dialogProps, titleProps } = useDialog({}, dialogRef)
  if (!isOpen) return null
  return (
    <OverlayContainer>
      <div
        {...underlayProps}
        ref={underlayRef}
        className="rudi-dialog__overlay"
      >
        {/* FocusScope provides tab trapping and restores focus on close */}
        <FocusScope contain restoreFocus autoFocus>
          <div
            {...modalProps}
            {...dialogProps}
            ref={dialogRef}
            className={cn(
              'rudi-dialog',
              `rudi-dialog--${size}`,
              className,
            )}
          >
            <h2 {...titleProps} className="rudi-dialog__title">
              {title}
            </h2>
            <div className="rudi-dialog__body">{children}</div>
          </div>
        </FocusScope>
```

```
      </div>
    </OverlayContainer>
  )
})
```

`useModalOverlay` handles focus trapping and Escape dismissal. `usePreventScroll` locks body scrolling. `OverlayContainer` from React Aria renders the dialog into a portal (appended to the document body), which prevents z-index stacking context issues.

Dialog uses `forwardRef` so consumers can hold a ref to the dialog element, which is useful for programmatic focus management or measuring position. The internal `useDialog` hook requires a `RefObject<HTMLDivElement>`, not the looser `ForwardedRef` type that `forwardRef` provides. `useObjectRef` from `@react-aria/utils` bridges the two: it normalizes a forwarded ref (which may be a callback ref, a ref object, or null) into a stable `RefObject`, and syncs them so both the consumer's ref and `useDialog`'s ref point to the same element.

The dialog is controlled-only (`isOpen` and `onClose` are required props). This is a deliberate choice. Uncontrolled dialogs (that manage their own open state) prevent the consumer from coordinating dialog visibility with the rest of their application state.

LIVE COMPONENT
Dialog in Storybook
rudiment-ui.netlify.app/?path=/story/components-dialog--default

Tooltip

`mergeProps` from `react-aria` merges multiple props objects, combining event handlers by chaining them rather than overwriting — so if both objects have an `onClick`, both fire. It's used here to attach the trigger props to the trigger element without discarding any existing handlers.

`React.cloneElement` creates a new React element by copying an existing one and merging in additional props. The TooltipTrigger uses it to inject the trigger and tooltip props into the caller's elements without requiring consumers to forward those props themselves.

TSX

```tsx
// src/components/Tooltip/Tooltip.tsx
import React, { forwardRef, useRef } from 'react'
import { useTooltipTrigger, useTooltip as useTooltipAria } from 'react-aria'
import { useTooltipTriggerState } from 'react-stately'
import { cn } from '@/utils/cn'
import './tooltip.css'
export interface RudiTooltipTriggerProps {
  delay?: number
  closeDelay?: number
  children: [React.ReactElement, React.ReactElement]
}
export const RudiTooltipTrigger = forwardRef<HTMLSpanElement, RudiTooltipTriggerProps>(
  function RudiTooltipTrigger({ delay = 500, closeDelay = 0, children }, ref) {
    const state = useTooltipTriggerState({ delay, closeDelay })
    const triggerRef = useRef<HTMLElement>(null)
    const { triggerProps, tooltipProps: triggerTooltipProps } =
      useTooltipTrigger({ delay, closeDelay }, state, triggerRef)
    const [trigger, tooltip] = children
    const {
      'aria-describedby': ariaDescribedBy,
      tabIndex: _tabIndex,
      ...eventProps
    } = triggerProps
    return (
      <span ref={ref} className="rudi-tooltip-trigger" {...eventProps}>
        {React.cloneElement(
          trigger as React.ReactElement<Record<string, unknown>>,
          {
            ref: triggerRef,
            'aria-describedby': ariaDescribedBy,
          },
        )}
        {state.isOpen && React.cloneElement(tooltip, triggerTooltipProps)}
      </span>
    )
  },
)
export interface RudiTooltipProps {
  children: React.ReactNode
  className?: string
}
export const RudiTooltip = forwardRef<
  HTMLSpanElement,
  RudiTooltipProps & Record<string, unknown>
>(function RudiTooltip({ children, className, ...props }, ref) {
  const { tooltipProps } = useTooltipAria(props)
  return (
    <span
      {...tooltipProps}
      ref={ref}
      className={cn('rudi-tooltip', className)}
```

```
      role="tooltip"
    >
      {children}
    </span>
  )
})
```

The tooltip appears on hover (after the configured delay) and on focus. It hides on pointer leave, blur, Escape, or scroll. React Aria manages the timing and the `aria-describedby` relationship between the trigger and the tooltip content. Both `TooltipTrigger` and `Tooltip` use `forwardRef`: `TooltipTrigger` exposes the outer wrapper span (useful for positioning logic), and `Tooltip` exposes the tooltip element itself.



LIVE COMPONENT

Tooltip in Storybook

rudiment-ui.netlify.app/?path=/story/components-tooltip--multiple-tooltips

Alert

Alert needs no React Aria at all. Its accessibility comes entirely from one attribute and a live region the browser already provides:

TSX

```tsx
// src/components/Alert/Alert.tsx
import { useState } from 'react'
import { cn } from '@/utils/cn'
import { RudiIcon } from '@/components/Icon/Icon'
import './alert.css'
export interface RudiAlertProps {
  variant: 'info' | 'success' | 'warning' | 'error'
  title?: string
  /** Iconify icon name shown beside the content. */
  icon?: string
  isPolite?: boolean
  dismissible?: boolean
  onDismiss?: () => void
  children: React.ReactNode
  className?: string
}
export function RudiAlert({
  variant,
  title,
  icon,
```

```
  isPolite = false,
  dismissible = false,
  onDismiss,
  children,
  className,
}: RudiAlertProps) {
  const [dismissed, setDismissed] = useState(false)
  if (dismissed) return null
  return (
    <div
      role={isPolite ? 'status' : 'alert'}
      className={cn(
        'rudi-alert',
        `rudi-alert--${variant}`,
        icon && 'rudi-alert--has-icon',
        dismissible && 'rudi-alert--dismissible',
        className,
      )}
    >
      {icon && (
        <RudiIcon icon={icon} size="md" className="rudi-alert__icon" aria-hidden="true" />
      )}
      <div className="rudi-alert__body">
        {title && <p className="rudi-alert__title">{title}</p>}
        <div className="rudi-alert__content">{children}</div>
      </div>
      {dismissible && (
        <button
          type="button"
          className="rudi-alert__dismiss"
          aria-label="Dismiss alert"
          onClick={() => {
            setDismissed(true)
            onDismiss?.()
          }}
        >
          <RudiIcon icon="mdi:close" size="sm" aria-hidden="true" />
        </button>
      )}
    </div>
  )
}
```

`role="alert"` triggers an assertive announcement in screen readers: the content is read immediately, interrupting whatever the screen reader was doing. `role="status"`, via `isPolite`, triggers a polite announcement read at the next natural pause. Use assertive for errors that need immediate attention, polite for success messages and informational updates. Getting this backwards is a

common and genuinely disruptive mistake: an assertive announcement for "Settings saved" cuts off whatever the user was listening to.

Three structural details matter more than they look.

The `rudi-alert__body` wrapper is not decorative. With an icon on one side and a dismiss button on the other, the alert is a three-column grid, and title plus content have to occupy one cell together. Flattening them into direct children of `.rudi-alert` puts the title and the content in separate grid cells and the layout falls apart. The `--has-icon` and `--dismissible` modifiers are what switch `grid-template-columns` between the three arrangements.

Dismissal is local state. `dismissible` hides the alert from inside the component and calls `onDismiss` so the parent can react. Requiring the consumer to own visibility would mean four lines of state management at every call site for the common case. Consumers who do want to own it can ignore `dismissible` and unmount the alert themselves.

The dismiss button needs its own label. The icon inside it is `aria-hidden`, so without `aria-label="Dismiss alert"` the button has no accessible name at all. This is the same argument as IconButton's required `aria-label` in chapter 7, and the reason that prop was made non-optional there.



LIVE COMPONENT

Alerts in Storybook

rudiment-ui.netlify.app/?path=/story/components-alert--all-variants

Badge

Badge is a presentational `<span>`. No hooks, no state, no ref forwarding:

```tsx
export interface RudiBadgeProps {
  variant?: 'default' | 'success' | 'warning' | 'error' | 'info'
  size?: 'sm' | 'md'
  /** Renders as a bare dot, suppressing children. */
  dot?: boolean
  children?: React.ReactNode
  className?: string
}
```

Defaults are `variant="default"`, `size="md"`, `dot={false}`.

The `dot` mode is the one surprising piece: it suppresses children entirely rather than shrinking them, rendering `{!dot && children}`. A dot badge is a status indicator with no text, so passing children alongside `dot` is a mistake, and dropping them is more honest than rendering a badge too small to read. Note that `children` is optional precisely because of this mode.

There is no `aria-label` prop. A badge is decorative or it duplicates adjacent text, and a bare dot conveys nothing on its own, so a consumer who needs it announced should label the surrounding element. Adding a label prop would invite the pattern where every badge gets a redundant announcement.

Card

Card is a compound component, and it is the first one in the guide:

```tsx
export interface RudiCardProps {
  variant?: 'default' | 'outlined' | 'elevated'
  padding?: 'none' | 'sm' | 'md' | 'lg'
  children: React.ReactNode
  className?: string
}
// …CardRoot, CardHeader, CardBody, and CardFooter defined above…
export const RudiCard = Object.assign(CardRoot, {
  Header: CardHeader,
  Body: CardBody,
  Footer: CardFooter,
})
```

Which gives consumers this:

```tsx
<RudiCard variant="outlined">
  <RudiCard.Header>
    <RudiHeading level={3}>Monthly report</RudiHeading>
  </RudiCard.Header>
  <RudiCard.Body>
    <RudiText>Revenue is up 12% over the previous period.</RudiText>
  </RudiCard.Body>
  <RudiCard.Footer>
    <RudiButton variant="secondary">Export</RudiButton>
  </RudiCard.Footer>
</RudiCard>
```

`Object.assign` attaches the subcomponents as properties of the root. The alternative, exporting `RudiCard`, `RudiCardHeader`, `RudiCardBody`, and `RudiCardFooter` separately, works identically and puts four names into the consumer's import statement where one would do. The dotted form also makes the relationship obvious at the call site: `RudiCard.Footer` outside a `RudiCard` reads as wrong.

The `padding` prop is plumbed through a local custom property rather than applied directly. `.rudi-card--padding-md` sets `--card-padding`, and the header, body, and footer rules each read it. That way one prop on the root controls the inset on all three regions without any of them knowing what the others do.

Card takes no React Aria and has no `as` prop. It is a container, so there is no press behavior to normalize and no focus to manage. If you need a clickable card, put a real link or button inside it rather than making the container itself interactive: a clickable `<div>` needs a role, a tab index, and key handlers to reach parity with an element the browser already ships.

What you have now

The overlay and feedback layer is complete: Dialog, Tooltip, Alert, Badge, and Card, on top of the form components from chapters 7 and 8 and the layout and typography layers before them.

Two claims worth stating precisely, because both are easy to overstate.

Not every component uses React Aria. Dialog, Tooltip, Button, Input, IconButton, Select, Checkbox, RadioGroup, and Switch do, because each manages focus, keyboard interaction, or ARIA state that is genuinely hard to get right. Alert, Badge, and Card do not, because they manage none of it.

Not every component uses `forwardRef`. Checkbox, Switch, Dialog, Tooltip, and Card forward refs because a consumer has a plausible reason to reach the underlying element. Select, CheckboxGroup, RadioGroup, Alert, and Badge are plain functions. A group component's ref target is ambiguous, and a badge's is useless. Adding `forwardRef` everywhere by reflex costs a wrapper and communicates a capability nobody needs.

This is roughly a third of what the companion repository ships. Chapters 14 through 16 cover the rest: the presentational and navigation set, the charts, and the drag-and-drop board.

The next chapter configures Storybook to document and demonstrate the system.

CHAPTER 10

Storybook as your documentation layer

This chapter configures Storybook 10 and establishes the story conventions that make the documentation useful.



A component library without documentation is a collection of files. Storybook turns your components into a browsable, interactive catalog that serves as both the development environment and the buyer-facing documentation. This chapter configures Storybook 10 and establishes the story conventions that make the documentation useful.

Installation

```bash
npm create storybook@latest
```

This guide uses Storybook 10. `@latest` installs whatever version is current at the time you follow the guide — if the current version is not Storybook 10, the configuration format may differ from the examples below.

The CLI detects your Vite + React setup and asks a few questions. When prompted for configuration, choose **Recommended** — this installs the a11y, docs, and Vitest addons along with their dependencies, including Playwright. When the CLI asks whether to install Playwright with Chromium, choose **Yes**.

The recommended configuration installs these devDependencies:

- `storybook`, `@storybook/react-vite`
- `@chromatic-com/storybook`, `@storybook/addon-vitest`, `@storybook/addon-a11y`, `@storybook/addon-docs`
- `vitest`, `playwright`, `@vitest/browser-playwright`, `@vitest/coverage-v8`

Add one more that the CLI does not install:

```bash
npm install -D remark-gfm
```

The CLI generates `.storybook/main.ts`, `.storybook/preview.ts`, and `.storybook/vitest.setup.ts`, and updates `vite.config.ts`. Rename `preview.ts` to `preview.tsx` before you start: the decorators below return JSX, and the file will not compile otherwise.

Configuration

```typescript
// .storybook/main.ts
```

```
import type { StorybookConfig } from '@storybook/react-vite'
import type { InlineConfig } from 'vite'
import remarkGfm from 'remark-gfm'
const config: StorybookConfig = {
  stories: ['../src/**/*.mdx', '../src/**/*.stories.@(ts|tsx)'],
  addons: [
    '@chromatic-com/storybook',
    '@storybook/addon-vitest',
    '@storybook/addon-a11y',
    {
      name: '@storybook/addon-docs',
      options: {
        mdxPluginOptions: {
          mdxCompileOptions: { remarkPlugins: [remarkGfm] },
        },
      },
    },
  ],
  framework: '@storybook/react-vite',
  async viteFinal(config: InlineConfig) {
    config.server ??= {}
    config.server.headers = {
      ...config.server.headers,
      'X-Frame-Options': 'ALLOWALL',
    }
    return config
  },
}
export default config
```

Three deviations from the generated file, each fixing something that otherwise fails quietly.

The `stories` array includes `../src/**/*.mdx`. Without it, Storybook finds your `.stories.tsx` files and silently ignores every hand-written MDX documentation page, so the entire Getting Started section of your sidebar never appears. Nothing errors; the pages are simply not there.

`@storybook/addon-docs` is registered in object form so it can take `remarkPlugins: [remarkGfm]`. MDX does not support GitHub-flavored Markdown tables out of the box, and documentation pages are mostly prop tables and token tables. Leave this out and those tables render as literal pipe characters.

`viteFinal` sets `X-Frame-Options: ALLOWALL` on the dev server so a running Storybook can be embedded in an iframe. That is what makes the live component previews in a guide like this one possible. Drop it if you never plan to embed.

TSX

```tsx
// .storybook/preview.tsx
import { useEffect } from 'react'
import type { Preview } from '@storybook/react'
import { useArgs } from 'storybook/preview-api'
import { create } from 'storybook/theming/create'
import '../src/app.css'
const docsTheme = create({ base: 'light' })
const preview: Preview = {
  parameters: {
    docs: { theme: docsTheme },
    options: {
      categorySort: {
        order: ['Appearance', 'Content', 'Layout', 'State', 'Events'],
      },
      storySort: {
        order: [
          'Getting Started',
          [
            'Introduction',
            'Accessibility',
            'Token Architecture',
            'Layout Primitives',
            'Typography',
            'Component Guidelines',
            'Theming',
          ],
          'Components',
          'Layouts',
          'Typography',
          'Examples',
        ],
      },
    },
    controls: {
      matchers: {
        color: /(background|color)$/i,
        date: /Date$/i,
      },
    },
    a11y: {},
  },
  globalTypes: {
    theme: {
      description: 'Color theme',
      toolbar: {
        title: 'Theme',
        icon: 'paintbrush',
        items: [
          { value: 'light', title: 'Default Light', icon: 'sun' },
          { value: 'dark', title: 'Default Dark', icon: 'moon' },
        ],
```

```
        dynamicTitle: true,
      },
    },
  },
  initialGlobals: {
    theme: 'light',
  },
  decorators: [
    (Story, context) => {
      const theme = context.globals.theme || 'light'
      useEffect(() => {
        document.documentElement.setAttribute('data-theme', theme)
        return () => document.documentElement.removeAttribute('data-theme')
      }, [theme])
      return (
        <div
          style={{
            padding: '2rem',
            backgroundColor: 'var(--rudi-color-background-surface)',
            color: 'var(--rudi-color-text-default)',
            fontFamily: 'var(--rudi-font-family-sans)',
          }}
        >
          <Story />
        </div>
      )
    },
  ],
}
export default preview
```

The theme decorator

The decorator sets a `data-theme` attribute on `document.documentElement`. It does not add a class, and it does not put the attribute on the wrapper `<div>`.

Both alternatives are tempting and both are wrong. Theme CSS is generated by Style Dictionary under `[data-theme="dark"]` selectors, so a class never matches anything. And portalled content, which includes every dialog, tooltip, popover, and select menu from chapters 8 and 9, renders outside the decorator's wrapper into `document.body`. An attribute on the wrapper themes the page and leaves every overlay in the default theme, which is a confusing bug to chase. On the root element, everything inherits.

The cleanup function removes the attribute rather than resetting it to `light`. That matches how the library expects to be used: there is no `data-theme="light"`, the default theme lives in `:root`, and light mode means no attribute at all.

The wrapper is styled with inline `var(--rudi-*)` references rather than utility classes. `bg-surface` and `text-text-default` do not exist, because chapter 3 deliberately did not map tokens into a Tailwind `@theme` block.

> **NOTE**
> This toolbar has two entries. Chapter 13 builds six more themes and extends the list to eight. Until then the toggle switches to a dark theme whose tokens do not exist yet, so it will appear to do nothing. That is expected, not a misconfiguration.

The docs theme and the controlled-args decorator

`parameters.docs.theme` is set to a light Storybook theme explicitly. Without it the "Show code" source block falls back to a dark syntax palette, which renders light-on-light against the docs page background and is genuinely unreadable. One line, for a bug that otherwise reads as a rendering glitch.

There is a second decorator worth adding once you have form components, and it prevents a problem that will otherwise make half your stories look broken.

Storybook renders components with the args from `meta.args`. When one of those args is a controlled prop such as `value`, `isSelected`, or `selectedKey`, the component becomes controlled, and a controlled input whose `onChange` never updates the arg cannot respond to typing or clicking. The checkbox refuses to check. Anyone evaluating the library concludes the component is broken.

The fix is a decorator that detects controlled props and wraps `onChange` to write the new value back:

```tsx
(Story) => {
  const [args, updateArgs] = useArgs<Record<string, unknown>>()
  const hasIsSelected = 'isSelected' in args
  const hasValue = 'value' in args
  if (!hasIsSelected && !hasValue) return <Story />
  const original = args.onChange as ((v: unknown) => void) | undefined
  return (
    <Story
      args={{
        ...args,
        onChange: (newValue: unknown) => {
          if (hasIsSelected) updateArgs({ isSelected: newValue })
          if (hasValue) updateArgs({ value: newValue })
          original?.(newValue)
        },
      }}
    />
```

```
  )
}
```

The companion repository's version also handles `selectedKey` and `onSelectionChange` for Select. The early return matters: without it every story gets an injected `onChange`, controlled or not.

Two more configuration files

`.storybook/manager.ts` themes the Storybook shell itself, which is what makes a deployed Storybook look like your documentation rather than a default install:

```typescript
import { addons } from 'storybook/manager-api'
import { create } from 'storybook/theming/create'
addons.setConfig({
  theme: create({ base: 'light', brandTitle: 'Rudiment UI' }),
})
```

`.storybook/preview-head.html` is where font `<link>` tags and docs-page CSS overrides belong. Worth knowing: no fonts ship with the library. If your themes reference IBM Plex Sans, or the Rubik and Nunito faces that chapter 13's themes use, and nothing loads them, text falls back to the system sans-serif. The result looks subtly off rather than obviously broken, which makes it a slow bug to notice. Load them here, or from the consuming application.

Story organization

The sidebar has five top-level sections, and their order is pinned by `storySort` rather than left alphabetical:

Section	Title prefix	Contents
Getting Started	`Getting Started/Introduction`	Hand-written MDX: installation, accessibility, tokens, theming
Components	`Components/Button`, `Components/Charts/BarChart`	One entry per UI component; nested groups allowed
Layouts	`Layouts/Stack`, `Layouts/Sidebar`	One entry per layout primitive
Typography	`Typography/Heading`, `Typography/Text`	The three typography components

Section	Title prefix	Contents
Examples	`Examples/CRM/Dashboard`, `Examples/eCommerce/Checkout`	Composed pages demonstrating the full system

The `title` field in each story's meta controls placement, and a slash creates nesting, so `Components/Charts/BarChart` becomes a subgroup.

Pinning the order earns its config. Alphabetically, Components sorts before Getting Started, so the first thing a new reader sees is a props table rather than an introduction. `storySort` also accepts a nested array for pages inside a group, which is how the Getting Started sequence above is fixed.

`categorySort` does the same job inside the controls panel, ordering the `table: { category }` values you started assigning in chapter 4. Without it, categories appear in whatever order Storybook encounters them, which differs per component.

What every story file includes

For layout primitives:

1. **Default**: The primitive with token defaults and placeholder children.
2. **Custom props**: The primitive with explicit prop values demonstrating customization.
3. **Composed**: The primitive nested inside or containing other primitives.

For UI components:

1. **Default**: The component in its default state.
2. **Variants**: One story per visual variant.
3. **States**: Stories for disabled, loading, error, and any other states.
4. **Composed**: The component inside a layout primitive, demonstrating a realistic arrangement.

Beyond the stories themselves, every meta object carries four things:

- `tags: ['autodocs']`, which generates a documentation page from the metadata and TypeScript prop types.
- `satisfies Meta<typeof RudiX>` paired with `type Story = StoryObj<typeof meta>`, which is what makes story args type-check against the real props instead of `any`.

- `parameters.docs.description.component`, the paragraph at the top of the generated page.
- A `description` and a `table: { category }` on every argType.

One exception to `autodocs` is worth noting: internal components that exist only to compose others, such as an application shell or a top bar, are usually better left out of the generated documentation. Give them stories for development and skip the tag.

The example pages

Composed example pages are the closest thing to an integration test your documentation has, and they are what someone evaluating the library actually looks at.

The instinct is to build one kitchen-sink page containing every component. Resist it. A page that uses everything resembles nothing, and it answers the wrong question. Adopters are not asking whether the library can render 40 components at once; they are asking whether it looks right for the thing they are building.

Organize by product domain instead, several pages each. The companion repository ships four sets:

Domain	Pages
CRM	Dashboard, Sales Dashboard, Finance Dashboard, Customers, Employees, Team Management
eCommerce	Landing Page, Category Page, Product Page, Checkout, Order Confirmation, Empty Cart
Project Management	Dashboard, Board, Backlog, Epics, Stories, Burndown, Profile
Biotech	Landing Page, About Us, Products & Services, Investors, Contact Us, Terms & Privacy

Each domain gets a `shared.tsx` holding its sample data and local presentational helpers, so the story files stay readable. Build these after the components are complete. They are where API gaps become obvious, and nearly every one produces a small fix somewhere upstream.

Deploying Storybook

The Storybook CLI adds the build script for you:

```json
{
  "scripts": {
    "dev": "storybook dev -p 6006",
    "build-storybook": "storybook build"
  }
}
```

Note the hyphen. `build-storybook` is the CLI's own convention and differs from the colon-separated `build:tokens` you added in chapter 3. It is tempting to rename it for consistency; leave it, because Storybook's own tooling and most deployment presets look for this exact name.

Then build the static site:

```bash
npm run build-storybook
```

The output lands in `storybook-static`, a plain static site you can deploy anywhere. Vercel and Netlify both deploy it from a Git repository with no extra configuration: point the build command at `npm run build-storybook` and the publish directory at `storybook-static`.

One deployment detail that is easy to miss. `npm run build-storybook` does not build your tokens. If `tokens/build/` is stale or missing on a clean CI checkout, Storybook builds successfully and every component renders unstyled, because the custom properties resolve to nothing. Either commit `tokens/build/` as chapter 3 suggested, or make the deploy command `npm run build:tokens && npm run build-storybook`.

The deployed URL becomes the public demo linked from your README and anywhere else you point people.

What you have now

A configured Storybook with accessibility audits on every story, a theme toggle wired to `data-theme`, a pinned sidebar order, generated documentation pages, and composed example pages organized by product domain. Every component

and layout primitive has stories that document its API and demonstrate its behavior.

The stories are written but nothing verifies them yet. The next chapter adds the test layer, including a project that renders every story in a real browser and runs axe against it.

CHAPTER 11

Testing accessible components

This chapter covers the testing tools and what to test for each component type.



Tests give you confidence to change things. In a component library, that confidence matters more than usual, because a regression in one component affects every consumer of the library. This chapter covers the testing tools and what to test for each component type.

Install dependencies

This installs the packages required for the test files introduced in Chapter 4. The Stack tests created there can now be run.

`vitest` was already installed in Chapter 10 as part of the Storybook setup. Install only the remaining dependencies:

```bash
npm install -D @testing-library/react @testing-library/user-event @testing-library/jest-dom jsdom vitest-axe
```

Test configuration

There is no `vitest.config.ts`. Vitest reads its configuration from `vite.config.ts`, and keeping it there means the test run inherits the same aliases, plugins, and JSX handling as the build. Two configs is two places for `@/` to resolve differently.

Add a `test` block to the `vite.config.ts` you created in chapter 2:

```typescript
/// <reference types="vitest/config" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import { resolve } from 'path'
import path from 'node:path'
import { fileURLToPath } from 'node:url'
import { storybookTest } from '@storybook/addon-vitest/vitest-plugin'
import { playwright } from '@vitest/browser-playwright'
const dirname =
  typeof __dirname !== 'undefined'
    ? __dirname
    : path.dirname(fileURLToPath(import.meta.url))
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src'),
```

```
    },
  },
  test: {
    projects: [
      {
        extends: true,
        test: {
          name: 'unit',
          globals: true,
          environment: 'jsdom',
          setupFiles: ['src/test-setup.ts'],
          include: ['src/**/*.test.{ts,tsx}'],
        },
      },
      {
        extends: true,
        plugins: [
          storybookTest({ configDir: path.join(dirname, '.storybook') }),
        ],
        test: {
          name: 'storybook',
          browser: {
            enabled: true,
            headless: true,
            provider: playwright({}),
            instances: [{ browser: 'chromium' }],
          },
          setupFiles: ['.storybook/vitest.setup.ts'],
        },
      },
    ],
  },
})
```

Two projects, not one

The `unit` project is the conventional one: jsdom, your `.test.tsx` files, fast.

The `storybook` project is the one that does the work you would otherwise never get around to. It launches headless Chromium and renders every story you wrote in chapter 10, running the a11y addon against each. Seventy-five story files become seventy-five smoke tests plus an accessibility audit, for the cost of the config block above.

That matters more than it sounds. Hand-written unit tests cover the components you remembered to write tests for, and every library has components nobody got to. The story-based project covers everything that has a story, which in practice is everything.

`extends: true` on each project pulls in the top-level `plugins` and `resolve` so both share the React plugin and the `@/` alias.

The `dirname` shim exists because `__dirname` is unavailable in an ES module, and `storybookTest` needs an absolute path to the Storybook config directory.

`npm test` runs both projects, which means it needs a browser:

```bash
npx playwright install chromium
```

If you skip that, the unit project passes and the storybook project fails with a browser launch error that does not obviously say "run this command."

Add the scripts:

```json
{
  "scripts": {
    "test": "vitest",
    "test:coverage": "vitest --coverage",
    "test:watch": "vitest --watch"
  }
}
```

Setup files and type shims

Create `src/test-setup.ts`:

```typescript
import '@testing-library/jest-dom/vitest'
import 'vitest-axe/extend-expect'
import * as axeMatchers from 'vitest-axe/matchers'
import { expect } from 'vitest'
expect.extend(axeMatchers)
```

The last two lines are the ones people leave out, and leaving them out produces a confusing failure. Under Vitest 4 the `extend-expect` side-effect import alone does not register the axe matchers, so every `expect(await axe(container)).toHaveNoViolations()` fails with "toHaveNoViolations is not a function" while the accessibility itself is fine.

Two type declaration files are also required, and both are pure TypeScript plumbing with no runtime effect.

`src/vitest-axe.d.ts` teaches TypeScript about the matcher you just registered:

```typescript
```

```
import type { AxeMatchers } from 'vitest-axe/matchers'
declare module '@vitest/expect' {
  interface Assertion extends AxeMatchers {}
  interface AsymmetricMatchersContaining extends AxeMatchers {}
}
```

`vitest.shims.d.ts` at the project root does the same for browser mode:

```
/// <reference types="@vitest/browser-playwright" />
```
TYPESCRIPT

Without these, the tests run green and `npx tsc --noEmit` fails, which is the worst combination because CI usually runs both.

Finally, add `"types": ["vitest/globals"]` to `compilerOptions` in `tsconfig.json`. That is what lets `describe`, `it`, `expect`, and `vi` be used without importing them, which the `globals: true` setting enables at runtime.

The two categories of tests

Layout primitive tests verify that the React component correctly applies the CSS classes and custom properties that the CSS depends on. They don't test the visual layout itself (that's the browser's job).

UI component tests verify behavior: keyboard interaction, ARIA attributes, focus management, and the absence of axe-core violations. They test that the component is usable by keyboard and screen reader users, not just mouse users.

Testing layout primitives

Layout primitive tests confirm:

1. The component renders its children.
2. The component applies the correct CSS class.
3. The component merges a custom `className`.
4. Props set inline custom properties.
5. Omitted props don't set inline styles; the CSS token fallback applies instead.
6. The `as` prop changes the rendered element.
7. Boolean props apply modifier classes.

The Stack tests from Chapter 4 demonstrate this pattern, and every layout primitive follows the same structure with its own additions. Sidebar checks that it received exactly two children. Switcher checks the threshold custom property. Do not write a test for Switcher's `limit` prop: chapter 5 covered why it is a stub, and a test asserting behavior that was never implemented is worse than no test.

Testing UI components

UI component tests have more to verify. Here's the standard checklist, followed by an example:

Rendering: The component is findable by role and accessible name.

Keyboard activation: The component responds to the correct keys (Enter, Space, Arrow keys, Escape).

Disabled state: The component has `aria-disabled="true"` and does not respond to interaction.

Error state: The component has `aria-invalid="true"` and the error message is linked via `aria-describedby`.

Focus behavior: Focus moves to the expected element on interaction (for example, into the dialog on open, back to the trigger on close).

Loading state: The component sets `aria-busy="true"`, does not respond to interaction, and renders a visible loading indicator.

Ref forwarding: The ref reaches the underlying DOM element so consumers can control focus programmatically.

axe-core audit: No automated accessibility violations.

TYPESCRIPT

```
// src/components/Input/Input.test.tsx
import { createRef } from 'react';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { axe } from 'vitest-axe';
import { describe, it, expect } from 'vitest';
import { RudiInput } from './Input'
describe('Input', () => {
  it('renders with its label as the accessible name', () => {
    render(<RudiInput label="Email" />);
    expect(screen.getByRole('textbox', { name: 'Email' })).toBeInTheDocument();
  });
  it('accepts text input', async () => {
```

```
    const onChange = vi.fn();
    render(<RudiInput label="Email" onChange={onChange} />);
    await userEvent.type(screen.getByRole('textbox'), 'hello');
    expect(onChange).toHaveBeenLastCalledWith('hello');
  });
  it('shows the description and links it via aria-describedby', () => {
    render(<RudiInput label="Email" description="Your work email" />);
    const input = screen.getByRole('textbox');
    const description = screen.getByText('Your work email');
    expect(input).toHaveAttribute('aria-describedby',
   expect.stringContaining(description.id));
  });
  it('shows the error message and sets aria-invalid', () => {
    render(<RudiInput label="Email" errorMessage="Required field" />);
    const input = screen.getByRole('textbox');
    expect(input).toHaveAttribute('aria-invalid', 'true');
    expect(screen.getByText('Required field')).toBeInTheDocument();
  });
  it('hides the description when an error message is present', () => {
    render(<RudiInput label="Email" description="Your work email"
   errorMessage="Required" />);
    expect(screen.queryByText('Your work email')).not.toBeInTheDocument();
  });
  it('marks the input as required', () => {
    render(<RudiInput label="Email" isRequired />);
    expect(screen.getByRole('textbox')).toHaveAttribute('aria-required', 'true');
  });
  it('disables the input', () => {
    render(<RudiInput label="Email" isDisabled />);
    expect(screen.getByRole('textbox')).toBeDisabled();
  });
  it('forwards ref to the underlying input', () => {
    const ref = createRef<HTMLInputElement>();
    render(<RudiInput label="Email" ref={ref} />);
    expect(ref.current).toBe(screen.getByRole('textbox'));
  });
  it('has no accessibility violations', async () => {
    const { container } = render(<RudiInput label="Email" />);
    expect(await axe(container)).toHaveNoViolations();
  });
  it('has no accessibility violations in error state', async () => {
    const { container } = render(<RudiInput label="Email" errorMessage="Required" />);
    expect(await axe(container)).toHaveNoViolations();
  });
});
```

The Button loading state tests follow the same pattern. They verify the ARIA contract and the absence of interaction side effects:

TYPESCRIPT

```
// src/components/Button/Button.test.tsx (loading state tests)
it('sets aria-busy when loading', () => {
  render(<RudiButton isLoading>Save</RudiButton>);
  expect(screen.getByRole('button', { name: 'Save' })).toHaveAttribute('aria-busy',
   'true');
});
it('does not call onPress while loading', async () => {
  const onPress = vi.fn();
  render(<RudiButton isLoading onPress={onPress}>Save</RudiButton>);
  await userEvent.click(screen.getByRole('button'));
  expect(onPress).not.toHaveBeenCalled();
});
it('renders a visible loading indicator', () => {
  render(<RudiButton isLoading>Save</RudiButton>);
  expect(screen.getByRole('button')).toContainElement(screen.getByRole('img', { hidden:
   true }));
});
it('has no accessibility violations while loading', async () => {
  const { container } = render(<RudiButton isLoading>Save</RudiButton>);
  expect(await axe(container)).toHaveNoViolations();
});
```

Notice the testing approach: assertions lean on accessible queries (`getByRole`, `getByText`) and ARIA attributes (`aria-invalid`, `aria-describedby`, `aria-required`) rather than on markup structure. Refactor the component's internals and these tests keep passing, as long as the behavior and the accessibility contract hold.

That principle has a limit, and it is worth stating rather than pretending otherwise. Some assertions do need to reach for class names, because the class name *is* the contract between the component and its stylesheet:

```
expect(container.firstChild).toHaveClass('rudi-stack--split')
```
TSX

Nothing in the accessibility tree distinguishes a Stack with `splitAfter` from one without it. If `splitAfter` silently stopped emitting `rudi-stack--split`, a role-only test suite would stay green while the layout broke for every consumer. The same applies to Button's icon slots: `button.querySelectorAll('.rudi-button__icon')` is the only way to assert that `iconBefore` rendered and that loading state removed it.

The useful rule is narrower than "never query by class." Query by role for anything a user perceives, and by class only for the contract your own CSS depends on.

What to actually avoid is asserting on incidental structure, such as depth of nesting or wrapper elements that exist only for layout.

Running tests

```bash
npm test                                                # both projects
npm run test:watch                                      # re-run on change
npm run test:coverage                                   # coverage report
npx vitest run --project unit                           # jsdom only, no browser
npx vitest run --project storybook                      # story smoke tests only
npx vitest run --project unit src/components/Button/Button.test.tsx
npx vitest run --project unit -t "sets aria-busy"       # single test by name
```

Reach for `--project unit` while iterating. A bare `npx vitest Input` matches files in both projects and boots Chromium to do it, which turns a two-second feedback loop into a fifteen-second one.

Coverage targets

Category	Target
Layout primitives	Every prop, class application, and custom property injection
UI components	Every keyboard path, every ARIA attribute, every state
axe-core, unit	A `toHaveNoViolations` assertion in every component's test file
axe-core, stories	Automatic, via the browser project, for anything with a story
Visual regression	Not covered; the browser project renders but does not compare

Full coverage of accessible behavior is worth more than full line coverage. A suite that verifies every keyboard path and ARIA attribute catches the regressions that actually reach consumers.

> **NOTE**
> The companion repository does not hit the unit-test target, and it is more useful to say so than to imply otherwise. `Select` and `Menu` have no test file, and six of the eight layout primitives have none either. Their accessibility coverage comes entirely from the storybook browser project. That is a reasonable safety net and not a substitute: a story renders a component in one state, while a test drives it through the states a user will.

The test examples in chapters 4 through 9 include the axe assertion for Stack, Input, and Button only, for brevity. Apply the same `toHaveNoViolations()` check to every component you build.

CHAPTER 12

Packaging and distribution

Your library works in Storybook. Your tests pass. Now you need to package it so other projects can install and use it.



Configure Vite library mode

Vite's library mode produces a clean build output with ECMAScript module (ESM) exports and TypeScript declarations. `vite-plugin-dts` generates `.d.ts` TypeScript declaration files alongside the compiled output so consumers get full type information without needing access to the library source. Install it:

```bash
npm install -D vite-plugin-dts
```

Add the library build configuration to `vite.config.ts`, alongside the test projects from chapter 11:

```typescript
/// <reference types="vitest/config" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import dts from 'vite-plugin-dts'
import { resolve } from 'path'
import path from 'node:path'
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
    dts({
      include: ['src'],
      exclude: [
        'src/**/*.test.{ts,tsx}',
        'src/**/*.stories.{ts,tsx}',
        'src/test-setup.ts',
        'src/stories/**',
      ],
    }),
  ],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src'),
    },
  },
  build: {
    lib: {
      entry: resolve(__dirname, 'src/index.ts'),
      formats: ['es'],
      fileName: 'index',
    },
    rollupOptions: {
      // Externalize every bare specifier. All runtime imports are declared in
      // `dependencies`, so consumers resolve their own copies — critical for
```

```
      // react-aria-components, whose React context breaks across duplicates.
      // `@/` is our own source alias and must stay bundled.
      external: (id) =>
        !id.startsWith('.') && !id.startsWith('@/') && !path.isAbsolute(id),
      output: {
        // Single ES output, so one directive at the top marks every export as a
        // client component for the Next.js App Router.
        banner: "'use client';",
      },
    },
  },
  // …the `test` block from chapter 11 stays here…
})
```

Externalize everything, not just React

The obvious version of `external` is an array:

```
external: ['react', 'react-dom', 'react/jsx-runtime']
```
TYPESCRIPT

It is the version most guides show, and for a library with no runtime dependencies it is correct. This library has eleven, and with that array every one of them gets bundled into `dist/index.js`: React Aria, React Stately, `react-aria-components`, Recharts, the three `@dnd-kit` packages, Iconify, `clsx`, and `tailwind-merge`.

Bundle size is the least of the problems. The real failure is duplicate module instances. A consumer who also uses `react-aria-components` directly now has two copies in their application, each with its own React context objects. Contexts compare by reference, so a component from your copy cannot read a provider from theirs. Nothing throws at build time. Menus stop closing, focus scopes stop containing, and the bug reproduces only in the consumer's application.

The predicate form says: bundle anything that is a relative path, our `@/` alias, or an absolute path, and externalize everything else. Every bare specifier becomes an import the consumer resolves, which means one copy of each package, deduplicated by their package manager.

That only works if the packages are actually declared as dependencies, which is the next section.

The other two details

`dts()` takes an `exclude` list. Without it, `vite-plugin-dts` walks everything under `src`, so the published package ships type declarations for 42 test files and 26

example pages. They are harmless and they roughly triple the size of what consumers download and what their editor indexes.

`output.banner` puts `'use client'` at the top of the bundle. Because the build emits a single ES file, that one directive marks every export as a client component, so the library drops into a Next.js App Router Server Component without a re-export barrel. Consumers who are not on Next.js ignore it entirely.

Configure `package.json` exports

JSON

```
{
  "name": "rudiment-ui",
  "version": "0.3.0",
  "description": "Accessible React component library built on React Aria, with layout
   primitives, charts, and a three-tier design token architecture.",
  "type": "module",
  "license": "MIT",
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Rudiment-UI/rudiment-ui-components.git"
  },
  "scripts": {
    "dev": "storybook dev -p 6006",
    "build": "vite build",
    "build:tokens": "style-dictionary build --config tokens/style-dictionary.config.mjs",
    "build-storybook": "storybook build",
    "prepare": "npm run build:tokens && npm run build",
    "prepublishOnly": "npm run build:tokens && npm run build",
    "test": "vitest",
    "test:watch": "vitest --watch",
    "test:coverage": "vitest --coverage"
  },
  "main": "dist/index.js",
  "module": "dist/index.js",
  "types": "dist/index.d.ts",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js"
    },
    "./styles": "./dist/index.css",
    "./package.json": "./package.json"
  },
  "files": ["dist"],
  "sideEffects": ["**/*.css"],
  "peerDependencies": {
    "react": "^19.0.0",
```

```
    "react-dom": "^19.0.0"
  },
  "dependencies": {
    "@dnd-kit/core": "^6.3.1",
    "@dnd-kit/sortable": "^10.0.0",
    "@dnd-kit/utilities": "^3.2.2",
    "@iconify/react": "^6.0.2",
    "@react-aria/utils": "^3.33.1",
    "clsx": "^2.1.1",
    "react-aria": "^3.47.0",
    "react-aria-components": "^1.16.0",
    "react-stately": "^3.45.0",
    "recharts": "^3.8.1",
    "tailwind-merge": "^3.5.0"
  }
}
```

Several of these fields are load-bearing in ways that are not obvious.

`dependencies` is required, not optional. Because the build externalizes every bare specifier, `dist/index.js` contains `import { useButton } from 'react-aria'` and nothing resolves it unless npm installs it for the consumer. Externalizing without declaring is the one combination that is strictly broken: the package installs cleanly and fails at import time.

React is a peer dependency, not a dependency. The consumer's React must be the same instance as the library's, which is precisely what a peer dependency expresses and a regular dependency does not.

`types` comes before `import` in `exports`. Node resolves export conditions in declaration order and takes the first match. `import` matches everything, so putting it first means the `types` condition is never reached and TypeScript falls back to guessing.

The CSS entry is `dist/index.css`. Vite names the emitted CSS asset after `build.lib.fileName`, which is `index`. Writing `dist/style.css` here, the default in older Vite versions, points the `./styles` subpath at a file that does not exist, and the failure surfaces as unstyled components rather than a missing-file error.

`sideEffects: ["**/*.css"]` tells bundlers that CSS imports have effects and must not be tree-shaken, while everything else is fair game. This is the other half of the colocated-stylesheet decision from chapter 4: it is what lets a consumer importing only `RudiButton` receive `button.css` and nothing else.

`prepare` and `prepublishOnly` both build tokens then code. `prepublishOnly` guarantees `npm publish` never ships a stale `dist/`. `prepare` covers installs from

a Git URL, where npm has no published tarball and must build from source. Both run the token build first, because the CSS is generated and the component styles are meaningless without it.

`files: ["dist"]` means only the build output is published. Source, stories, tests, and tokens stay in the repository.

Build and verify

```bash
npm run build:tokens
npm run build
```

Vite generates `dist/index.js`, `dist/index.d.ts`, and `dist/index.css`.

Verifying the output is less straightforward than it looks. Because every dependency is external, you cannot import the bundle from a bare `node -e` and read its exports; Node resolves `react-aria` from the current directory and, in a fresh clone without `node_modules`, fails. Check what the build actually produced instead:

```bash
ls dist
grep -c "^import" dist/index.js       # externalized imports, should be non-zero
head -1 dist/index.js                 # should print 'use client';
```

Then verify it properly, in a real consumer:

```bash
npm pack                              # produces rudiment-ui-0.3.0.tgz
cd ../some-test-app
npm install ../rudiment-ui/rudiment-ui-0.3.0.tgz
```

`npm pack` produces exactly the tarball `npm publish` would upload, so installing it into a scratch application exercises the real resolution path: the `exports` map, the peer dependency warning, the CSS subpath, and the type declarations. This catches packaging mistakes that no amount of inspecting `dist/` will, and it takes two minutes.

Apply semantic versioning

For a component library past 1.0, the versioning rules are:

Major (1.0.0 → 2.0.0): A component's prop interface changes in a way that breaks existing usage. A prop is removed or renamed. The rendered HTML structure changes in a way that breaks consumer CSS selectors. A token name changes.

Minor (1.0.0 → 1.1.0): A new component is added. A new prop is added with a default that preserves existing behavior. A new token is added.

Patch (1.0.0 → 1.0.1): A bug is fixed without changing the API. A dependency is updated. A documentation error is corrected.

Before 1.0, those rules do not apply, and pretending otherwise causes more confusion than it prevents. SemVer treats `0.x` as unstable by design, so the working convention is to shift everything down a level: minor for new components and features, major reserved for the day you actually commit to stability.

That has a consequence worth being upfront about. Breaking changes ship in minor releases while you are on `0.x`, which is exactly what the companion repository did at `0.2.0` when the token prefix changed and at `0.3.0` when the theme set was replaced. Both were genuinely breaking, both were minor bumps, and both were correct under `0.x`. What makes it workable is not the version number but the changelog entry.

So write the changelog properly from the first release. Follow Keep a Changelog: an `[Unreleased]` section at the top, then one section per version with a date, grouped into Added, Changed, Fixed, and Removed. Prefix anything breaking so it cannot be skimmed past:

```markdown
## [0.3.0] - 2026-08-04
### Changed
- **BREAKING** — Replaced the previous theme set with Roomy, Soft, and
  Compressed, each in light and dark. The old theme names no longer resolve.
### Added
- `RudiBreadcrumb`, `RudiPagination`, `RudiStepper`, `RudiRating`, `RudiTextarea`.
```

Consuming teams read this to decide whether to upgrade and what to test afterward. On `0.x` it is the only signal they have, since the version number is not carrying that information.

Make the changelog entry part of the pull request rather than something assembled at release time. Reconstructing three weeks of changes from commit messages is how breaking changes go unannounced.

What you have now

A publishable library package with ESM output, TypeScript declarations, correctly externalized dependencies, a `'use client'` banner for the Next.js App Router, an `exports` map, and a release workflow that rebuilds tokens and code before publishing.

The library is complete as a library. The remaining chapters extend it: theming, the components the guide has not yet covered, data visualization, and drag and drop.

CHAPTER 13

Theming: eight themes from one pipeline

This chapter turns the single token build from Chapter 3 into a family of themes that change palette, spacing, radius, and typeface without touching a line of component code.



Every component you have built reads its values from custom properties. Nothing hardcodes a color, a padding, or a font. That was the point of the three-tier token system, and this is the chapter where it pays for itself: you can now produce an entirely different-looking library by generating a second set of custom property values and letting the cascade do the rest.

No component changes. No conditional logic. No theme context provider.

What a theme actually is

A theme is one CSS file containing custom property declarations, scoped to an attribute selector:

```css
[data-theme='dark'] {
  --rudi-color-text-default: var(--rudi-color-neutral-50);
  --rudi-color-background-surface: var(--rudi-color-neutral-900);
  /* …and so on */
}
```

Set `data-theme="dark"` on an ancestor element and every custom property inside that subtree resolves to the dark value. A component that reads `var(--rudi-color-text-default)` picks up the new value without knowing a theme system exists.

The base theme lives in `:root`, which means there is no `data-theme="light"`. Light is what you get with no attribute at all. This trips people up when they build a theme switcher, so it is worth being explicit: to return to the default theme you *remove* the attribute rather than setting it to something.

```javascript
// Switch to dark
document.documentElement.setAttribute('data-theme', 'dark')
// Back to default
document.documentElement.removeAttribute('data-theme')
```

Why an attribute rather than a media query

The obvious alternative is `prefers-color-scheme`, and for a dark mode alone it would be simpler. Two reasons not to.

A media query cannot express a user's explicit choice. Someone whose system is in dark mode and who wants your documentation in light mode has no way to say so, and "respect the OS" stops being respectful the moment it overrides a preference the person stated.

More importantly, only two of the eight themes here are about light and dark. The rest change spacing density, corner radius, and typeface. `prefers-color-scheme` has nothing to say about whether a button should have 4px or 12px of vertical padding.

An attribute handles both, and you can still read `prefers-color-scheme` once at startup to pick the initial value.

The theme set

Three modifier themes, each in light and dark, plus the default pair:

Theme	`data-theme`	What changes
Default light	none	Base: IBM Plex, neutral brand, standard spacing
Default dark	`dark`	Palette inverts; everything else cascades from `:root`
Roomy light	`roomy`	Enlarged spacing, rounded radii, warm neutrals, slate accent, Rubik
Roomy dark	`roomy-dark`	Roomy geometry and typeface over the dark palette
Soft light	`soft`	Soft warm palette, periwinkle accent, Nunito, large radii, pill buttons
Soft dark	`soft-dark`	Soft geometry and typeface over the dark palette
Compressed light	`compressed`	Tight spacing, small radii, borders removed; palette and fonts unchanged
Compressed dark	`compressed-dark`	Compressed geometry over the dark palette

Compressed is the clearest illustration of what token architecture buys you. It overrides the spacing and radius scales, then sets about twenty `component.*` border tokens to `transparent` to get a flat, borderless look. It never touches the

palette or the typeface. That is enough to turn a comfortable marketing-site library into a dense data tool, from one JSON file and no component work.

One config per theme

Each theme is a separate Style Dictionary run. Create `tokens/style-dictionary.config.dark.mjs`:

JAVASCRIPT

```
// Primitive token groups that are theme-invariant and excluded from dark output
const PRIMITIVE_GROUPS = new Set(['blue', 'neutral', 'red', 'green', 'amber'])
const PRIMITIVE_CATEGORIES = new Set(['spacing', 'radius', 'font', 'motion'])
export default {
  source: [
    'tokens/tokens.json',
    'tokens/themes/dark/semantic.json',
    'tokens/components.json',
    'tokens/themes/dark/components.json',
  ],
  platforms: {
    css: {
      transformGroup: 'css',
      prefix: 'rudi',
      buildPath: 'tokens/build/',
      files: [
        {
          destination: 'tokens-dark.css',
          format: 'css/variables',
          options: {
            selector: '[data-theme="dark"]',
            outputReferences: true,
          },
          filter: (token) => {
            const category = token.path[0]
            if (PRIMITIVE_CATEGORIES.has(category)) return false
            if (category === 'color') {
              const group = token.path[1]
              if (group === 'dataviz') {
                return String(token.path[2]).startsWith('series')
              }
              return !PRIMITIVE_GROUPS.has(group)
            }
            return true
          },
        },
      ],
    },
  },
```

```
}
```

Two options do the work.

`selector` replaces `:root` in the output. Everything this config emits lands inside `[data-theme="dark"] { … }`.

`source` swaps the semantic layer and appends a component override. `tokens/themes/dark/semantic.json` replaces the base semantic file, redefining the same token paths against different primitives, and `tokens/themes/dark/components.json` adjusts the handful of component values that need more than a semantic swap. Tier 1 stays the same file, because the palette itself has not changed:

```json
{
  "color": {
    "text": {
      "default": { "$value": "{color.neutral.50}" },
      "subtle": { "$value": "{color.neutral.400}" }
    },
    "background": {
      "surface-base": { "$value": "{color.neutral.900}" },
      "surface-raised": { "$value": "{color.neutral.800}" }
    }
  }
}
```

Light mode points `text.default` at `neutral.900` and `surface-base` at `neutral.0`. Dark mode points them the other way. The neutral ramp is identical in both.

The filter is the important part

Without a `filter`, each theme file re-emits every token in the system. Eight themes then produce eight complete copies of the token set, which is both enormous and wrong: a theme that redeclares `--rudi-spacing-4` inside `[data-theme="dark"]` has taken ownership of a value it never intended to change, so a consumer overriding spacing in `:root` finds their override ignored in dark mode.

The filter answers one question per token: does this theme have an opinion about it?

The dark filter says no to spacing, radius, font, and motion, because dark mode is only about color. It says no to the primitive color ramps, because those hex values

are identical in both themes. It says yes to semantic and component color tokens, which is the whole point, and it carves out one exception for `dataviz.series-*`, because chart series colors genuinely do need to be brighter against a dark background. The dark theme sets those eight to literal hex values rather than aliasing the base ramp.

Compressed inverts the logic:

```javascript
filter: (token) => {
  const category = token.path[0]
  if (category === 'color' || category === 'font' || category === 'motion') return false
  return true
}
```

Everything except color, typeface, and motion. That is broader than "spacing and radius": it also lets through the `component.*` and `typography.*` tokens that resolve *through* spacing and radius, which is exactly how Compressed sets its border tokens to `transparent` without touching a color primitive.

> **NOTE**
> A filtered-out token is not missing, it is inherited. `--rudi-motion-duration-base` is declared once in `:root`, filtered out of all eight themes, and every theme resolves to it. Emitting it again per theme would produce the same value with worse override behavior. Be careful reasoning about which tokens this applies to, though: Roomy and Compressed both redefine the spacing scale, so `--rudi-spacing-4` genuinely differs per theme.

Composing themes

`compressed-dark` needs the dark palette and the compressed geometry. There is no `tokens/themes/compressed-dark/` directory, because it does not need one:

```javascript
source: [
  'tokens/tokens.json',
  'tokens/themes/dark/semantic.json',
  'tokens/components.json',
  'tokens/themes/dark/components.json',
  'tokens/themes/compressed/tokens.json',
],
```

Two existing override sets, composed in the `source` array. Order matters and the base files stay in: `tokens/components.json` sits between the two dark files

because the dark component overrides are a partial layer on top of it, not a replacement. Drop it and the build has no base component tier at all.

`roomy-dark` and `soft-dark` work the same way, each adding only a small semantic file for the accent color that differs between their light and dark forms.

This is why the source array is an explicit list rather than a glob. A glob would sweep in every theme's overrides at once, and the failure would be silent.

Adding a theme

Five steps, and missing any one of them produces a theme that half-works:

1. Create `tokens/themes/<name>/` with the token files you are overriding.
2. Create `tokens/style-dictionary.config.<name>.mjs` with the right `selector`, `source`, and `filter`.
3. Add the config to the `build:tokens` chain in `package.json`.
4. Add an `@import` for the generated file to **both** `src/styles.css` and `src/app.css`.
5. Add a toolbar item to `.storybook/preview.tsx`.

Step four is the one people miss. Import into `app.css` only and the theme works perfectly in Storybook and does not exist for anyone who installs the package.

The build script chains every config:

```json
{
  "scripts": {
    "build:tokens":
   "style-dictionary build --config tokens/style-dictionary.config.mjs && style-dictionary
   build --config tokens/style-dictionary.config.dark.mjs && style-dictionary build --
   config tokens/style-dictionary.config.roomy.mjs && style-dictionary build --config
   tokens/style-dictionary.config.roomy-dark.mjs && style-dictionary build --config tokens/
   style-dictionary.config.soft.mjs && style-dictionary build --config tokens/style-
   dictionary.config.soft-dark.mjs && style-dictionary build --config tokens/style-
   dictionary.config.compressed.mjs && style-dictionary build --config tokens/style-
   dictionary.config.compressed-dark.mjs"
  }
}
```

It is a long line. Chaining with `&&` means a failure in any theme stops the build rather than leaving a stale file on disk.

Both CSS entry points import all eight, base first:

```css
@import '../tokens/build/tokens.css';
@import '../tokens/build/tokens-dark.css';
@import '../tokens/build/tokens-roomy.css';
@import '../tokens/build/tokens-roomy-dark.css';
@import '../tokens/build/tokens-soft.css';
@import '../tokens/build/tokens-soft-dark.css';
@import '../tokens/build/tokens-compressed.css';
@import '../tokens/build/tokens-compressed-dark.css';
```

The specificity trap

This is the part that generates support questions, so it is worth understanding before you ship.

`:root` and `[data-theme="dark"]` have the same specificity: both are a single selector matching one element. When two rules of equal specificity set the same property, the later one wins. Theme files are imported after the base file, so theme values win inside their scope. That is exactly what you want.

It also means a consumer override written the obvious way only works in the default theme:

```css
/* Looks right. Works in light mode only. */
:root {
  --rudi-color-brand-primary: #7c3aed;
}
```

In dark mode, `[data-theme="dark"]` still sets `--rudi-color-brand-primary`, comes later in the cascade, and wins. The consumer's brand color silently disappears when the user switches themes.

The reliable fix is to repeat the override for every theme the consumer intends to use:

```css
:root,
[data-theme='dark'],
[data-theme='roomy'],
[data-theme='roomy-dark'],
[data-theme='soft'],
[data-theme='soft-dark'],
[data-theme='compressed'],
```

```
[data-theme='compressed-dark'] {
  --rudi-color-brand-primary: #7c3aed;
}
```

Verbose, and correct in every theme.

The tempting shortcut is to override a primitive instead, on the theory that themes redefine semantic tokens and leave the raw ramps alone. `outputReferences: true` from chapter 3 preserves the alias chain into the CSS, so overriding a link moves everything downstream, and for the dark and compressed themes that works.

It does not work in general, and this is worth checking before you recommend it. Roomy and Soft both ship their own `color.neutral` ramp, and their configs deliberately omit `neutral` from the excluded primitive groups, so each emits more than twenty `--rudi-color-neutral-*` declarations of its own. A consumer who overrides `--rudi-color-neutral-900` in `:root` sees it apply in the default and dark themes and vanish in four others.

Whichever you recommend, verify it against every theme you ship, and document it. Consumers will hit this.

Wiring the toolbar

Extend the `globalTypes` block from chapter 10 to all eight:

TSX

```
globalTypes: {
  theme: {
    description: 'Color theme',
    toolbar: {
      title: 'Theme',
      icon: 'paintbrush',
      items: [
        { value: 'light', title: 'Default Light', icon: 'sun' },
        { value: 'dark', title: 'Default Dark', icon: 'moon' },
        { value: 'roomy', title: 'Roomy Light', icon: 'sun' },
        { value: 'roomy-dark', title: 'Roomy Dark', icon: 'moon' },
        { value: 'soft', title: 'Soft Light', icon: 'sun' },
        { value: 'soft-dark', title: 'Soft Dark', icon: 'moon' },
        { value: 'compressed', title: 'Compressed Light', icon: 'sun' },
        { value: 'compressed-dark', title: 'Compressed Dark', icon: 'moon' },
      ],
      dynamicTitle: true,
    },
  },
```

```
},
```

The decorator written in chapter 10 already handles all of them; it sets whatever value it is given. Cycle a complex component through the eight and you will find the places where a hardcoded value slipped through, because it is the one thing that does not move.



LIVE COMPONENT
Theming documentation
rudiment-ui.netlify.app/?path=/docs/getting-started-theming--docs

A note on fonts

Roomy uses Rubik and Soft uses Nunito. Neither ships with the library, and neither loads automatically.

Bundling fonts in a component library is a bad trade. Font files are large, licensing varies, and consumers who use two of your eight themes would still pay for all of them. So the library declares a font family in a token and assumes the application provides the file.

The failure mode is quiet: an unloaded family falls through to the next entry in the stack, and the theme renders in the system sans-serif looking almost right. Document which families each theme expects, load them in `.storybook/preview-head.html` for development, and say so plainly in your README.

What you have now

Eight themes generated from one token source, switchable with a single attribute, with no component code involved in any of it.

More usefully, you now have the mechanism for a ninth. A consumer who wants their own brand theme writes a `tokens/themes/<brand>/` directory and a config, and their theme is a peer of yours rather than a fork of your stylesheet. That is the difference between a library that can be themed and a library that happens to ship a dark mode.

The next chapter returns to components and fills in the set the guide has not covered yet.

CHAPTER 14

The component set beyond the core

This chapter covers the seventeen components that fill the gap between a working library and one people can build a product with.



Chapters 7 through 9 built the components that justify a library existing: the ones with real accessibility complexity, where a hand-rolled version would be subtly broken. That is roughly a third of what ships.

The rest is this chapter. Individually these components are unremarkable, and that is the point. Nobody adopts a library for its Divider. They abandon one for the absence of a Divider, because the first thing they build needs a horizontal rule and now they are writing CSS again, and once they are writing CSS they are making spacing decisions your token system was supposed to make for them.

Rather than seventeen implementations at chapter 7's depth, this chapter covers the API of each and stops to explain the ones where the design decision is genuinely interesting.

Menu

Menu is the exception in this chapter: it has real complexity, and it is the one component in the library not built from React Aria hooks.

```tsx
<RudiMenuTrigger>
  <RudiIconButton icon="mdi:dots-vertical" aria-label="Actions" />
  <RudiMenu onAction={handleAction}>
    <RudiMenuSection title="Edit">
      <RudiMenuItem id="rename" icon="mdi:pencil" label="Rename" shortcut="⌘R" />
      <RudiMenuItem id="duplicate" icon="mdi:content-copy" label="Duplicate" />
    </RudiMenuSection>
    <RudiMenuSeparator />
    <RudiMenuItem id="delete" icon="mdi:delete" label="Delete" isDestructive />
  </RudiMenu>
</RudiMenuTrigger>
```

Seven exports: `RudiMenuTrigger`, `RudiMenu`, `RudiMenuItem`, `RudiMenuSection`, `RudiMenuSeparator`, `RudiSubmenuTrigger`, and `RudiKeyboard` for rendering shortcut hints.

Each is a thin wrapper over the equivalent from `react-aria-components`, adding class names and, on `RudiMenuItem`, five convenience props:

```typescript
export interface RudiMenuItemProps extends AriaMenuItemProps {
  isDestructive?: boolean
  icon?: string
  label?: string
  description?: ReactNode
```

```
  shortcut?: ReactNode
}
```

Why hooks lost here

Everything else in the library composes React Aria hooks, and that was the right call: assembling a Button from `useButton` teaches you what the hook handles, and the assembly is twenty lines.

A menu is not twenty lines. It needs a collection, virtual focus, typeahead, submenu positioning with flip and shift behavior, hover intent so a diagonal mouse path toward a submenu does not close it, and a focus contract between trigger and popover. Hand-assembling that from `useMenu`, `useMenuTrigger`, `useMenuItem`, and `useMenuSection` is several hundred lines whose bugs surface only under keyboard use.

`react-aria-components` is the same team's higher-level layer over the same hooks. Using it here costs consistency with the rest of the library and saves the part most likely to be wrong.

The rule worth taking from this: drop to hooks when you need control over structure, and use the component layer when you need behavior you would otherwise reimplement badly. Deciding by principle rather than per component is how you end up with either a library of thin wrappers or a broken menu.

> **NOTE**
> This dependency has a packaging consequence, covered in chapter 12: `react-aria-components` context breaks across duplicate module instances. If it gets bundled into your build instead of externalized, menus in a consumer that also uses the package will fail in ways that never reproduce locally.

Icon

Eight of the components in this chapter render an Icon, so it comes first.

TYPESCRIPT

```ts
export interface RudiIconProps {
  icon: string
  size?: 'sm' | 'md' | 'lg' | number
  color?: string
  label?: string
  className?: string
```

```
}
```

A wrapper over `@iconify/react` that maps `size` onto the token scale and handles the accessibility branch. The `icon` prop is an Iconify name such as `mdi:check`, and icon data loads on demand rather than shipping a set.

The `label` prop controls semantics. Without it the icon renders `aria-hidden`, which is correct for the common case: an icon next to a text label is decorative, and announcing "checkmark icon, Complete" is noise. With a label, the icon becomes `role="img"` with an accessible name, for the rarer case where the icon carries meaning alone.

Defaulting to hidden is deliberate. The failure mode of a decorative icon accidentally announced is a slightly worse experience. The failure mode of a meaningful icon silently hidden is missing information, and requiring an explicit `label` makes that a decision rather than an oversight.

Text and content

Textarea mirrors Input from chapter 7 almost exactly. It adds two props:

```
rows?: number
resize?: 'none' | 'vertical' | 'horizontal' | 'both'
```

TYPESCRIPT

and drops Input's `type` union, which has no meaning on a textarea. Everything else, including the `label` / `description` / `errorMessage` triple and the `aria-describedby` wiring, is identical. Build it by copying Input.

Link is a styled anchor:

```
export interface RudiLinkProps
  extends React.AnchorHTMLAttributes<HTMLAnchorElement> {
  href: string
  variant?: 'default' | 'subtle' | 'standalone'
  underline?: 'always' | 'hover' | 'none'
  external?: boolean
  children: React.ReactNode
  className?: string
}
```

TYPESCRIPT

Extending `AnchorHTMLAttributes` and spreading the rest onto the anchor is the pattern to copy for any component wrapping a single element. It means `download`, `hreflang`, and every data attribute work without you enumerating them.

`external` adds `target="_blank"`, `rel="noreferrer noopener"`, and an outbound-arrow icon. The `rel` pair is the security part: `noopener` stops the new page reaching back through `window.opener`, and `noreferrer` withholds the referrer.

> **NOTE**
> The icon is `aria-hidden`, which means `external` currently gives a sighted user a signal and a screen reader user nothing. A link that opens a new tab without warning is disorienting for anyone who cannot see it happen. If you are building this yourself, add visually-hidden text alongside the icon; putting it in the component is the only way it happens consistently, because nobody remembers it at the call site.

`underline="none"` exists because designers ask for it. Use it for navigation and never for links inside body text, where the underline is the only non-color signal that something is a link.

Image:

```typescript
export interface RudiImageProps {
  src: string
  alt: string
  aspectRatio?: CSSProperties['aspectRatio']
  fit?: 'cover' | 'contain'
  radius?: 'none' | 'sm' | 'md' | 'lg' | 'full'
  loading?: 'lazy' | 'eager'
  overlay?: ReactNode
  className?: string
}
```

`alt` is required, with no `?`. It is the same argument as IconButton's required `aria-label` in chapter 7: a decorative image passes `alt=""`, which is explicit and takes one keystroke, while an optional prop makes forgetting invisible.

`aspectRatio` prevents layout shift by reserving space before the image loads.

Status and indicators

Avatar takes `src`, `alt`, `name`, `size`, and `status`. When `src` is absent or fails to load it falls back to initials derived from `name`, so a missing image degrades to something readable rather than a broken-image glyph.

Tag carries `variant`, `dismissible`, `onDismiss`, `onPress`, and `isDisabled`. It differs from Badge, which chapter 9 built, in exactly one way that matters: a Tag can be interactive. Badge is a label the user reads; Tag is a chip the user removes or selects. That is why Tag has press handling and Badge does not, and why they are separate components rather than one component with an `interactive` prop.

Dot is a status indicator with `tone`, `size`, `pulse`, `label`, and a `color` escape hatch that overrides `tone` for one-off cases. Unlabeled it is `aria-hidden`. Labeled it becomes `role="status"` with visually-hidden text, rather than Icon's `role="img"` with an `aria-label`. The difference is that a status dot usually changes while the page is open, and `role="status"` announces the change; a labeled icon is static.

ProgressBar is the most interesting of the group:

```typescript
export type RudiProgressBarVariant = 'default' | 'success' | 'warning' | 'error'
export interface RudiProgressBarThreshold {
  /** Minimum percentage (0–100) at which this variant takes effect. */
  at: number
  variant: RudiProgressBarVariant
}
export interface RudiProgressBarProps {
  value: number
  minValue?: number
  maxValue?: number
  label: string
  showValueLabel?: boolean
  variant?: RudiProgressBarVariant
  thresholds?: RudiProgressBarThreshold[]
  size?: 'sm' | 'md'
  className?: string
}
```

`thresholds` lets the bar change color as it fills:

```tsx
<RudiProgressBar
  label="Storage used"
  value={usedGb}
  maxValue={totalGb}
```

```
  thresholds={[
    { at: 80, variant: 'warning' },
    { at: 95, variant: 'error' },
  ]}
/>
```

Without it, every consumer with a quota meter writes the same `value > 80 ? 'warning' : 'default'` expression, and each writes it slightly differently. Note that the color is never the only signal: `label` is required and `showValueLabel` renders the number, so the state is available to someone who cannot distinguish the colors.

CircularProgress is the same idea in a ring, and accepts `children` to render content in the center.

StatCard takes `label`, `value`, `delta`, and `trend`, with `children` as a slot for a sparkline. `trend` defaults to `'neutral'` and drives the delta's color through a `rudi-stat-card__delta--{trend}` modifier.

Color is the only thing it drives, which makes trend direction a color-only signal. The usual fix is an arrow glyph or a `+` / `−` prefix rendered from `trend` rather than left to whatever string the consumer passes as `delta`. Worth adding when you build this.

Rating takes `value`, `max`, `onChange`, `showValue`, `count`, and `label`. It is read-only without `onChange` and interactive with it. Note that read-only mode is the common case for a rating display, so the interactive branch is opt-in rather than the reverse.

Navigation

Breadcrumb:

TYPESCRIPT

```
export interface RudiBreadcrumbItem {
  label: string
  href?: string
}
export interface RudiBreadcrumbProps {
  items: RudiBreadcrumbItem[]
  separator?: string
  label?: string
  className?: string
}
```

The last item always renders as plain text with `aria-current="page"`, even if you gave it an `href`. Any other item without an `href` renders as text with no `aria-current`. Dropping the final link is deliberate: a link to the page you are already on is a dead control, and `aria-current` is what communicates position.

The whole thing is a `<nav>` with an accessible name, so screen reader users can skip it.

Note that `separator` is an Iconify name rather than a character. Separators rendered as text get announced ("slash Home slash Products"), so the icon is `aria-hidden` and the list structure carries the relationship instead.

Pagination:

```typescript
export interface RudiPaginationProps {
  page: number
  pageCount: number
  onPageChange: (page: number) => void
  siblingCount?: number
  label?: string
  className?: string
}
```

`siblingCount` controls how many page numbers appear either side of the current one before collapsing into an ellipsis. The component computes the window; the consumer supplies only the current page and the total.

NavItem takes `label`, `icon`, `isActive`, `href`, `onPress`, `badge`, and `isDisabled`. The rendered element branches on `href` alone: with one it is an `<a>`, without one it is a `<button>`. Passing both gives you a link that also fires `onPress`.

That branch matters. A navigation item that changes the URL should be a link, so it works with middle-click, keyboard modifiers, and "copy link address". A button calling `history.push` looks identical and breaks all three.

Stepper:

```typescript
export type RudiStepStatus = 'complete' | 'current' | 'upcoming'
export interface RudiStepperStep {
  label: string
  description?: string
  icon?: string
  status?: RudiStepStatus
```

```
}
```

Takes `steps`, `orientation`, and `size`. Status drives the node appearance, and the current step carries `aria-current="step"`.

Structure

Divider takes `orientation`, `spacing`, and an optional `label` that renders inline within the rule. Its element depends on both: a plain horizontal divider is an `<hr>`, while a vertical one, or a horizontal one with a label, is a `<div role="separator" aria-orientation="…">` wrapping rule, label, rule. A labeled divider cannot be an `<hr>`, since `<hr>` takes no children.

The `spacing` prop is worth noting as a counterexample to chapter 5's rule that spacing props take CSS lengths. Divider uses a keyword scale (`none | sm | md | lg`) because a divider's margins should come from a small set of consistent values, not arbitrary lengths. Layout primitives are general-purpose and take any length; a component with one job can be opinionated.

The unexported five

The companion repository also contains `AppShell`, `TopBar`, `Footer`, `PageHeader`, and `SectionHeader`. They are built, styled, and have stories, and they are deliberately absent from `src/index.ts`.

They exist to compose the example pages from chapter 10 without those pages duplicating a hundred lines of chrome each. Exporting them would be a commitment: an application shell with a responsive drawer has opinions about routing, breakpoints, and where navigation state lives, and every one of those opinions becomes an API you support forever.

> **NOTE**
> Whether they *should* stay unexported is a live question rather than a settled one. An `AppShell` is one of the more useful things a library can ship, and the current arrangement means anyone who wants one copies the source. If you follow this pattern, be deliberate: unexported-because-not-ready is a reasonable position, unexported-by-accident is not, and consumers cannot tell the difference from the outside.

What you have now

The library covers what a product actually needs: forms, overlays, navigation, status, and content. A page built from it should not require any custom CSS.

That is a testable claim, and the example pages from chapter 10 are the test. Build one and count how many times you reach for a raw element or a one-off style rule. Each instance is either a missing component or a missing prop, and finding them this way is considerably cheaper than hearing about them from someone who already adopted the library.

The next two chapters cover the parts that need real dependencies: charts, and a drag-and-drop board.

CHAPTER 15

Data visualization

This chapter wraps a charting library so that charts read design tokens, follow the active theme, and stay accessible.



Charts are where a token system usually gives up. A charting library draws with SVG and canvas, takes colors as JavaScript strings, and knows nothing about CSS custom properties. So a chart gets a hardcoded palette, and the moment someone switches to a dark theme the chart is still drawing 2563eb on a near-black background.

This chapter solves that. The interesting work is not wrapping the chart components; it is the hook that bridges CSS custom properties into the JavaScript values a chart library needs, and keeps them in sync when the theme changes.

Choosing a charting library

```bash
npm install recharts
```

Recharts is composable React components over D3, which suits this library: `<BarChart>` contains `<XAxis>`, `<CartesianGrid>`, and `<Bar>` as real children, so wrapping it means composing components rather than building a configuration object.

The alternatives are reasonable. Visx gives lower-level primitives and more control at the cost of writing your own axes and tooltips. Chart.js is canvas-based, which performs better past a few thousand points but leaves nothing in the DOM for assistive technology or your stylesheet. Plotting from scratch with D3 is a project, not a chapter.

Recharts is a substantial dependency, pulling in a set of D3 modules behind it. Chapter 12's `external` predicate keeps it out of your bundle, and consumers who never import a chart never download it. If you would rather not put it in `dependencies` at all, publishing charts as a separate package is a defensible split.

The theme bridge

A chart needs `"#2563eb"`. Your token system has `--rudi-color-dataviz-series-1`. Nothing resolves one to the other automatically, because custom properties are resolved by the browser during styling and never exposed to JavaScript on their own.

`getComputedStyle` is the bridge:

```typescript
```

```
const styles = getComputedStyle(document.documentElement)
const value = styles.getPropertyValue('--rudi-color-dataviz-series-1').trim()
```

That reads the value as it currently resolves, which means it already accounts for whichever `data-theme` is active. Read it once at mount and you have theme-correct colors. But read it only once and the chart keeps those colors after the user switches themes, because nothing told React to re-render.

`useChartTheme` handles both halves:

TYPESCRIPT

```
import { useCallback, useEffect, useState } from 'react'
const TOKEN_MAP = {
  axisColor: '--rudi-component-chart-axis-color',
  gridColor: '--rudi-component-chart-grid-color',
  labelColor: '--rudi-component-chart-label-color',
  legendColor: '--rudi-component-chart-legend-color',
  tooltipBg: '--rudi-component-chart-tooltip-bg',
  tooltipText: '--rudi-component-chart-tooltip-text',
  tooltipBorder: '--rudi-component-chart-tooltip-border',
  chartBg: '--rudi-component-chart-bg',
} as const
const SERIES_TOKENS = Array.from(
  { length: 8 },
  (_, i) => `--rudi-color-dataviz-series-${i + 1}`,
)
const FALLBACK_COLORS = [
  '#2563eb', '#d946ef', '#f59e0b', '#10b981',
  '#ef4444', '#8b5cf6', '#06b6d4', '#f97316',
]
function resolveTokens(): ChartTheme {
  if (typeof window === 'undefined') {
    return { colors: FALLBACK_COLORS, axisColor: '#6b7280' /* …etc */ }
  }
  const styles = getComputedStyle(document.documentElement)
  const resolve = (token: string) => styles.getPropertyValue(token).trim()
  const colors = SERIES_TOKENS.map(
    (token, i) => resolve(token) || FALLBACK_COLORS[i],
  )
  const theme: Record<string, string> = {}
  for (const [key, token] of Object.entries(TOKEN_MAP)) {
    theme[key] = resolve(token) || ''
  }
  return { colors, ...theme } as ChartTheme
}
export function useChartTheme(): ChartTheme {
  const [theme, setTheme] = useState<ChartTheme>(resolveTokens)
  const updateTheme = useCallback(() => {
    setTheme(resolveTokens())
  }, [])
```

```
  useEffect(() => {
    updateTheme()
    const observer = new MutationObserver((mutations) => {
      for (const mutation of mutations) {
        if (
          mutation.type === 'attributes' &&
          mutation.attributeName === 'data-theme'
        ) {
          updateTheme()
        }
      }
    })
    observer.observe(document.documentElement, {
      attributes: true,
      attributeFilter: ['data-theme'],
    })
    return () => observer.disconnect()
  }, [updateTheme])
  return theme
}
```

Four decisions in that hook are worth understanding, because each one is a bug you would otherwise ship.

The `MutationObserver` watches `data-theme` on the root element. This is the piece that makes charts theme-aware rather than theme-initialized. Chapter 13's theme switching is pure CSS, so React has no idea it happened; every other component re-styles through the cascade without re-rendering. Charts are the one place that needs an explicit signal, and `attributeFilter: ['data-theme']` keeps the observer from firing on unrelated attribute changes.

`typeof window === 'undefined'` guards server rendering. `getComputedStyle` does not exist on the server, and a library that throws during SSR is unusable in Next.js and Remix. The fallback palette renders a reasonable chart, and the effect corrects it on hydration.

`useEffect` calls `updateTheme()` immediately, even though `useState` already initialized from `resolveTokens`. That looks redundant and is not. During hydration the initial state came from the server-side branch, so this call is what replaces the fallback palette with the real tokens. It also covers the case where stylesheets had not finished applying at first render.

The series colors fall back and the chrome tokens do not. `getPropertyValue` returns an empty string for a property that does not exist, and an empty string is

not a valid color, so a missing token produces an invisible element rather than an error.

Series colors are protected by `|| FALLBACK_COLORS[i]`. The chrome tokens are not: the loop writes `resolve(token) || ''`, so a missing `--rudi-component-chart-axis-color` yields an empty string and the axis disappears. That is worth fixing in your own version by giving `TOKEN_MAP` fallbacks the same way, and worth understanding either way, because an invisible axis reads as a styling bug rather than a missing token.

The tokens behind it

The eight series colors come from chapter 3's `dataviz` ramp, aliased through semantic tokens:

```json
"dataviz": {
  "series-1": { "$value": "{color.dataviz.1}" },
  "series-2": { "$value": "{color.dataviz.2}" }
}
```

Chart chrome, the axes, grid, labels, legend, and tooltip, lives in the component tier as `component.chart.*`. Both re-emit per theme, which is why chapter 13's dark filter makes a specific exception for `dataviz.series-*` while excluding the other primitive color ramps. A blue that reads well on white is often too dark on near-black, so series colors are one of the few primitives a theme legitimately overrides.

Eight is a practical ceiling, not an arbitrary one. Past roughly eight categories, distinguishing series by color stops working for everyone, not only for colorblind readers. If a chart needs a ninth color, it usually needs a different chart.

The chart components

All three follow one shape. BarChart:

```typescript
export interface RudiBarChartProps {
  /**
   * Row data. Values may be `null` to represent gaps in a series.
   */
  data: Array<Record<string, string | number | null>>
  dataKeys: string[]
  indexKey: string
  layout?: 'vertical' | 'horizontal'
```

```
  stacked?: boolean
  showLegend?: boolean
  showGrid?: boolean
  height?: number
  label: string
  className?: string
}
```

LineChart swaps the layout props for `curved` and `showDots`. DonutChart takes `data` as `{ name, value }` pairs plus `innerRadius` and `showLabels`.

The implementation composes Recharts inside a labelled container:

TSX

```tsx
export function RudiBarChart({
  data,
  dataKeys,
  indexKey,
  layout = 'vertical',
  stacked = false,
  showLegend = true,
  showGrid = true,
  height = 300,
  label,
  className,
}: RudiBarChartProps) {
  const theme = useChartTheme()
  const isHorizontal = layout === 'horizontal'
  return (
    <div className={cn('rudi-chart', className)} role="img" aria-label={label}>
      <ResponsiveContainer width="100%" height={height}>
        <RechartsBarChart
          data={data}
          layout={isHorizontal ? 'vertical' : 'horizontal'}
        >
          {showGrid && <CartesianGrid stroke={theme.gridColor} />}
          {/* Axis pair swaps type/dataKey depending on orientation;
              both pass tick={{ fill: theme.axisColor }} */}
          <XAxis dataKey={indexKey} tick={{ fill: theme.axisColor }} />
          <YAxis tick={{ fill: theme.axisColor }} />
          {dataKeys.map((key, i) => (
            <Bar
              key={key}
              dataKey={key}
              fill={theme.colors[i % theme.colors.length]}
              stackId={stacked ? 'stack' : undefined}
            />
          ))}
        </RechartsBarChart>
      </ResponsiveContainer>
```

```
    </div>
  )
}
```

Three things there generalize beyond charts.

`label` is required and becomes `aria-label` on a `role="img"` wrapper. A chart is a picture built from hundreds of SVG nodes. Left alone, a screen reader either announces nothing useful or reads every tick mark. Collapsing it to a single labelled image is the honest representation, which makes the label the entire accessible content: `"Chart"` is useless, `"Monthly revenue by region, January through June"` is not. Making it required forces the decision.

Series colors cycle with `i % theme.colors.length`. Nine series reuse the first color rather than crashing on `undefined`, which is a degraded chart instead of a broken one.

`layout` is inverted on the way through. The `layout` prop means the direction bars grow, which is how people describe bar charts. Recharts' `layout` prop means the axis orientation, which is the opposite. Rather than exposing that inversion, the wrapper absorbs it. This is most of what wrapping a third-party library is for: a stable, sensible API that does not change when the dependency does.

`data` values are typed to allow `null`, which Recharts renders as a gap. Coercing missing data to zero draws a line to the floor and implies a measurement that was never taken.

What charts do not solve

Two limitations worth documenting rather than discovering.

A `role="img"` chart is described, not explored. A sighted user reads individual values off the axes; a screen reader user gets your one-sentence summary. For charts where the numbers matter, pair the chart with a data table, visually hidden if you like. Nothing in this component does that for you.

And color is the only channel distinguishing series in a default Recharts chart. The eight-color ramp is chosen for distinguishability, but for a stacked bar or a multi-line chart, patterns, direct labels, or varied dash styles carry information that color alone does not.



LIVE COMPONENT

BarChart in Storybook

rudiment-ui.netlify.app/?path=/docs/components-charts-barchart--docs

What you have now

Three chart components that follow the theme, drawn from the same token pipeline as everything else, and a hook that is reusable for any other case where JavaScript needs a token value.

`useChartTheme` is the piece worth remembering. Any component that must hand a token value to a library that does not speak CSS, a canvas renderer, a map, a syntax highlighter, needs exactly this pattern: resolve through `getComputedStyle`, observe `data-theme`, fall back sensibly, and guard the server.

The next chapter covers the other component with a real dependency: a drag-and-drop board that stays operable from the keyboard.

CHAPTER 16

Drag and drop that works from the keyboard

This chapter builds a Kanban board, and uses it to work through the accessibility problem that drag-and-drop interfaces almost always get wrong.

Drag and drop is the most reliably inaccessible pattern on the web. The interaction is defined by a physical gesture, the affordance is usually a cursor change, and the state that matters lives in coordinates rather than in the DOM. Most implementations are unusable without a mouse, and the ones that are not tend to have been built that way from the start.

This chapter builds a Kanban board, which is the canonical case: cards that reorder within a column and move between columns. The board itself is a fine component. The reason it earns a chapter is that it forces every decision the pattern gets wrong.

Choosing a library

```bash
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
```

`@dnd-kit` is the right choice for one reason above the others: keyboard support is a first-class sensor, not an afterthought. Its architecture treats pointer, keyboard, and touch as interchangeable input sources feeding the same drag state.

The HTML5 drag-and-drop API, the platform's own answer, is not a real option here. Its keyboard support is nonexistent, it does not work on touch devices, and its styling hooks are famously limited.

`react-beautiful-dnd` was the previous default and had good keyboard support, but it is no longer maintained. `@dnd-kit` is the successor most projects moved to.

The API

```typescript
export interface RudiKanbanItem {
  id: string
  [key: string]: unknown
}
export interface RudiKanbanColumnData {
  id: string
  title: string
  items: RudiKanbanItem[]
}
export interface RudiKanbanCardMoveEvent {
  cardId: string
  fromColumnId: string
  toColumnId: string
```

```
  fromIndex: number
  toIndex: number
}
export interface RudiKanbanBoardProps {
  columns: RudiKanbanColumnData[]
  onCardMove: (event: RudiKanbanCardMoveEvent) => void
  renderCard: (item: RudiKanbanItem) => React.ReactNode
  className?: string
}
```

The board is fully controlled. It never mutates `columns`; it reports a move and re-renders from whatever the consumer passes next.

That matters more here than for a typical controlled input. Card moves usually need to persist, and persistence fails. If the board owned its own state, a rejected server write would leave the UI showing a move that did not happen, and reconciling that means fighting the component. Controlled, the consumer applies the move optimistically, and reverts by passing the old array back.

`RudiKanbanCardMoveEvent` carries both the column identifiers and the indices because a consumer needs both. The column tells you what status to write; the index tells you where in the ordering it landed.

`renderCard` is a render prop rather than a `card` component, because card content is entirely the consumer's business. The board owns drag behavior and layout; what a card shows is not its problem.

Sensors: where accessibility is decided

TYPESCRIPT

```
const sensors = useSensors(
  useSensor(PointerSensor, {
    activationConstraint: { distance: 5 },
  }),
  useSensor(KeyboardSensor, {
    coordinateGetter: sortableKeyboardCoordinates,
  }),
)
```

Four lines, and they are the whole accessibility story.

`KeyboardSensor` makes the board operable without a pointer: Space or Enter picks a card up, arrow keys move it, Space drops it, Escape cancels. Omit this sensor

and the board works perfectly with a mouse and cannot be used at all otherwise. Nothing warns you, because the mouse path is the one you test.

The keyboard entry point is not the card itself. `KanbanCard` spreads dnd-kit's `attributes` and `listeners` onto a dedicated `<button className="rudi-kanban__card-handle">` with its own `aria-label`, and that button is what receives focus. Putting them on the card `<div>` instead would mean either a non-focusable drag target or a card that swallows Space and Enter from any control inside it.

`sortableKeyboardCoordinates` translates arrow presses into positions within the sortable collections. Without it the keyboard sensor moves the card by a fixed pixel offset, which is meaningless for a list-based layout: the card drifts across the screen without ever landing in a slot.

`activationConstraint: { distance: 5 }` requires five pixels of movement before a drag begins. This is what lets a card contain a link or a button. Without it, the pointer sensor claims the interaction on mousedown and a click inside a card starts a drag instead of activating the control.

Announcing what happened

A sighted user watching a card move needs no narration. A screen reader user needs the whole interaction described, because the visual change is the entire feedback channel.

`DndContext` ships default announcements and its own live region, which is more than most libraries give you. They are generic by necessity: dnd-kit knows a draggable moved over a droppable, not that a card moved into "In Progress". Replacing them with domain-specific messages is the point of what follows.

> **NOTE**
> Adding a custom region without configuring `DndContext`'s `accessibility.announcements` leaves two live regions announcing the same events, which is what the code below does and what the companion repository ships. The result is duplicated, overlapping speech. Pass your messages through `DndContext`'s own `announcements` option instead, or suppress its defaults. The mechanics of writing announcements are the same either way.

```tsx
const announcementRef = useRef<HTMLDivElement>(null)
const announce = useCallback((message: string) => {
  if (announcementRef.current) {
```

```
    announcementRef.current.textContent = message
  }
}, [])
// …rendered at the end of the board:
<div
  ref={announcementRef}
  role="status"
  aria-live="assertive"
  aria-atomic="true"
  className="rudi-kanban__sr-only"
/>
```

The live region is written at four points:

```TYPESCRIPT
// On pick up
announce(`Picked up card from ${col.title}`)
// While moving over a column
announce(`Over column ${overCol.title}`)
// On drop
announce(`Dropped card in ${toColumn.title} at position ${toIndex + 1}`)
// On drop with no droppable under the pointer
announce('Cancelled drag')
```

That last one is worth reading carefully, because its placement is a real gap. It lives in `handleDragEnd`, guarded by `if (!over)`, so it fires when a card is released over empty space. It does not fire when the user presses Escape: dnd-kit routes a genuine cancel to `onDragCancel`, and the board does not implement that handler.

The consequence is that Escape mid-drag announces nothing and never clears `activeId`, so the drag overlay stays on screen. Add the handler:

```TYPESCRIPT
const handleDragCancel = useCallback(() => {
  setActiveId(null)
  announce('Cancelled drag')
}, [announce])
```

and pass `onDragCancel={handleDragCancel}` to `DndContext`. This is exactly the class of bug that only shows up on the keyboard path, which is why the test below matters.

Several details here are easy to get wrong.

The element is always in the DOM, empty. Live regions announce changes to their content. Mounting a region and populating it in the same render usually announces

nothing, because assistive technology never observed the change. Render it empty at mount and write text into it later.

`aria-live="assertive"`, not `polite`. Assertive interrupts, which is normally rude. During a drag it is correct: the announcement is a direct response to a key the user just pressed, and a polite announcement queued behind other output arrives after they have already pressed the next arrow key.

`aria-atomic="true"` makes the whole message read on each change, rather than only the diff. Without it, "Over column In Progress" followed by "Over column Done" can announce as a fragment.

The messages name things, not coordinates. "Dropped card in Done at position 3" is the state the user cares about. "Moved to 420, 180" is not.

The drop announcement fires before `onCardMove`. Setting the text first means the announcement is not waiting on whatever the consumer's handler does, which may be a network request.

Detecting the destination

The drag end handler has one piece of real logic, and it exists because a card can be dropped on two different kinds of thing:

```typescript
const toColumnDirect = columns.find((col) => col.id === overIdStr)
const toColumnViaItem = findColumnOfItem(columns, overIdStr)
const toColumn = toColumnDirect || toColumnViaItem
if (!toColumn) return
let toIndex: number
if (toColumnDirect) {
  // Dropped on the column itself — append to end
  toIndex = toColumn.items.length
} else {
  // Dropped on a specific card
  toIndex = toColumn.items.findIndex((item) => item.id === overIdStr)
}
```

Dropping on empty space below the cards means "put it in this column," and the only sensible index is the end. Dropping onto a specific card means "put it here," and the index is that card's position. Both are droppable targets, and the handler has to distinguish them because an empty column has no cards to drop onto and would otherwise reject every drag.

Then a guard that is worth keeping:

```typescript
if (fromColumn.id === toColumn.id && fromIndex === toIndex) return
```

A card picked up and put back has not moved. Firing `onCardMove` anyway means a network request and an entry in an activity log for a change that did not happen, and users pick cards up and think better of it constantly.

`collisionDetection={closestCorners}` is the other choice worth naming. The default `rectIntersection` requires actual overlap, which makes narrow gaps between columns feel unresponsive. `closestCorners` picks the nearest target by corner distance, which is more forgiving for the large, loosely spaced regions a board is made of.

The drag overlay

```tsx
<DragOverlay>
  {activeItem ? (
    <div className="rudi-kanban__card rudi-kanban__card--overlay">
      {renderCard(activeItem)}
    </div>
  ) : null}
</DragOverlay>
```

`DragOverlay` renders a copy of the dragged card in a fixed-position layer that follows the cursor, while the original stays in place at reduced opacity.

Without it, the dragged card is transformed inside its column, which means it is clipped by any ancestor with `overflow: hidden` and constrained by the column's stacking context. Since a board is horizontally scrolling columns, that clipping is guaranteed. `position: fixed` takes the overlay out of the flow and escapes both.

Fixed positioning is defeated by an ancestor with a `transform`, `filter`, or `perspective`, which establishes a new containing block. If your board sits inside an animated panel, wrap the overlay in `createPortal` to move it to the document body. `@dnd-kit` supports that and does not do it by default.



LIVE COMPONENT

Kanban board in Storybook

rudiment-ui.netlify.app/?path=/docs/components-kanbanboard--docs

Testing it

Test the keyboard path, because it is the one that regresses. A pointer drag in jsdom requires simulating a sequence of pointer events with real coordinates, and it tends to test your event simulation rather than the component. The keyboard path is a sequence of key presses:

```tsx
await user.tab()                    // focus the first card
await user.keyboard('{ }')          // pick up
await user.keyboard('{ArrowRight}') // move to next column
await user.keyboard('{ }')          // drop
expect(onCardMove).toHaveBeenCalledWith(
  expect.objectContaining({ toColumnId: 'in-progress' }),
)
```

That single test covers the sensor configuration, the coordinate getter, the destination logic, and the move event. If someone later removes `KeyboardSensor`, it fails immediately rather than shipping.

Assert on the live region too, and assert on its content rather than its presence. Announcements are exactly the kind of thing that survives a refactor as dead code, because nothing visible breaks when they stop firing.

> NOTE
>
> The companion repository's Kanban tests check rendering, drag-handle labels, the presence of live regions, and axe. There is no keyboard-drag test. The `onDragCancel` gap described above is a direct consequence: nothing exercised Escape, so nothing caught it. Write this test.

What you have now

A drag-and-drop board that is fully operable from the keyboard, announces its state, and reports moves rather than owning them.

The general lesson is not about boards. Any interface whose primary interaction is a gesture, a sortable list, a resizable panel, a range slider with two handles, needs the same three things: a keyboard path to the same outcome, a live region describing what changed, and a controlled API so the consumer can reject the change. Retrofitting those onto a working mouse implementation is substantially harder than building them in, which is why so few implementations have them.

One chapter left, and it is about what you do after the code is finished.

CHAPTER 17

What comes next

This chapter covers what to build next, how to keep design and code aligned, and what separates a component library from a design system.



You have a working component library: a three-tier token system, eight themes, layout primitives, typography, forms, overlays, navigation, charts, a drag-and-drop board, Storybook documentation, and a test suite. It builds, it publishes, and someone can install it.

That is the end of the guide and roughly the beginning of the work. This chapter covers what usually comes next.

Where the gaps are

The most useful thing you can do at this point is stop adding components and go find out what is missing.

The example pages from chapter 10 are the instrument. Build a real page for a domain you have not covered, and record every time you reach for a raw element, write a one-off style rule, or work around a component's API. That list is your backlog, ordered by something better than intuition.

Done honestly across four domains, the same gaps show up repeatedly.

A data table. This is consistently the largest one, and it is not close. Almost every application domain needs sortable columns, aligned cells, row selection, and horizontal scroll on small screens. It is also the single most complex component in any library, which is why it tends to be deferred past the point where the deferral is defensible. React Aria's `useTable` family gives you the accessibility layer, which removes the hardest part but not the work.

A toast or notification system. Timed, non-blocking feedback. The interesting part is not the visual component, which is close to the Alert you already built. It is the queue: a store, a portal, stacking behavior, timers that pause on hover, and a live region that announces without interrupting. That is a system, not a component.

A popover. A floating panel triggered by a button, without a modal's focus trap. `usePopover` handles positioning and dismissal, and you already used it inside Select in chapter 8, so most of the pattern is familiar.

An accordion. Collapsible sections. React Aria does not expose a `useAccordion` hook; the equivalent is `Disclosure` and `DisclosureGroup` from `react-aria-components`, which is already a dependency after chapter 14's Menu.

A combobox. Select with typeahead filtering. `useComboBox` exists, and the component is genuinely harder than Select because the input is simultaneously a text field and a collection trigger.

Add these in the order your own usage demands, not in the order a competitor's documentation lists them. Every component is a permanent maintenance and documentation obligation, and a library of forty components where twelve are unloved is worse than a library of twenty-five that are all solid.

Keeping Figma and code aligned

Design-code drift is the most common friction point in design system adoption, and it is a naming problem before it is a tooling problem.

Figma Variables map onto the token architecture directly. Create collections mirroring your tiers: a primitives collection with the raw palette and scales, a semantic collection of aliases, and optionally a component collection. Name the variables exactly as you named the tokens, so `color/brand/primary` in Figma is `color.brand.primary` in JSON. Shared names mean a conversation about `brand.primary` refers to the same thing in both places, which is most of the value before any sync exists.

Tokens Studio, a free Figma plugin, can read token JSON and sync values into Figma. Point it at your token files and it keeps Figma's values current. Run the sync in one direction, code to design: the code is what ships, so the code is the source of truth, and a bidirectional sync mostly produces conflicts nobody wants to arbitrate.

One practical caveat. This library's tokens are split across `tokens.json`, `semantic.json`, `components.json`, and six theme override directories. Plugin setups that assume a single token file need configuring for that structure, and the theme overrides in particular need explicit mapping onto Figma's mode system.

Figma Code Connect maps Figma component properties to code props, so a designer inspecting a component sees the real import statement and prop names rather than a CSS dump. It is worth the setup cost on teams with both designers and developers, and hard to justify on teams without.

None of this is set up in the companion repository. Treat this section as a direction, not a description.

Governance, and the word "design system"

A component library is code. A design system is code plus the operating model around it: contribution guidelines, a decision process, a release cadence, support channels, adoption metrics, and someone whose job includes its evolution.

The library you have built is a component library. It becomes a design system when an organization adopts it as the standard and invests in keeping it useful. That transition is organizational rather than technical, and the code cannot make it happen. It can make it easier or harder.

Three artifacts do most of that work, and all three are cheap now and expensive later.

A `CONTRIBUTING.md` stating what a complete contribution includes: the component, its stylesheet, its stories with descriptions and categories, its tests, its tokens, and a changelog entry. Without it, every pull request negotiates the standard from scratch.

A `CHANGELOG.md` maintained per pull request, as chapter 12 described. Assembled at release time from commit messages, breaking changes get missed.

A written record of decisions. Why React Aria over Radix. Why the published package excludes Tailwind. Why component tokens are not optional. These get re-litigated every time someone new joins, and a paragraph written once ends the argument faster than reconstructing the reasoning under pressure.

> **NOTE**
> One warning from watching this go wrong. Documentation drifts from code faster than anyone expects, and stale documentation is worse than none, because people trust it. The companion repository's own README currently describes components under names that no longer exist and lists example pages that were removed two releases ago. That is not unusual; it is the normal state of a README nobody checks. Whatever you write, put something in the release process that verifies it.

If you want the longer version of this, "Design Systems Are Easy Until You Ship One" covers the governance, adoption, and support side in detail.

What you have now

A complete component library. Three tiers of design tokens compiled by Style Dictionary into eight themes. Eight layout primitives that respond to available space

rather than viewport width. Three typography components. Accessible forms, overlays, navigation, and status components built on React Aria. Theme-aware charts. A keyboard-operable drag-and-drop board. Storybook documentation with generated pages, example compositions, and an accessibility audit on every story. A test suite spanning unit tests and browser-rendered stories. A published package with correct externals and type declarations.

More than the inventory, you have the reasons. Why tokens have three tiers, why layout belongs in the library, why the package does not ship a CSS reset, why some components use React Aria and some do not, why a chart needs a `MutationObserver`, why a drag interaction needs a live region.

Where to go from here

The specific components and token values will change as you use this. That is the point. The architecture absorbs those changes without requiring you to rethink the foundation: tokens define values, components consume tokens, layout handles arrangement, tests verify behavior, and each layer moves without the others.

The parts that should not change are the constraints. Every component reads tokens rather than literals. Every interactive component is operable from the keyboard. Every visual state has a non-color signal. Every published export is one you are willing to support. Those hold whether the library has twenty-five components or a hundred and twenty.

If building this from scratch is not the right investment for your team, Rudiment UI is a pre-built version of this architecture, ready to customize and extend. And the companion repository is the complete implementation of everything in this guide, available to read, clone, or take issue with.

Index

Page numbers refer to discussion in the prose. Code listings are not indexed separately.

A

accessibility, **179**. *See also axe-core, React Aria, WAI-ARIA*
  announcements, 81, 101, 111–113, 181–182
  focus management, 6, 82, 89, 96, 106, 108, 133, 188
  keyboard navigation, 6, 82, 96, 178
  live regions, 110, 180–181, 184–185, 188, 191
  screen readers, 6, 67, 88, 102, 111, 133, 163, 166, 175, 180
Alert, 3, 105–106, **110**, 114, 188
ARIA attributes, 6, 78, 82, 87–89, 96, 98, 106, 112–113, 133, 136–137, 162–164, 166–167, 175, 180. *See also WAI-ARIA*
Avatar, 164
axe-core, 4, 44, 132–134, 137

B

Badge, 3, 105–106, **112**, 114, 164
Box, 3, 8, 25, **46**, 47–48
Breadcrumb, 165
Button, **79**, **82**. *See also IconButton*
  icon slots, 78, 81, 89, 136
  loading state, 21, 82

C

Card, 3, 38, 105–106, **113**, 114, 179
Center, 3, 8, **48**, 49
charts, 123–124, 170, 173–174, 176. *See also data visualization, Recharts*
Checkbox, 3, 91–92, **97**, 99, 101–103, 114
Cluster, 3, 8, **50**, 51
colocated stylesheets, 9, 14, 143
Cover, 3, 8, **58**, 59
CSS custom properties, 2, 7, 9, 18, 26, 28, 35–36, 44, 46, 56, 62, 66, 114, 126, 133–134, 137, 148, 170. *See also design tokens*
CSS Modules, 7

D

data visualization, 4, 21, 146, 152, **169**, 170, 173. *See also charts*
data-theme attribute, 4, 121, 126, 148–149, 151, 154, 171–172, 176. *See also theming*
design tokens. *See also Style Dictionary*
  component tier, 8, 18, 23, 82, 153, 173
  DTCG format, 8–9, 18
  global tier, 8, 84, 151
  outputReferences, 26, 155
  semantic tier, 8, 47, 84
  token prefix, 9, 26, 28, 34, 36, 46, 66, 69, 74–75, 84, 122, 145, 148, 151–152, 154–155, 170, 173
Dialog, 3, 105, **106**, 108, 114. *See also overlays*
Divider, 160, 167
dnd-kit, 4, 141, 178, 180–181, 183. *See also drag and drop*
drag and drop, 4, 146, **177**, 178–180, 184. *See also dnd-kit, Kanban board*

E

Every Layout, 9, 56, 61. *See also layout primitives*
exports map, 38, 140, **142**, 143–144, 146, 160. *See also packaging*

F

Figma, 4, 8, **189**
  Code Connect, 189
focus ring, 8, 23, 82
forwardRef, 34, 48, 79, 81, 98, 108, 110, 114. *See also polymorphic components*

G

governance, 4, **190**
Grid, 3, 8, **56**, 58, 61

H

Heading, 3, 62, **63**, 66, **67**, 69, 71–72, 76, 123. *See also typography*

HeroUI, 2, 6, 9

I

Icon, 78, 89, 141, 162, 166

IconButton, 3, 77–79, **87**, 89, 103, 112, 114, 163

Image, 163

Input, **85**

intrinsic design, 2, 8–9, 53, 61. *See also layout primitives*

K

Kanban board, 4, 177–178, 184. *See also drag and drop*

L

layout primitives, **31**. *See also Every Layout*
- Sidebar, 3, 8–9, **52**, 53–54, 61, 123, 134
- Stack, **31**
- Switcher, 3, 8, 46, **54**, 56, 61, 134

library mode, 4, **140**. *See also packaging, Vite*

Link, 162

M

Menu, 4, 78, 138, **160**, 188

N

NavItem, 166

npm publishing, 4, 143–144, 146, 170. *See also packaging, versioning*

O

Option, 97, 104. *See also Select*

overlays, 23, 106, 108, 114, 121, 181, **183**, 188. *See also Dialog, Tooltip*

P

packaging, 4, 7, **139**, 143–144, 161. *See also exports map, library mode*

Pagination, 166

Playwright, 118

polymorphic components, 34–35, 48. *See also forwardRef*

preflight, 7, 9, 27, 75. *See also Tailwind CSS*

ProgressBar, 164

Prose, 3, 62, **63**, 67, **72**, **74**, 75–76. *See also typography*

R

RadioGroup, 3, 91–92, **99**, 101, 103–104, 114

Radix, 6, 190

React Aria, **77–78**. *See also accessibility, React Stately*

React Stately, 78, 92, 96–99, 141

Recharts, 141, 170–171, 174–176. *See also charts*

S

Select, 3, 91, **92**, 96–97, 103–104, 114, 123, 138, 188

SemVer, 145. *See also versioning*

StatCard, 165

Stepper, 166

Storybook, **117**, **126**
- argTypes, 41
- decorators, **121–122**, 156
- example pages, 14
- toolbar, 122, 153, **155**

Style Dictionary, **18**, **25**. *See also design tokens*

styled-components, 7

Switch, 3, 38, 91–92, 97, **102**, 103, 114

T

Tailwind CSS, 2–3, 7, 9, 11–12, 14–15, 27, 75, 122, 190. *See also preflight*

testing, 41, **129**, 130, **133–134**, 136, **184**, 188, 191. *See also axe-core, Vitest*
- coverage targets, **137**
- Testing Library, 4, 9, 14

Text, 3, 38, 62, **63**, 67–68, **69**, 72, 76, 123, **162**. *See also typography*

Textarea, 104, 162

theming, 4, 26, 123, 146, **147**, 156, 189. *See also data-theme attribute*
- adding a theme, **153**
- composing themes, **152**
- specificity trap, 15, **154**

tokens. *See design tokens*

Tooltip, 3, 105–106, **108**, 110, 114
TypeScript, **12**
  declaration files, 140–141
  generics, 35, 180
typography, **63–64**. *See also Heading, Prose, Text*

U

Untitled UI React, 2, 6, 9
use client directive, 142, 146

V

versioning, 4, **144**, 145. *See also SemVer*
Vite, 3–4, 9, 11–12, **13**, 15, 118, **140**, 143–144. *See also library mode*
Vitest, 4, 9, 13–14, 118, 130, 132. *See also testing*

W

WAI-ARIA, 6. *See also ARIA attributes*

Every team eventually builds a component library. *Most of them build it twice.*

The first version starts with a Button. Six months later there are forty components, three competing spacing scales, a dark mode half of them ignore, and a modal that traps focus incorrectly. Nobody wants to touch it, so someone proposes a rewrite.

This book is the second version, written down before you build the first one. Seventeen chapters take a React component library from an empty directory to a published npm package, and argue for every decision along the way. **Chapter 1 contains no code at all**, because the expensive mistakes are architectural and they compound.

WHAT YOU BUILD

- A **three-tier token system** compiled with Style Dictionary into namespaced custom properties
- **Eight themes from one pipeline**, switched by a single attribute, with no component changes and no theme context
- Eight intrinsic **layout primitives** and typography that holds its rhythm over long-form content
- **Accessible components on React Aria**: dialogs, selects, and forms with real keyboard paths and focus management
- Charts that read your tokens, and a **drag-and-drop board that works without a mouse**
- Storybook documentation, an **axe-core test suite**, and a versioned, publishable package

---

Assumes you are comfortable with React, TypeScript, and CSS. Assumes nothing about Tailwind, Storybook, design tokens, or accessibility tooling — each arrives when it is needed.

Joshua Briley is a product design engineer with twenty years in production UI, most of it design systems and data-dense product work for insurance platforms and consumer products. Rudiment UI, the library this book builds toward, is an open-source project. joshuabriley.com



rudiment-guide.netlify.app

Read the web edition · the code is on GitHub

JOSHUA BRILEY

208 PP. · FIRST EDITION · 2026